© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

LFD259-JP

# Kubernetes アプリ開発

Version 2023-10-10

THE LINUX FOUNDATION | Training & Certification

Version 2023-10-10

© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

The Linux Foundation により開発および提供されているトレーニング資料は、著作権および知的財産権により保護されています。

本書に記載されているオープンソースコードは他の著作権者が存在する可能性があり、該当のオープンソースライセンスに基づき使用されています。

トレーニング資料は受講者の個人利用のために提供されます。The Linux Foundation およびクリエーションライン株式会社の許可なしでの複製、修正、受講者以外の人への再配布、または他者へのトレーニング供与目的での使用を禁じます。

本書のいかなる部分も、事前の書面による許可なしでの複製、複写、送信、情報検索システムへの保存を禁じます。

発行元

The Linux Foundation
https://www.linuxfoundation.org

No representations or warranties are made with respect to the contents or use of this material, and any express or implied warranties of merchantability or fitness for any particular purpose or specifically disclaimed.

Although third-party application software packages may be referenced herein, this is for demonstration purposes only and shall not constitute an endorsement of any of these software applications.

**Linux** is a registered trademark of Linus Torvalds. Other trademarks within this course material are the property of their respective owners.

If there are any questions about proper and fair use of the material herein, please contact https://trainingsupport.linuxfoundation.org.


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# Contents


2 **Kubernetes** アーキテクチャ 2-1

3 ビルド 3-1

4 デザイン 4-1

5 **Deployment** 設定 5-1

6 セキュリティの理解 6-1

7 アプリケーションの公開 7-1

8 アプリケーションのトラブルシューティング 8-1

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 2 章

# Kubernetes アーキテクチャ

## 課題 2.1: 概要と前置き

2 ノードの **Ubuntu 20.04** クラスターを作成します。2 つのノードを使用することで、本番環境で見られるいくつかの問題や構成を理解できます。今回は、2 つの vCPU と 8G のメモリで、演習を行います。イメージを扱うので、コントロールプレーン（cp）ノードに少なくとも **20G** のディスクスペースを割り当てることが推奨されます（多いほど良い）。他の Linux ディストリビューションも同様に動作するはずですが、テストはしていません。

サマリ

- 2 つの CPU
- 8G メモリ
- コントロールプレーンノードに 20GB 以上のディスクスペース
- ノードに 192.168 のネットワークを使用しない
- Firewall の無効化
- SWAP の無効化
- SELinux と AppArmor の無効化

**重要**

使用するプラットフォーム（**VirtualBox**、**VMWare**、**AWS**、**GCE**、ベアメタル）に関わらず、**SELinux**、**AppArmor**、ファイアウォール設定などのセキュリティソフトウェアによって演習が動作しないことがあることをご留意ください。商用環境で行うべきことではありませんが、ファイアウォールやセキュリティソフトウェアを無効にすることを検討してください。

GCE では、新しい `VPC` を作成し、すべてのトラフィックを通すことを許可するルールが必要です。すべてのポートを開くことができない場合、ネットワークと接続性の問題のトラブルシューティングを開始するには、**wireshark** を使用することが有用です。

**kubeadm** ユーティリティは現在、すべてのノードで swap をオフにすることを要求しています。**swapoff -a** コマンドは次の再起動が行われるまでの間、swap をオフにします。永続的に swap をオフにする方法はいくつかあります。クラウドプロバイダは swap をオフにした状態でインスタンスのデプロイを行う場合が多いです。

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


**シェルスクリプトと YAML ファイルのダウンロード**

クラスターのセットアップを支援するために、シェルスクリプトと YAML ファイルの tarball をダウンロードしてください。`k8scp.sh`と`k8sWorker.sh`は、**kubeadm** を使用して Kubernetes クラスターをデプロイし、ネットワークに `Project Calico` を使用するスクリプトです。ファイルが見つからない場合は、ブラウザを使って親ディレクトリを探してみてください。

```
$ wget https://training.linuxfoundation.org/cm/LFD259-JP/LFD259-JP_V2023-10-10_SOLUTIONS.tar.xz \
              --user=LFtraining --password=Penguin2014

$ tar -xvf LFD259-JP_V2023-10-10_SOLUTIONS.tar.xz
```

（注意: お使いのソフトウェアによっては、上記の指示をカット＆ペーストすると、アンダースコアが消えてスペースに置き換わることがありますので、コマンドラインを手で編集する必要があるかもしれません!）

## 課題 2.2: 新しいクラスターをデプロイする

### Kubeadm を使用してコントロールプレーンノードをデプロイする

1. **PuTTY** を使用するか、ターミナルウィンドウから **SSH** を使用してノードにログインします。講師の指示がない限り、使用するユーザー名は **student** になります。以下のコマンドに示すように、`pem`（Windows の場合は`ppk`）ファイルのパーミッションを変更する必要がある場合があります。ファイル名とノードの IP アドレスはおそらく異なるでしょう。

   PuTTY を使用する場合は、`ppk` キーを使用してインスタンスにアクセスする手順を使ってください。PuTTY で聞かれた場合は、ユーザー名 **student** を使用します。

   ```
   localTerm:~$ chmod 400 LF-Class.pem
   localTerm:~$ ssh -i LF-Class.pem student@WW.XX.YY.ZZ
   ```

   ```
   student@cp:~$ ^^I^^I
   ^^I
   ```

2. 上記の **wget** と **tar** コマンドを使用して、コースの tarball をダウンロードし、cp ノードに展開します。**wget** が見つからない場合は、インストールしてください。

3. 練習問題に出てくるコマンドは、コピー＆ペーストではなく、必ずタイプしてください。タイプすることでコマンドを覚えやすくなりますが、コピー＆ペーストではコマンドが覚えにくい傾向があります。また、ホワイトスペースの問題は演習の速度を低下させ、やる気を失いやすいので、コースの tarball に含まれている YAML ファイルを使用してください。

4. cp kubernetes サーバーのインストールと設定を開始するためのスクリプトを確認します。**find** コマンドが対象とする検索ディレクトリを、tarball をダウンロードした場所と方法に応じて、あなたのホームディレクトリなどに変更する必要があるかもしれません。

   ファイルを作成するのではなく、位置を特定してカレントディレクトリにコピーする場合に備えて、下記の例では **find** コマンドが表示されます。今後のコマンドでは表示されない可能性があるので、参考までにマークしておいてください。

   ```
   student@cp:~$ find $HOME -name <YAML File>
   student@cp:~$ cp LFD259-JP/<Some Path>/<YAML File> .
   ```

   ```
   student@cp:~$ find $HOME -name k8scp.sh


   student@cp:~$ more LFD259-JP/SOLUTIONS/s_02/k8scp.sh


   ....
   # Install the Kubernetes software, and lock the version
   sudo apt update
   sudo apt -y install kubelet=1.27.1-00 kubeadm=1.27.1-00 kubectl=1.27.1-00
   ```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
sudo apt-mark hold kubelet kubeadm kubectl

# Ensure Kubelet is running
sudo systemctl enable --now kubelet

# Disable swap just in case
sudo swapoff -a
....
```

5. スクリプトを **bash** シェルの引数として実行します。今後のステップでワーカー／ミニオンノードを追加する際に、スクリプトの出力の末尾付近に表示される`kubeadm join` コマンドが必要になります。スクリプトの出力で`kubeadm join` を見つける際にスクロールバックできない場合に備えて、**tee** コマンドを使用してスクリプトの出力を保存してください。以下は、1 つのコマンドとその出力であることに注意してください。

   **Ubuntu 20.04** を使用すると、インストール中に質問されることがあります。再起動を許可し（yes）、アップデート中に質問された場合は、通常（オプション 2）、ローカルにインストールされているソフトウェアを使用します。

   まずホームディレクトリにファイルをコピーします。

```
student@cp:~$ cp LFD259-JP/SOLUTIONS/s_02/k8scp.sh .

student@cp:~$ bash k8scp.sh | tee $HOME/cp.out
```

```
<output_omitted>

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

Then you can join any number of worker nodes by running the following on each as root:

kubeadm join 10.128.0.93:6443 --token zu7icq.2o0ouw7s044yi9w9 \
^^I--discovery-token-ca-cert-hash sha256:a26f2166ad39b6aba0fc0dd663ac2b3455af19526bd9fe80af5439255dd
<output_omitted>

NAME        STATUS     ROLES           AGE       VERSION
cp          NotReady   control-plane   20s       v1.27.1


Script finished. Move to the next step
```

## ワーカーノードをデプロイする

6. ワーカーとなる **2** 台目のノードに、別のターミナルを開きます。両方のターミナルセッションを持つことで、2 番目のノードを追加している間、クラスターのステータスを監視できます。2 台目のノードの色やその他の特徴を変更するターミナルを作成し、最初のものと視覚的に区別できるようにします。これにより、間違ったインスタンスでコマンドを実行することを防げます。

   先ほどの **wget** コマンドで、ワーカーノードに tarball をダウンロードします。前回と同様に **tar** でファイルを解凍します。`k8sWorker.sh`ファイルを探し、`student` のホームディレクトリにコピーし、それを表示します。cp セットアップスクリプトと同じ初期ステップが表示されるはずです。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止 )

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@worker:~$ more k8sWorker.sh


....
# Install the Kubernetes software, and lock the version
sudo apt update
sudo apt -y install kubelet=1.27.1-00 kubeadm=1.27.1-00 kubectl=1.27.1-00
sudo apt-mark hold kubelet kubeadm kubectl

# Ensure Kubelet is running
sudo systemctl enable --now kubelet

# Disable swap just in case
sudo swapoff -a
....
```

7. **2** 台目のノードでスクリプトを実行します。更新中に質問が出るかもしれないので、再度注意してください。デーモンの再起動を許可し、yes と入力し、ローカルにインストールされたバージョン（通常はオプション 2）を使用します。トラブルシューティングのために、**tee** コマンドを使ってスクリプトの出力をファイルに書き出すことが有効な場合もあります。

```
student@worker:~$ bash k8sWorker.sh | tee worker.out
```

```
<output_omitted>
```

8. スクリプトが完了すると、ワーカーノードがクラスターに参加する準備が整います。kubeadm join ステートメントは、cp ノードの kubeadm init 出力の終わり近くにあります。また、cp.out ファイルにも記載されているはずです。あなたのノードは、以下の例とは異なる IP アドレスとハッシュを使用します。cp ノードからコピーしたスクリプトを実行するために、事前に **sudo** を実行する必要があります。また、Linux 以外の OS やツールでは、複数行のサンプルをコピー＆ペーストすると余計な文字が挿入されることがあります。一度に 1 行ずつコピーすることで、この問題は回避できます。コントロールプレーンノード上で **grep -A2 join cp.out** コマンドを実行することで、ファイル内から kubeadm join の行を探し出せることでしょう。

```
student@worker:~$ sudo kubeadm join --token 118c3e.83b49999dc5dc034 \
 10.128.0.3:6443 --discovery-token-ca-cert-hash \
sha256:40aa946e3f53e38271bae24723866f56c86d77efb49aedeb8a70cc189bfe2e1d
```

```
<output_omitted>
```

### コントロールプレーンノードを設定する

9. cp ノードに戻ります。テキストエディターをインストールします。この演習では **vim** を使用していますが、**emacs** や **nano** など、どのテキストエディターでもかまいません。Windows のエディターでは、特殊文字に問題がある場合がありますのでご注意ください。まだ **bash-completion** がインストールされていない場合は、そのパッケージもインストールします。ローカルにインストールされているパッケージのバージョンを使用するように指示された場合は、そのパッケージを使用します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
student@cp:~$ sudo apt-get install bash-completion vim -y
```

```
<output_omitted>^^I
^^I
```

10. コマンドライン補完を設定し、両方のノードがクラスターに追加されたことを確認します。最初のコマンドは、現在のシェルで補完を設定します。2 番目のコマンドは、今後のシェルで補完が行われるようにするためのコマンドです。コマンド補完がエラーなく動作するように、シェルを終了してログインし直す必要がある場合があります。

```
student@cp:~$ source <(kubectl completion bash)
```

```
student@cp:~$ echo "source <(kubectl completion bash)" >> $HOME/.bashrc
```

11. 両方のノードがクラスターの一部になっていることを確認します。そして、STATUS が`Ready` になっていることを表示します。

```
student@cp:~$ kubectl get node
```

```
NAME     STATUS   ROLES           AGE       VERSION
cp       Ready    control-plane   4m11s     v1.27.1
worker   Ready    <none>          61s       v1.27.1
```

12. Kubernetes を使った作業の大半は、**kubectl** コマンドを使用して行うことになります。コマンドのオプションと引数に慣れるために、ヘルプの出力を確認してください。

```
student@cp:~$ kubectl --help
```

```
kubectl controls the Kubernetes cluster manager.

Find more information at:
  https://kubernetes.io/docs/reference/kubectl/overview/

Basic Commands (Beginner):
  create         Create a resource from a file or from stdin.
  expose         Take a replication controller, service,
 deployment or pod and expose it as a new Kubernetes Service
  run            Run a particular image on the cluster
  set            Set specific features on objects

Basic Commands (Intermediate):
<output_omitted>
```

13. 40 以上の引数がありますが、`--help` オプションで各引数を調べることができます。ここでは、`Taint` を例にとって、いくつかの引数を詳しく見てみましょう。

```
student@cp:~$ kubectl taint --help
```

```
Update the taints on one or more nodes.

  * A taint consists of a key, value, and effect. As an argument
   here, it is expressed as key=value:effect.
  * The key must begin with a letter or number, and may contain
   letters, numbers, hyphens, dots, and underscores, up to
   253 characters.
  * Optionally, the key can begin with a DNS subdomain prefix
   and a single '/',
like example.com/my-app
<output_omitted>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


14. デフォルトでは、cp ノードはセキュリティ上の理由から一般的なコンテナをデプロイすることを許可しません。これはTaint を介したものです。この Taint を許容するコンテナだけが、このノードへスケジュールされます。クラスターには 2 つのノードしかないので、Taint を削除し、両方のノードでコンテナをデプロイできるようにします。これは、セキュリティとリソース競合の理由から、通常、商用環境では行われません。次のコマンドは、すべてのノードから Taint を削除するため、1 つの成功と 1 つのnot found エラーが表示されるはずです。ワーカー／ミニオンノードにはもともと Taint がありません。コマンドの最後にあるマイナス記号に注意してください。これは先行する値を削除するものです。

```
student@cp:~$  kubectl describe nodes | grep -i taint
```

```
Taints:             node-role.kubernetes.io/control-plane:NoSchedule
Taints:             <node>
```

```
student@cp:~$ kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

```
node/cp untainted
error: taint "node-role.kubernetes.io/control-plane" not found
```

15. 両方のノードにTaint がないことを確認します。両方のノードに Taint がない場合、ノードはReady と表示されるはずです。すべてのインフラストラクチャ Pod がReady 状態になり、ノードがReady 状態を示すようになるまで 1〜2 分かかる場合があります。

```
student@cp:~$ kubectl describe nodes | grep -i taint
```

```
Taints:           <none>
Taints:           <none>
```

```
student@cp:~$ kubectl get nodes
```

```
NAME     STATUS   ROLES           AGE     VERSION
cp       Ready    control-plane   6m1s    v1.27.1
worker   Ready    <none>          5m31s   v1.27.1
```

## 課題 2.3: 基本的な Pod を作成する

1. Kubernetes で直接制御する最小単位が Pod です。最小限の YAML ファイルを作成することで、Pod を作成していきます。まず、現在の API オブジェクトとそのAPIGROUP のリストを取得します。値が表示されない場合は、SHORTNAMES と同様、存在しない可能性があります。Pods はAPIGROUP を宣言していないことに注意してください。現時点では、これは安定したv1 API グループの一部であることを示しています。

```
student@cp:~$ kubectl api-resources
```

```
NAME                SHORTNAMES   APIVERSION   NAMESPACED   KIND
bindings                         v1           true         Binding
componentstatuses   cs           v1           false        ComponentStatus
configmaps          cm           v1           true         ConfigMap
endpoints           ep           v1           true         Endpoints
.....
pods                po           v1           true         Pod
....
```

2. 出力から、ほとんどが安定したオブジェクトを示す v1 であることがわかります。この情報をもとに、Pod に必要な他の 3 つのセクション、たとえば metadata、name、そして使用するコンテナイメージとコンテナの名前を宣言するspec を追加します。8 行の YAML ファイルを作成することになります。ホワイトスペースとインデントが重要であり、タブは使わないでください。tarball にはbasic.yamlと、このファイルがどのようになるかを示すbasic-later.yamlがあります。インデントを把握するのに役立つことでしょう。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



basic.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: basicpod
spec:
  containers:
  - name: webcont
    image: nginx
```

3. 作成した YAML ファイルを使って、新しい Pod を作成します。

```
student@cp:~$ kubectl create -f basic.yaml
```

```
pod/basicpod created
```

4. Pod が作成されたことを確認し、**describe** サブコマンドを使用して詳細を表示します。出力される他の値の中に、イメージとコンテナ名を見つけることができるはずです。

```
student@cp:~$ kubectl get pod
```

```
NAME      READY   STATUS    RESTARTS   AGE
basicpod  1/1     Running   0          23s
```

```
student@cp:~$ kubectl describe pod basicpod
```

```
Name:               basicpod
Namespace:          default
Priority:           0
<output_omitted>
```

5. Pod をシャットダウンして、実行されなくなったことを確認します。

```
student@cp:~$ kubectl delete pod basicpod
```

```
pod "basicpod" deleted
```

```
student@cp:~$ kubectl get pod
```

```
No resources found in default namespace.
```

6. ここで、ポート 80 を公開するように Pod を設定します。この設定では、どのポートを開くかを決定するためにコンテナと対話することはありません。コンテナ内のプロセスが使用しているポート、この場合はウェブサーバーとしてのポート 80 を知る必要があります。ファイルの末尾に`image` 定義のインデントに合わせる形で `ports` の 2 行追加します。

```
student@cp:~$ vim basic.yaml
```

basic.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: basicpod
spec:
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```yaml
  containers:
  - name: webcont
    image: nginx
    ports:                      #<--Add this and following line
    - containerPort: 80
```

7. Pod を作成し、実行されていることを確認します。`-o wide` オプションを使用すると、Pod に割り当てられた内部 IP、スケジューラが使用する`NOMINATED NODE`、実験的な機能が有効かどうかを示す`READINESS GATES` を確認できます。**curl** と Pod の IP アドレスを使用すると、デフォルトの nginx のウェルカム Web ページが表示されるはずです。

```
student@cp:~$ kubectl create -f basic.yaml
```

```
pod/basicpod created
```

```
student@cp:~$ kubectl get pod -o wide
```

```
NAME     READY STATUS  RESTARTS AGE  IP           NODE
  NOMINATED NODE  READINESS GATES
basicpod 1/1   Running 0        9s   192.168.1.3  cp
  <none>          <none>
```

```
student@cp:~$ curl http://192.168.1.3
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>

<output_omitted>
```

```
student@cp:~$ kubectl delete pod basicpod
```

```
pod "basicpod" deleted
```

8. ここで、クラスター内の他のノードや他の Pod に、Pod を公開するためのシンプルな Service を作成します。`Service` の YAML は Pod と同じ 4 つのセクションを持ちますが、異なる`spec` の設定と`selector` の追加を行います。ここでも、ファイルを手入力する代わりに、tarball から例をコピーしてください。

```
student@cp:~$ vim basicservice.yaml
```

basicservice.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: basicservice
spec:
  selector:
    type: webserver
  ports:
  - protocol: TCP
    port: 80
```

9. また、Pod に`label` を追加し、Service に`selector` を追加して、どのオブジェクトと通信すればよいかを知ることができるようにします。

```
student@cp:~$ vim basic.yaml
```

basic.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: basicpod
  labels:                    #<-- Add this line
    type: webserver          #<-- and this line which matches selector
spec:
....
```

10. 新しい Pod と Service を作成します。両方が作成されたことを確認します。出力の詳細については、後の章で学びます。

```
student@cp:~$ kubectl create -f basic.yaml
```

```
pod/basicpod created
```

```
student@cp:~$ kubectl create -f basicservice.yaml
```

```
service/basicservice created
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~$ kubectl get pod
```

```
NAME       READY   STATUS    RESTARTS   AGE
basicpod   1/1     Running   0          110s
```

```
student@cp:~$ kubectl get svc
```

```
NAME           TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)     AGE
basicservice   ClusterIP   10.96.112.50    <none>       80/TCP      14s
kubernetes     ClusterIP   10.96.0.1       <none>       443/TCP     4h
```

11. basicservice Service のCLUSTER-IP を使用して、Web サーバーへのアクセスをテストします。

```
student@cp:~$ curl http://10.96.112.50
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>

<output_omitted>
```

12. これから、この Service をクラスターの外にも公開することにします。Service を削除し、ファイルを編集してtype 宣言を追加します。

```
student@cp:~$ kubectl delete svc basicservice
```

```
service "basicservice" deleted
```

```
student@cp:~$  vim basicservice.yaml
```

basicservice.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
    name: basicservice
spec:
  selector:
    type: webserver
  type: NodePort       #<--Add this line
  ports:
  - protocol: TCP
    port: 80
```

13. Service を再度作成します。TYPE とCLUSTER-IP が異なり、高番号のポートもあることに注意してください。

```
student@cp:~$  kubectl create -f basicservice.yaml
```

```
service/basicservice created
```

```
student@cp:~$ kubectl get svc
```

```
NAME          TYPE      CLUSTER-IP     EXTERNAL-IP PORT(S)        AGE
basicservice  NodePort  10.100.139.155 <none>      80:31514/TCP   3s
kubernetes    ClusterIP 10.96.0.1      <none>      443/TCP        47h
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



14. ノードのパブリック IP アドレスとハイポートを使用して、Web サーバーへのアクセスをテストできるはずです。以下の例では、パブリック IP は ifconfig.ioへの **curl** によって表示されている 35.238.3.83 になります。環境によって IP は異なるでしょう。また、ハイポートもおそらく異なるはずです。GCE や AWS のノードからテストしても正常に動作しないことに注意してください。テストには、ローカルのターミナルやウェブブラウザを使用してください。

```
student@cp:~$ curl ifconfig.io
```

```
35.238.3.83
```

```
local$ curl http://35.238.3.83:31514
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
<output_omitted>
```

## 課題 2.4: マルチコンテナ Pod

1 つの Pod に 1 つのコンテナを使用することで、最もきめ細かく、デカップリングが可能になります。一方で 1 つの Pod に複数のコンテナ（コンポジットコンテナと呼ばれることもある）をデプロイする理由もまだあります。セカンダリコンテナはプライマリコンテナのログを処理したり強化したりするサイドカーコンセプトであったり、外部へのプロキシとして機能するアンバサダーコンセプト、外部フォーマットに合わせてデータを修正するアダプターコンセプトなどがその理由です。3 つのコンセプトはすべて、プライマリコンテナにはない機能を実行するためのセカンダリコンテナです。

1. ロギングを処理するために、Pod に 2 つ目のコンテナを追加します。**fluentd** の使い方の詳細は省きますが、既存の Pod に独自のリポジトリからロギングコンテナを追加していきます。2 つ目のコンテナはサイドカーとして機能することになります。この状態では、2 つ目のコンテナを追加して、それが実行されていることを確認するだけです。**Deployment** 設定の章では、この Pod に対して作業を続け、永続的なストレージの追加、configMap を使って **fluentd** を設定します。

   YAML ファイルを編集し、**fluentd** コンテナを追加します。追加するコンテナの前につけるダッシュ（-）は前のコンテナのダッシュと並ぶはずです。この時点で、2 つ目のコンテナを起動するには、名前とイメージで十分なはずです。

```
student@cp:~$ vim basic.yaml
```

basic.yaml

```yaml
....
  containers:
  - name: webcont
    image: nginx
    ports:
    - containerPort: 80
  - name: fdlogger
    image: fluent/fluentd
```

2. 削除して、再度 Pod を作成します。コマンドはセミコロンで区切って 1 行で打てます。今回はREADY の欄に2/2 と表示されているはずです。また、kubectl describe の出力内に **fluentd** コンテナの情報があるはずです。

```
student@cp:~$ kubectl delete pod basicpod ; kubectl create -f basic.yaml
```

```
pod "basicpod" deleted
pod/basicpod created
```

```
student@cp:~$ kubectl get pod
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
NAME       READY   STATUS    RESTARTS   AGE
basicpod   2/2     Running   0          2m8s
```

```
student@cp:~$ kubectl describe pod basicpod
```

```
Name:               basicpod
Namespace:          default
Priority:           0
Node:               cp/10.128.0.11

....

  fdlogger:
    Container ID:   docker://f0649457217f00175ce9aec35022d0b238b9b....
    Image:          fluent/fluentd

....
```

3. とりあえず、Pod をシャットダウンします。今後の演習で再び使用します。

```
student@cp:~$ kubectl delete pod basicpod
```

```
pod "basicpod" deleted
```

## 課題 2.5: シンプルな Deployment を作成する

Pod を作成しても、Kubernetes のオーケストレーション能力は活かせません。ここでは、`Deployment` を作成し、拡張性、信頼性、更新性を実現しています。

1. ここで、コンテナ化されたウェブサーバー **nginx** を実行します。**kubectl create** を使用して、nginx ウェブサーバーを実行するシンプルな単一レプリカの Deployment を作成します。前回行ったように単一の Pod が作成されますが、他の機能と同様に実行を保証するための新しいコントローラーが追加されます。

```
student@cp:~$  kubectl create deployment firstpod --image=nginx
```

```
deployment.apps/firstpod created
```

2. 新しい Deployment が存在し、希望する Pod 数が現在の数と一致することを確認します。カンマを使用すると、一度に 2 つのリソースタイプを要求できます。**Tab**b キーは役に立ちます。単語が一意になるところまで入力したところで、**Tab** キーを押すと、単語が補完されます。Deployment では、希望する Pod 数と実行中の Pod 数が一致するように、各値に1 が表示される必要があります。Pod には再起動がゼロであることが表示されるはずです。

```
student@cp:~$ kubectl get deployment,pod
```

```
NAME                        READY UP-TO-DATE AVAILABLE AGE
deployment.apps/firstpod    1/1   1          1         2m42s

NAME                            READY STATUS  RESTARTS AGE
pod/firstpod-7d88d7b6cf-lrsbk   1/1   Running 0        2m42s
```

3. Deployment の詳細を表示し、次に Pod を表示します。コマンドの出力を確認してください。健全な Deployment がどのようなものかを知っておくと、問題のトラブルシューティングの際に役に立ちます。自動生成された長いオブジェクト名を使用する場合は、再び **Tab** キーを使用すると便利です。Pod を表示するときに、firstpod**Tab** と入力すれば、その名前が完成します。

```
student@cp:~$ kubectl describe deployment firstpod
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
Name:                   firstpod
Namespace:              default
CreationTimestamp:      Tue, 21 Sep 2022 16:36:29 +0000
Labels:                 app=firstpod
Annotations:            deployment.kubernetes.io/revision=1
Selector:               app=firstpod
Replicas:               1 desired | 1 updated | 1 total | 1 available....
StrategyType:           RollingUpdate
MinReadySeconds:        0
<output_omitted>
```

```
student@cp:~$ kubectl describe pod firstpod-6bb4574d94-rqk76
```

```
Name:               firstpod-6bb4574d94-rqk76
Namespace:          default
Priority:           0
PriorityClassName:  <none>
Node:               cp/10.128.0.2
Start Time:         Tue, 21 Sep 2022 16:36:29 +0000
Labels:             pod-template-hash=2660130850
                    app=firstpod
Annotations:        cni.projectcalico.org/podIP: 192.168.200.65/32
Status:             Running
IP:                 192.168.200.65
Controlled By:      ReplicaSet/firstpod-6bb4574d94

<output_omitted>
```

4. リソースは default の Namespace にあることに注意してください。利用可能な Namespace の一覧を取得します。

```
student@cp:~$ kubectl get namespaces
```

```
NAME              STATUS     AGE
default           Active     20m
kube-node-lease   Active     20m
kube-public       Active     20m
kube-system       Active     20m
```

5. 初期状態の Namespace は 4 つあります。`kube-system` Namespace 内の Pod を見てください。

```
student@cp:~$ kubectl get pod -n kube-system
```

```
NAME                                READY   STATUS    RESTARTS   AGE
calico-node-5ftrr                   2/2     Running   0          24m
calico-node-f7zrw                   2/2     Running   0          21m
coredns-fb8b8dccf-cmkds             1/1     Running   0          24m
coredns-fb8b8dccf-grltk             1/1     Running   0          24m
etcd-v141-r24p                      1/1     Running   0          23m
<output_omitted>
```

6. 今度は、存在しない Namespace の Pod の参照を試みてください。エラーが表示されないことでしょう。

```
student@cp:~$ kubectl get pod -n fakenamespace
```

```
No resources found in fakenamespaces namespace.
```

7. また、すべての Namespace のリソースを一度に表示することもできます。すべての Namespace のオブジェクトを一度に選択するには、`--all-namespaces` オプションを使用します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~$ kubectl get pod --all-namespaces
```

```
NAMESPACE     NAME                          READY   STATUS    RESTARTS   AGE
default       firstpod-69cfdfd8d9-kj6ql     1/1     Running   0          44m
kube-system   calico-node-5ftrr             2/2     Running   0          92m
kube-system   calico-node-f7zrw             2/2     Running   0          89m
kube-system   coredns-fb8b8dccf-cmkds       1/1     Running   0          92m
<output_omitted>
```

8. 複数のリソースを一度に見ることができます。なお、ほとんどのリソースは、ReplicaSet は `rs`、Pod は `po`、Service は `svc`、Endpoint は `ep` のような短い名前を持っています。Pod を削除した後も Endpoint は存在することに注意してください。

```
student@cp:~$ kubectl get deploy,rs,po,svc,ep
```

```
NAME                      READY  UP-TO-DATE  AVAILABLE  AGE
deployment.apps/firstpod  1/1    1           1          4m

NAME                                        DESIRED   CURRENT   READY....
replicaset.apps/firstpod-6bb4574d94-rqk76   1         1         1 ....

NAME                           READY  STATUS    RESTARTS   AGE
pod/firstpod-6bb4574d94-rqk76 1/1    Running   0          4m

NAME                 TYPE       CLUSTER-IP    EXTERNAL-IP PORT(S)      AGE
service/basicservice NodePort   10.108.147.76 <none>      80:31601/TCP 21m
service/kubernetes   ClusterIP  10.96.0.1     <none>      443/TCP      21m

NAME                   ENDPOINTS         AGE
endpoints/basicservice <none>            21m
endpoints/kubernetes   10.128.0.3:6443   21m
```

9. `ReplicaSet` を削除し、リソースを再度表示します。`ReplicaSet` とそれが制御する Pod の年齢が 1 分未満になったことに注意してください。Deployment オペレーターは、既存のものを削除したときに新しい`ReplicaSet` オペレーターを作成しています。新しい`ReplicaSet` は、希望する仕様が現在のステータスと一致しない場合に、別の Pod を作成します。

```
student@cp:~$ kubectl delete rs firstpod-6bb4574d94-rqk76
```

```
replicaset.apps "firstpod-6bb4574d94-rqk76" deleted
```

```
student@cp:~$ kubectl get deployment,rs,po,svc,ep
```

```
NAME                          READY  UP-TO-DATE AVAILABLE AGE
deployment.apps/firstpod  1/1    1          1         7m

NAME                                        DESIRED   CURRENT....
replicaset.apps/firstpod-6bb4574d94-rqk76   1         1     ....

NAME                          READY     STATUS    RESTARTS   AGE
pod/firstpod-7d99ffc75-p9hbw  1/1       Running   0          12s

NAME                 TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   24m

NAME                   ENDPOINTS         AGE
endpoints/kubernetes   10.128.0.2:6443   80m
endpoints/basicservice   <none>            21m
```

10. 今回はトップレベルコントローラーを削除します。約 30 秒後にすべてがシャットダウンされると、クラスター Service と作成した Service の Endpoint だけが残っているのが見えるはずです。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
student@cp:~$ kubectl delete deployment firstpod
```

```
deployment.apps "firstpod" deleted
```

```
student@cp:~$ kubectl get deployment,rs,po,svc,ep
```

```
NAME                 TYPE      CLUSTER-IP    EXTERNAL-IP PORT(S)      AGE
service/basicservice NodePort  10.108.147.76 <none>      80:31601/TCP 35m
kubernetes           ClusterIP 10.96.0.1     <none>      443/TCP      24m

NAME                   ENDPOINTS         AGE
endpoints/basicservice <none>            21m
kubernetes             10.128.0.3:6443   24m
```

11. しばらくは必要ないので、basicservice の Service も削除してください。

```
student@cp:~$ kubectl delete svc basicservice
```

```
service "basicservice" deleted
```

## 課題 2.6: 出題範囲の確認

**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

1. ブラウザでhttps://www.cncf.io/certification/ckad/にアクセスし、プログラムの説明に目を通してください。
2. 「**Exam Resources**」セクションで、Curriculum Overview とCandidate Handbook を新しいタブで開きます。どちらも受験前に読み、理解しておく必要があります。
3. Curriculum Overviewタブに移動します。試験のさまざまなバージョンの関連情報のリンクが表示されるはずです。**CKAD_Curriculum_V1.25.1.pdf** のように、最新バージョンを選択します。表示されるバージョンは異なる場合があります。PDF を表示する新しいページが表示されるはずです。
4. ドキュメントに目を通しましょう。Understand Services のような Understand という用語は、単にその存在を知っているだけでは十分ではないことを認識しておいてください。この場合、作成、更新、トラブルシューティングの意味も含んでいると考えてください。
5. **Application Design and Build** のセクションを確認しましょう。演習を見直すと、マルチコンテナ Pod の設計など、これらのステップのいくつかをカバーしていることがわかります。繰り返しますが、このドキュメントは本書とは別に変更されることに注意してください。オンラインドキュメントに変更がないか確認するのは、あなたの責任です。不定期に変更される可能性があります。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


## Certified Kubernetes Application Developer (CKAD) Exam Curriculum

This document provides the curriculum outline of the Knowledge, Skills and Abilities that a Certified Kubernetes Application Developer (CKAD) can be expected to demonstrate.

### CKAD Curriculum

**20% - Application Design and Build**

- Define, build and modify container images
- Understand Jobs and CronJobs
- Understand multi-container Pod design patterns (e.g. sidecar, init and others)
- Utilize persistent and ephemeral volumes

**25% - Application Environment, Configuration and Security**

- Discover and use resources that extend Kubernetes (CRD)
- Understand authentication, authorization and admission control
- Understanding and defining resource

Figure 2.1: **Application Design and Build Domain**

6. Candidate Handbook タブに移動します。試験を受ける前に、この文書全体を読み、理解することが強く推奨されます。繰り返しますが、この文書は、本書とは別に変更されることにご注意ください。オンライン上の文書に変更がないか確認するのは、あなたの責任です。不定期に、予告なく変更されることがあります。
7. ドキュメント内の **Important Instructions: CKA and CKAD** セクションを探します。このセクションに目を通し、v1.27 など試験用の現在の Kubernetes のバージョンを確認し、**Resources allowed during exam** のセクションを見つけます。
8. 試験中に使用できるドメインとサブドメインは、いくつかの条件が明記されているので注意してください。

**Resources allowed during exam**

During the exam, candidates may:

- review the Exam content instructions that are presented in the command line terminal
- review Documents installed by the distribution (i.e. /usr/share and its subdirectories)
- use their Chrome or Chromium browser to open one additional tab in order to access assets at: https://kubernetes.io/docs/, https://github.com/kubernetes/, https://kubernetes.io/blog/ and their subdomains. This includes all available language translations of these pages (e.g. https://kubernetes.io/zh/docs/)

No other tabs may be opened and no other sites may be navigated to (including https://discuss.kubernetes.io/).

*The allowed sites above may contain links that point to external sites. It is the responsibility of the candidate not to click on any links that cause them to navigate to a domain that is not allowed.*

Exam Details
System Requirements to take...
Acceptable Testing Location
ID Requirements to take the ...
Sanctioned Countries
Resources allowed during ex...
Exam Technical Instructions
CKA & CKAD Environment
Additional Resources

Figure 2.2: **Exam Handbook Resources Allowed**

9. 許可されたリソースから良い YAML の例を検索しましょう。良いページがどのように見えるかを知りましょう。試験中、あなたは


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



安全なブラウザを使用することになります。既存のブックマークがあるはずですが、必要な数より多いかもしれません。そのページをすぐに再認識することで、時間を節約できます。受験する試験のバージョンで動作することを確認しましょう。

10. タイマーを使って、各項目を作成し検証するのにかかる時間を確認します。もう一度、各ステップをどれだけ速く完了できるか試してみてください。

    - **nginx** イメージの新しい Pod です。すべてのコンテナが稼働し、`Ready` status ステータスが表示されています。
    - **nodePort** として Pod を公開する新しい Service で、前のステップで設定した動作する Web サーバーに対応します。
    - **nginx:1.11-alpine** イメージを実行するように Pod を更新し、nodePort を介して Web サーバーを表示できることを再確認します。

11. コースの tarball に含まれる`architecture-review1.yaml`ファイルを検索して使用します。コース番号などのパスは、以下の例のものと異なる場合があります。**find** の出力を使用します。Pod が動作しているかどうかを判断します。遭遇する可能性のあるエラーを修正します。が **kubectl describe** コマンドが役に立つことがあります。

```
student@cp:~$ find $HOME -name architecture-review1.yaml
```

```
<some_long_path>/architecture-review1.yaml
^^I
```

```
student@cp:~$ cp <copy-paste-from-above>  .

student@cp:~$ kubectl create -f architecture-review1.yaml
```

12. 次のセクションに進む前に、レビューの一部として作成した Pod や Service を削除してください。たとえば、次のようになります。

```
student@cp:~$ kubectl delete -f architecture-review1.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 3 章
# ビルド

## 課題 3.1: 新しいアプリケーションのデプロイ

**概要**

この演習では、非常にシンプルな **Python** アプリケーションをデプロイします。**Podman** を使用してそのアプリケーションをテストし、Kubernetes にデプロイし、稼働状態を継続するためにプローブを設定します。この演習では、Kubernetes クラスターのインストールと設定を行った前回の演習が完了していることを前提としています。

**Podman** と **crictl** は **docker** と同じ構文を使っているので、docker を代わりに使っている場合でも、本演習は同じように動作するはずです。

### シンプルな **Python** スクリプトの操作

1. cp ノードに python をインストールします。以下の出力にあるように、すでにインストールされている場合もあります。

```
student@cp:~$ sudo apt-get -y install python3
```

```
Reading package lists... Done
Building dependency tree
Reading state information... Done
python3 is already the newest version (3.8.2-0ubuntu2).
The following package was automatically installed and is no longer required:
  libnuma1
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

2. システム上の python バイナリを探します。

```
student@cp:~$ which python3
```

```
/usr/bin/python3
```

3. 新しいディレクトリを作成し、カレントディレクトリをそこに変更します。Podman のビルドプロセスは、デフォルトでカレントディレクトリからすべてをイメージファイルに引き込みます。選択したディレクトリが空であることを確認してください。

```
student@cp:~$ mkdir app1
student@cp:~$ cd app1
student@cp:~/app1$ ls -l
```

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
total 0
```

4. 5 秒ごとに時刻とホスト名を表示する簡単な python スクリプトを作成します。このスクリプトには 6 つのコメントされた部分があり、それぞれの部分が何をするのかを説明する必要があります。このスクリプトは本コースの tar ファイルに他の部分と一緒に含まれているので、以前に使った **find** コマンドを使ってファイルを見つけてコピーするとよいでしょう。

   テキストエディターコマンドとして **vim** を例にしていますが、**nano** など他のテキストエディターも同様に動作します。

```
student@cp:~/app1$ vim simple.py
```

simple.py

```python
#!/usr/bin/python3
## Import the necessary modules
import time
import socket

## Use an ongoing while loop to generate output
while True :

## Set the hostname and the current date
  host = socket.gethostname()
  date = time.strftime("%Y-%m-%d %H:%M:%S")

## Convert the date output to a string
  now = str(date)

## Open the file named date in append mode
## Append the output of hostname and time
  f = open("date.out", "a" )
  f.write(now + "\n")
  f.write(host + "\n")
  f.close()

## Sleep for five seconds then continue the loop
  time.sleep(5)
```

5. このファイルを実行可能な状態にし、動作確認をしてください。`Ctrl-C` を使って、20 秒か 30 秒後に`while` ループを中断してください。出力は、カレントディレクトリに新しく作成した `date.out`というファイルに送られます。

```
student@cp:~/app1$ chmod +x simple.py
student@cp:~/app1$ ./simple.py
```

```
^CTraceback (most recent call last):
  File "./simple.py", line 42, in <module>
    time.sleep(5)
KeyboardInterrupt
```

6. 出力にノード名と時刻のスタンプがあることを確認します。

```
student@cp:~/app1$ cat date.out
```

```
2018-03-22 15:51:38
cp
2018-03-22 15:51:43
cp
2018-03-22 15:51:48
cp
<output_omitted>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



7. Dockerfileという名のテキストファイルを作成します。

**重要**

名前は重要で、接尾辞をつけることはできません。

Dockerfile ファイル内において、FROM は使用する Python のバージョン、ADD はスクリプト、CMD はコンテナの動作を示すものです。より複雑な作業を行う場合は、追加のライブラリをインストールする必要があります。次の例では、RUN pip install をコメントアウトしています。

```
student@cp:~/app1$ vim Dockerfile
```

**Dockerfile**

```
FROM docker.io/library/python:3
ADD simple.py /
## RUN pip install pystrich
CMD [ "python", "./simple.py" ]
```

8. オープンソースのプロジェクトでは、アップストリームのバージョンに問題があったり、利用できなかったりすることがあります。このような場合、ソースコードからツールを自身で作成するか、誰かが作成したバイナリをインストールできます。ここでは時間の都合上、既に作成されているバイナリをインストールすることにします。最初のコマンドが 2 行にまたがっていることに注意してください。それぞれを 1 行ずつ入力するのであれば、バックスラッシュは必要ありません。また、**cp** コマンドと末尾のスラッシュにも注意してください。これはコピーの動作を変更する可能性があります。

   以下のバイナリが存在しない場合、最新のバイナリを検索してください。

```
student@cp:~/app1$ curl -fsSL -o podman-linux-amd64.tar.gz \
https://github.com/mgoltzsche/podman-static/releases/latest/download/podman-linux-amd64.tar.gz

student@cp:~/app1$ tar -xf podman-linux-amd64.tar.gz

student@cp:~/app1$ sudo cp -r podman-linux-amd64/usr podman-linux-amd64/etc /
```

9. コンテナをビルドします。以下の出力例では必要なソフトウェアがダウンロードされたので、ビルドが終了したことを表示しています。このコマンド **podman build** を実行するためには、**sudo** を使用する必要があります。ビルドプロセスが完了した後、出力されたメッセージの最後の行が成功を示しているはずです。コマンドの最後にあるドット(.)は、カレントディレクトリを示すことに注意してください。

   **podman** コマンドは、**docker** の機能をすべて置き換えるため、**docker** コマンドと同じ構文を受け入れるように作られています。しかし変化の激しいオープンソースプロジェクトのため、若干の違いがあるかもしれません。ビルドプロセスが完了するまでに 1 分近くかかり、途中 1〜2 回の休止に直面するかもしれません。**docker** のエイリアスとして **podman** が設定されていることもあります。

   docker.io バージョンの python を使用することを選択します。

```
student@cp:~/app1$ sudo podman build -t simpleapp .
```

```
STEP 1/3: FROM python:3
Resolved "python" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull docker.io/library/python:3...
Getting image source signatures
Copying blob 0c6b8ff8c37e done
Copying blob 724cfd2dc19b done
Copying blob 808edda3c2e8 done
Copying blob 461bb1d8c517 done
Copying blob e6d3e61f7a50 done
Copying blob 412caad352a3 done
Copying blob 1bb6570cd7ac done
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
Copying blob aca06d6d45b1 done
Copying blob 678714351737 done
Copying config dfce7257b7 done
Writing manifest to image destination
Storing signatures
STEP 2/3: ADD simple.py /
--> 0e0bd8b3671
STEP 3/3: CMD [ "python", "./simple.py" ]
COMMIT simpleapp
--> 5b825f8d5f3
Successfully tagged localhost/simpleapp:latest
5b825f8d5f3a69660f2278a7119716e2a0fc1a5756d066800429f8030f83e978
```

10. ビルドプロセス中にダウンロードされたイメージ、クラスターをサポートするためにインストールされたイメージ、また既に使用したことがあるイメージとともに、新しいイメージが存在することを podman images で確認してください。新しく作成された simpleapp イメージがコンテナイメージのリストの最初に表示されています。

```
student@cp:~/app1$ sudo podman images
```

```
REPOSITORY                          TAG       IMAGE ID      CREATED         SIZE
localhost/simpleapp                 latest    8dd4fa975329  39 seconds ago  925 MB
docker.io/calico/node               v3.19.0   b0744cc52c19  12 days ago     155 MB
docker.io/calico/pod2daemon-flexvol v3.19.0   a5decf77918d  12 days ago     21.8 MB
docker.io/calico/cni                v3.19.0   3d17cd6307a4  12 days ago     146 MB
<output_omitted>
```

11. **sudo podman** を使用して、新しいイメージを使用したコンテナを実行します。スクリプトが実行されている間は、何も出力されず、シェルはバックグラウンドでイメージを実行することに専念します。30 秒後に **ctrl-c** で中断してください。ローカルの date.out ファイルは新しい時刻で更新されず、代わりにコンテナイメージのファイルが出力されます。

```
student@cp:~/app1$ sudo podman run localhost/simpleapp
```

```
^CTraceback (most recent call last):
  File "./simple.py", line 24, in <module>
    time.sleep(5)
KeyboardInterrupt
```

12. 新しく作成された date.out ファイルを探します。次のコマンドで、この名前のファイルが 2 つ表示されるはずです。simple.py を実行したときに作成されたものと、Podman もしくは crio コンテナを介して実行したときに /var/lib/containers の下に作成されたものがあります。

```
student@cp:~/app1$ sudo find / -name date.out
```

```
/home/student/app1/date.out
/var/lib/containers/storage/overlay/
0dea104afa098f608dff06b17b2196d0ba12d09a775243781862abf016e3378a/diff/date.out
```

13. Podman で作成した date.out ファイルの中身を表示します。今回は Podman でファイルを作成したため、**sudo** が必要であり、所有者は root になっています。以下の例では長い名前のため数行で表示されていますが、タイプしたりコピーしたりすると 1 行になります。

```
student@cp:~/app1$ sudo tail \
 /var/lib/containers/storage/overlay/
0dea104afa098f608dff06b17b2196d0ba12d09a775243781862abf016e3378a/diff/date.out
```

```
2018-03-22 16:13:46
53e1093e5d39
2018-03-22 16:13:51
53e1093e5d39
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
2018-03-22 16:13:56
53e1093e5d39
```

## 課題 3.2: ローカルリポジトリの設定

アカウントを作成し、アプリケーションをhttps://hub.docker.com やhttps://artifacthub.io/にアップロードして世界と共有することもできますが、ここでは代わりにローカルリポジトリを作成し、クラスターのノードで利用できるようにします。

1. 本コースの tarball に含まれる`easyregistry.yaml`ファイルを使用して、簡単なレジストリを作成します。以下の例のように **find** によって返されるパスに存在するファイルを使用します。

```
student@cp:~/app1$ find $HOME -name easyregistry.yaml
```

```
<some_long_path>/easyregistry.yaml
```

```
student@cp:~/app1$ kubectl create -f <path_from_output_above>
```

```
service/nginx created
service/registry created
deployment.apps/nginx created
persistentvolumeclaim/nginx-claim0 created
deployment.apps/registry created
persistentvolumeclaim/registry-claim0 created
persistentvolume/vol1 created
persistentvolume/vol2 created
```

/clearpage

2. 新しいレジストリサービスの`ClusterIP` をメモしておきます。以下の例では、`10.97.40.62` です。

```
student@cp:~/app1$ kubectl get svc | grep registry
```

```
registry      ClusterIP   10.97.40.62    <none>      5000/TCP,8080/TCP   5m35s
```

3. そのリポジトリが動作していることを確認します。接続が失敗する場合は、ファイアウォールの問題の可能性があります。GCE を使用してノードを運用している場合、インスタンスが VPC セットアップを使用し、すべてのポートへの通信が許可されていることを確認してください。AWS を使用している場合も、すべてのポートへの通信が許可されていることを確認してください。

   次の例において curl コマンドで指定する IP アドレスは、前のコマンドで見つけた`localrepo` サービスのものに変更し、ポート番号は `5000` にします。

```
student@cp:~/app1$ curl 10.97.40.62:5000/v2/_catalog
```

```
{"repositories":[]}
```

4. **podman** を非 TLS のリポジトリで動作できるように設定します。この設定方法は、使用するコンテナエンジンに依存します。今回の設定では、`/etc/containerd/config.toml`ファイルを編集して、先ほど作成したレジストリを追加します。

   ローカルリポジトリをセットアップするためのスクリプト **local-repo-setup.sh** を tar ファイルに同梱しています。

```
student@cp:~/app1$ find $HOME -name local-repo-setup.sh
student@cp:~/app1$ cp /home/student/LFD459/SOLUTIONS/s_03/local-repo-setup.sh $HOME
student@cp:~/app1$ chmod +x $HOME/local-repo-setup.sh
student@cp:~/app1$ . $HOME/local-repo-setup.sh       # dot <space> scriptname to set correct variables
```

```
. local-repo-setup.sh
Configuring local repo, Please standby
[[registry]]
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
location = "10.97.40.62:5000"
insecure = true
[plugins."io.containerd.grpc.v1.cri".registry.mirrors."*"]
endpoint = ["http://10.97.40.62:5000"]

Local Repo configured, follow the next steps
```

5. hub.docker.comから代表的なイメージをダウンロードし、タグを付けます。$repo 変数を用いてレジストリの IP とポートを含んだ形式で、イメージにタグを付けます。

```
student@cp:~app1/$ sudo podman pull docker.io/library/alpine
```

```
Resolved "alpine" as an alias (/etc/containers/registries.conf.d/000-shortnames.conf)
Trying to pull docker.io/library/alpine:latest...
Getting image source signatures
Copying blob 540db60ca938 done
Copying config 6dbb9cc540 done
Writing manifest to image destination
Storing signatures
6dbb9cc54074106d46d4ccb330f2a40a682d49dda5f4844962b7dce9fe44aaec
```

```
student@cp:~/app1$ sudo podman tag alpine $repo/tagtest
```

6. 新しくタグ付けしたイメージをローカルレジストリにプッシュします。**an HTTP request to an HTTPS client** に関連したエラーが発生した場合、/etc/containers/registry.confファイルを正しく編集し、サービスを再起動してください。

```
student@cp:~/app1$ sudo podman push $repo/tagtest
```

```
Getting image source signatures
Copying blob b2d5eeeaba3a done
Copying config 6dbb9cc540 done
Writing manifest to image destination
Storing signatures
```

7. ローカルリポジトリからイメージをプルできることを確認するためのテストを行います。まず、ローカルにキャッシュされたイメージを削除することから始めます。

```
student@cp:~/app1$ sudo podman image rm alpine
```

```
Untagged: docker.io/library/alpine:latest
```

```
student@cp:~/app1$ sudo podman image rm $repo/tagtest
```

```
Untagged: 10.97.40.62:5000/tagtest:latest
Deleted: 6dbb9cc54074106d46d4ccb330f2a40a682d49dda5f4844962b7dce9fe44aaec
```

8. ローカルレジストリからイメージをプルします。より新しいイメージがダウンロードされるはずです。

```
student@cp:~/app1$ sudo podman pull $repo/tagtest
```

```
Trying to pull 10.97.40.62:5000/tagtest:latest...
Getting image source signatures
Copying blob 74782b667c7d done
Copying config 6dbb9cc540 done
Writing manifest to image destination
Storing signatures
6dbb9cc54074106d46d4ccb330f2a40a682d49dda5f4844962b7dce9fe44aaec
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



9. ワーカーノードが cp サーバー上で動作するレジストリを使用するように設定します。ワーカーノードに接続し **local-repo-setup.sh** を実行します。

```
student@worker:~$ find $HOME -name local-repo-setup.sh
student@worker:~$ cp /home/student/LFD459/SOLUTIONS/s_03/local-repo-setup.sh $HOME
student@worker:~$ chmod +x $HOME/local-repo-setup.sh
student@worker:~$ . $HOME/local-repo-setup.sh       # dot <space> scriptname to set correct variables
student@worker:$ sudo podman pull $repo/tagtest # if needed export repo=10.97.40.62:5000
```

```
. local-repo-setup.sh
Configuring local repo, Please standby
[[registry]]
location = "10.97.40.62:5000"
insecure = true
[plugins."io.containerd.grpc.v1.cri".registry.mirrors."*"]
endpoint = ["http://10.97.40.62:5000"]

Local Repo configured, follow the next steps


Trying to pull 10.97.40.62:5000/tagtest:latest...
Getting image source signatures
Getting image source signatures
Copying blob 74782b667c7d done
Copying config 6dbb9cc540 done
Writing manifest to image destination
Storing signatures
a313e647ae05302fc54b57065c250ff58d2f580fab7b7625949651b141d9fca7
```

10. すべてのノード上で **podman** がそのリポジトリを使えることが確認できました。次に Kubernetes に対して新しいリポジトリに関する設定が行われていることを確認する必要があります。最も簡単な方法は すべて のノードを再起動することです。再起動に伴い接続が切れたらログインし直します。

```
student@cp:~$ sudo reboot

student@worker:~$ sudo reboot
```

11. 良いアドミニストレータとして設定が永続的に有効であることを確認するため、再起動後にリポジトリが動作することをテストしてください。再起動後、kube-apiserver が完全に起動するまでに 1〜2 分かかることがありますので、ご注意ください。変数`$repo` が設定されていない場合は、`.bashrc`の source ステートメントを確認してください。

```
student@cp:~$ curl $repo/v2/_catalog
```

```
{"repositories":[]}
```

12. **podman tag** を使って `simpleapp` のイメージを割り当て、ローカルレジストリにそのイメージをプッシュします。そのイメージと依存イメージは、ローカルリポジトリにプッシュする必要があります。

```
student@cp:~/app1$ sudo podman tag simpleapp $repo/simpleapp

student@cp:~/app1$ sudo podman push $repo/simpleapp
```

```
Getting image source signatures
Copying blob 47458fb45d99 done
Copying blob a3c1026c6bcc done
Copying blob f1d420c2af1a done
Copying blob 461719022993 done
Copying blob d35c5bda4793 done
Copying blob 46829331b1e4 done
Copying blob ceee8816bb96 done
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
Copying blob da7b0a80a4f2 done
Copying blob e571d2d3c73c done
Copying blob 5c2db76bc949 done
Copying config a313e647ae done
Writing manifest to image destination
Storing signatures
```

13. cp と worker の両方から、そのイメージがリポジトリに存在することを確認します。

```
student@cp:~$ curl $repo/v2/_catalog
```

```
{"repositories":["simpleapp"]}
```

14. cp ノードに戻り、Kubernetes に複数のレプリカで `simpleapp` をデプロイします。その Deployment の名前は`try1` とします。その後、レプリカ数が 6 になるようにその Deployment をスケールさせます。cp と worker の両ノードに Pod がデプロイされるまで、レプリカの数を増やしていきます。

```
student@cp:~$ kubectl create deployment try1 --image=$repo/simpleapp
```

```
deployment.apps/try1 created
```

```
student@cp:~$  kubectl scale deployment try1 --replicas=6
```

```
deployment.apps/try1 scaled
```

```
student@cp:~$  kubectl get pod -o wide
```

```
NAME                        READY   STATUS    RESTARTS   AGE     IP                NODE  ....
basicpod                    1/1     Running   0          20m     192.168.95.15     worker ....
registry-ddf6bcb7c-b8wcr    1/1     Running   0          16m     192.168.95.16     worker ....
try1-55f675ddd-28vgs        1/1     Running   0          17s     192.168.95.30     worker ....
try1-55f675ddd-2nrsj        1/1     Running   0          17s     192.168.95.26     worker ....
try1-55f675ddd-4vzt7        1/1     Running   0          17s     192.168.219.100   cp ....
<output_omitted>
```

15. 2 番目のノードでは **sudo crictl ps** を使用して、`simpleapp` のコンテナが実行されていることを確認します。**podman** にも ps オプションがありますが、podman を使う場合は cri-o がコンテナランタイムとしてコンテナを実行します。スケジューラは通常、ノード間で Pod 数のバランスを取ります。cp ノードにはすでにいくつかの Pod が実行されているので、新しい Pod は worker ノード上に存在する可能性があり、Pod の数は変化します。

    エラーメッセージを避けるために、最初に **crictl** を設定する必要がある場合があります。次の例ではバックスラッシュを省略し、以下のコマンドを 1 行で入力することもできます。

```
student@worker:~$ sudo crictl config \
  --set runtime-endpoint=unix:///run/containerd/containerd.sock \
  --set image-endpoint=unix:///run/containerd/containerd.sock
```

```
student@worker:~$ sudo crictl ps | grep simple
```

```
855750df82cd4       10.97.177.111:5000/
simpleapp@sha256:f7fc297b3c20c47c506896ee13f4d7f13b4c4c0ee0fafd2ab88c9809b2a1aed6   15 minutes ago
Running             simpleapp                 0                   6e31ba7491117
237af1260265a       10.97.177.111:5000/
simpleapp@sha256:f7fc297b3c20c47c506896ee13f4d7f13b4c4c0ee0fafd2ab88c9809b2a1aed6   15 minutes ago
Running             simpleapp                 0                   bc0d63564cf49
d0ae2910cf8b8       10.97.177.111:5000/
simpleapp@sha256:f7fc297b3c20c47c506896ee13f4d7f13b4c4c0ee0fafd2ab88c9809b2a1aed6   15 minutes ago
Running             simpleapp                 0                   b9bb90e04bec1
046a0c439fa43       10.97.177.111:5000/
simpleapp@sha256:f7fc297b3c20c47c506896ee13f4d7f13b4c4c0ee0fafd2ab88c9809b2a1aed6   15 minutes ago
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
Running             simpleapp                0                   fe464425e41b1
```

16. cp ノードに戻ります。try1 Deployment を YAML として保存します。

```
student@cp:~/app1$ cd $HOME/app1/
student@cp:~/app1$ kubectl get deployment try1 -o yaml > simpleapp.yaml
```

17. YAML ファイルを使用してtry1 Deployment を削除し、再作成します。Deployment が期待通りの 6 つのレプリカで実行されていることを確認します。

```
student@cp:~$ kubectl delete deployment try1
```

```
deployment.apps "try1" deleted
```

```
student@cp:~/app1$ kubectl create -f simpleapp.yaml
```

```
deployment.apps/try1 created
```

```
student@cp:~/app1$ kubectl get deployment
```

```
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
nginx      1/1     1            1           15m
registry   1/1     1            1           15m
try1       6/6     6            6           5s
```

## 課題 3.3: プローブの設定

クライアントからのアクセスに先立ち、大きなデータセットをロードしたり、複雑なアプリケーションを起動する必要がある場合、readinessProbe を活用できます。チェックが満たされ成功した終了コードを返すまで、その Pod はクラスターから利用可能になりません。readinessProbes とlivenessProbes はどちらも同じ構文を使用し、名前以外は同じチェック機能を持っています。Pod が ready になる前、readinessProbe がチェックされ、その後引き続きlivenessProbe がチェックされます。

livenessProbe は 3 種類あります。コマンドが成功を意味するゼロの終了値を返すかどうかチェックする機能。HTTP リクエストが 200 から 399 の範囲の応答コードを返すかどうかチェックする機能。最後の 3 つ目は TCP ソケットを使用するものです。次の例では、ファイル/tmp/healthyが作成され、アクセスできるようになったときにゼロの終了コードを返すコマンド **cat** を使用することにします。

1. YAML 形式のデプロイメントファイルを編集し、readinessProbe のための数行を追加します。YAML を編集する際は、空白が重要であることに注意してください。インデントは、関連する行やファイル全体のどこに情報を関連付けるべきかを解析するために使用されます。インデントにはタブを使わず、スペースを使ってください。データの検証に関するエラーが発生した場合は、インデントを確認するとよいでしょう。また、ウェブサイトhttps://www.json2yaml.com/に YAML ファイルを貼り付けて、インデントが JSON 値にどのように影響するかを確認することも役立ちます。これは実際に Kubernetes が YAML ファイルを取り込む際に内部で行っていることです。今回編集に利用したファイルは tarball に含まれていますが、イメージ名をレジストリの IP アドレスに合うように編集する必要があります。

```
student@cp:~/app1$ vim simpleapp.yaml
```

simpleapp.yaml

```
....
    spec:
      containers:
      - image: 10.111.235.60:5000/simpleapp
        imagePullPolicy: Always
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```yaml
        name: simpleapp
        readinessProbe:         #<--This line and next five
          periodSeconds: 5
          exec:
            command:
            - cat
            - /tmp/healthy
        resources: {}
....
```

2. `try1` の Deployment を削除し、再作成します。

```
student@cp:~/app1$ kubectl delete deployment try1
```

```
deployment.apps "try1" deleted
```

```
student@cp:~/app1$ kubectl create -f simpleapp.yaml
```

```
deployment.apps/try1 created
```

3. 新しい Deployment `try1` では、6 つの Pod が作られるはずですが、利用可能な Pod はゼロと表示されます。これは`/tmp/healthy`ファイルが欠落しているからです。

```
student@cp:~/app1$ kubectl get deployment
```

```
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
nginx      1/1     1            1           19m
registry   1/1     1            1           19m
try1       0/6     6            0           15s
```

4. それらの Pod を詳しく見てみましょう。**kubectl describe pod** と **kubectl logs** を使用して問題を調査します（ログがない場合もあります）。ヘルスチェックファイルを作成するテストを行うため、`try1` Pod のうちの 1 つを選択します。

```
student@cp:~/app1$ kubectl get pods
```

```
NAME                        READY     STATUS    RESTARTS   AGE
nginx-6b58d9cdfd-g7lnk      1/1       Running   1          40m
registry-795c6c8b8f-7vwdn   1/1       Running   1          40m
try1-9869bdb88-2wfnr        0/1       Running   0          26s
try1-9869bdb88-6bknl        0/1       Running   0          26s
try1-9869bdb88-786v8        0/1       Running   0          26s
try1-9869bdb88-gmvs4        0/1       Running   0          26s
try1-9869bdb88-lfvlx        0/1       Running   0          26s
try1-9869bdb88-rtchc        0/1       Running   0          26s
```

5. 対話的に bash シェルを実行し、空の`/tmp/healthy` ファイルを作ります。

```
student@cp:~/app1$ kubectl exec  -it try1-9869bdb88-rtchc -- /bin/bash

root@try1-9869bdb88-rtchc:/# touch /tmp/healthy

root@try1-9869bdb88-rtchc:/# exit
```

```
exit
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



6. 少なくとも 5 秒待って、もう一度 Pod を確認します。プローブが再び実行されると、コンテナはすぐに利用可能と表示されるはずです。/tmp/healthy ファイルを持つ Pod は Running 状態となり、READY 状態の1/1 が表示されるはずです。残りは0/1 を表示し続けるでしょう。

```
student@cp:~/app1$ kubectl get pods
```

```
NAME                        READY     STATUS    RESTARTS   AGE
nginx-6b58d9cdfd-g7lnk      1/1       Running   1          44m
registry-795c6c8b8f-7vwdn   1/1       Running   1          44m
try1-9869bdb88-2wfnr        0/1       Running   0          4m
try1-9869bdb88-6bknl        0/1       Running   0          4m
try1-9869bdb88-786v8        0/1       Running   0          4m
try1-9869bdb88-gmvs4        0/1       Running   0          4m
try1-9869bdb88-lfvlx        0/1       Running   0          4m
try1-9869bdb88-rtchc        1/1       Running   0          4m
```

7. 残りの Pod でも同様に空の/tmp/healthy ファイルを作ります。各 Pod を更新する簡単な方法として、**for** ループが使用できます。なお、コマンド出力に表示されている > はセカンダリプロンプトを表しており、その文字を入力することはありません。

```
student@cp:~$ for name in try1-9869bdb88-2wfnr try1-9869bdb88-6bknl \
> try1-9869bdb88-786v8 try1-9869bdb88-gmvs4 try1-9869bdb88-lfvlx
> do
> kubectl exec $name -- touch /tmp/healthy
> done
```

8. プローブが/tmp/healthy ファイルをチェックし、ヘルスチェックが成功するまでには、しばらく時間がかかる場合があります。

```
student@cp:~/app1$ kubectl get pods
```

```
NAME                        READY     STATUS    RESTARTS   AGE
nginx-6b58d9cdfd-g7lnk      1/1       Running   1          1h
registry-795c6c8b8f-7vwdn   1/1       Running   1          1h
try1-9869bdb88-2wfnr        1/1       Running   0          22m
try1-9869bdb88-6bknl        1/1       Running   0          22m
try1-9869bdb88-786v8        1/1       Running   0          22m
try1-9869bdb88-gmvs4        1/1       Running   0          22m
try1-9869bdb88-lfvlx        1/1       Running   0          22m
try1-9869bdb88-rtchc        1/1       Running   0          22m
```

9. Pod が健全であることがわかったので、livenessProbe を使用して Pod が健全であることを追跡したいと思うかもしれません。1 つのプローブreadinessProbe で Pod が利用可能になったときを判断し、2 つ目のプローブとしてlivenessProbe を使うことで継続的な健全性を確認できます。

   Deployment を再度編集します。以下のように、livenessProbe セクションを追加します。今回は、ポート 8080 に応答するシンプルな アプリケーションを実行する Sidecar コンテナを Pod に追加します。名前の前にダッシュ(-)があることに注意してください。また、goproxy は、先ほどの simpleapp のimage: 行の前にある-と同じ数のスペースでインデントされています。この例では、7 つのスペースになります。

```
student@cp:~/app1$ vim simpleapp.yaml
```

simpleapp.yaml

```
....
          terminationMessagePath: /dev/termination-log
          terminationMessagePolicy: File
       - name: goproxy                 #<-- Indented 7 spaces, add lines from here...
         image: registry.k8s.io/goproxy:0.1
         ports:
         - containerPort: 8080
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
8          readinessProbe:
9            tcpSocket:
10             port: 8080
11           initialDelaySeconds: 5
12           periodSeconds: 10
13         livenessProbe:                #<-- This line is 9 spaces indented, fyi
14           tcpSocket:
15             port: 8080
16           initialDelaySeconds: 15
17           periodSeconds: 20           #<-- ....to here
18       dnsPolicy: ClusterFirst
19       restartPolicy: Always
20 ....
```

10. Deployment を削除して再作成します。

```
student@cp:~$ kubectl delete deployment try1
```

```
deployment.apps "try1" deleted
```

```
student@cp:~$ kubectl create -f simpleapp.yaml
```

```
deployment.apps/try1 created
```

11. 新しく作成された Pod を表示します。1 つの Pod に 2 つのコンテナがあり、1 つのコンテナだけが実行されていることに気づくでしょう。新しい simpleapp コンテナは/tmp/healthyファイルを持たないので、再び/tmp/healthyファイルが作られるまで利用できません。コンテナの引数に、ファイルを作成するコマンドを含めることができます。以下の出力では、古い Pod が終了するのに少し時間がかかることを示しています。

```
student@cp:~$ kubectl get pods
```

```
NAME                        READY   STATUS              RESTARTS   AGE
nginx-6b58d9cdfd-g7lnk      1/1     Running             1          13h
registry-795c6c8b8f-7vwdn   1/1     Running             1          13h
try1-76cc5ffcc6-4rjvh       1/2     Running             0          3s
try1-76cc5ffcc6-bk5f5       1/2     Running             0          3s
try1-76cc5ffcc6-d8n5q       0/2     ContainerCreating   0          3s
try1-76cc5ffcc6-mm6tw       1/2     Running             0          3s
try1-76cc5ffcc6-r9q5n       1/2     Running             0          3s
try1-76cc5ffcc6-tx4dz       1/2     Running             0          3s
try1-9869bdb88-2wfnr        1/1     Terminating         0          12h
try1-9869bdb88-6bknl        1/1     Terminating         0          12h
try1-9869bdb88-786v8        1/1     Terminating         0          12h
try1-9869bdb88-gmvs4        1/1     Terminating         0          12h
try1-9869bdb88-lfvlx        1/1     Terminating         0          12h
try1-9869bdb88-rtchc        1/1     Terminating         0          12h
```

12. readinessProbe のヘルスチェックファイルを作成します。各アクションに対して再度 **for** ループを使用できます。このセットアップでは Label を活用するため、Pod 名を調べる必要がありません。今回の Pod には 2 つのコンテナがあるので、どちらのコンテナでコマンドを実行するかコンテナ名を指定する必要があります。コンテナ名を指定しない場合、最初のコンテナがデフォルトとなります。YAML ファイルをどのように編集したかによりますが、simpleapp が最初の Pod で、goproxy が 2 番目の Pod になるはずです。正しいコンテナが更新されるようにするには、**kubectl** コマンドに **-c simpleapp** を追加します。みなさんの環境の Pod 名は以下とは異なるものになります。**kubectl get pods** コマンドの出力から、新しく起動したコンテナの名前を使用します。次の実行例において > 文字がセカンダリプロンプトを表していることに注意してください。その文字を入力することはありません。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
student@cp:~$ for name in $(kubectl get pod -l app=try1 -o name)
> do
> kubectl exec $name -c simpleapp -- touch /tmp/healthy
> done
```

13. その後 1 分ほどで、各 Pod の Sidecar コンテナ（Running ではなかったもの）のステータスがRunning に変更されます。それぞれ2/2 のコンテナが稼働していることがわかるはずです。

```
student@cp:~$ kubectl get pods
```

```
NAME                        READY     STATUS           RESTARTS   AGE
nginx-6b58d9cdfd-g7lnk      1/1       Running          1          13h
registry-795c6c8b8f-7vwdn   1/1       Running          1          13h
try1-76cc5ffcc6-4rjvh       2/2       Running          0          3s
try1-76cc5ffcc6-bk5f5       2/2       Running          0          3s
try1-76cc5ffcc6-d8n5q       2/2       Running          0          3s
try1-76cc5ffcc6-mm6tw       2/2       Running          0          3s
try1-76cc5ffcc6-r9q5n       2/2       Running          0          3s
try1-76cc5ffcc6-tx4dz       2/2       Running          0          3s
```

14. 特定の Pod の Event を表示します。現在、両方のコンテナが動作しており Pod の状態は良好であるにもかかわらず、Event セクションには問題のみが表示され、ステータスの変化やプローブの成功が表示されていないことに注意してください。

```
student@cp:~/app1$ kubectl describe pod try1-76cc5ffcc6-tx4dz | tail
```

```
Events:
  Type     Reason     Age                   From               Message
  ----     ------     ----                  ----               -------
  Normal   Scheduled  7m46s                 default-scheduler  Successfully assigned
     default/try1-754bf9c75b-vx58x to cp
  Normal   Pulling    7m44s                 kubelet            Pulling image
     "10.97.177.111:5000/simpleapp"
  Normal   Pulled     7m44s                 kubelet            Successfully pulled image
     "10.97.177.111:5000/simpleapp" in 96.710062ms
  Normal   Created    7m43s                 kubelet            Created container simpleapp
  Normal   Started    7m43s                 kubelet            Started container simpleapp
  Warning  Unhealthy  7m6s (x9 over 7m42s)  kubelet            Readiness probe failed:
     cat: /tmp/healthy: No such file or directory
```

15. Pod 内の各コンテナのステータスを見ると、どちらもRunning で Ready 項目はTrue であることがわかるはずです。

```
student@cp:~/app1$ kubectl describe pod try1-76cc5ffcc6-tx4dz | grep -E 'State|Ready'
```

```
    State:          Running
    Ready:          True
    State:          Running
    Ready:          True
  Ready             True
  ContainersReady   True
```

## 課題 3.4: 出題範囲の確認

**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

カリキュラム概要にある CKAD の出題範囲リストを再確認し、この章で取り上げたトピックの参考にしてください。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


- プローブとヘルスチェックの実装
- コンテナイメージの定義、構築、修正
- マルチコンテナ Pod のデザインパターンの理解
- コンテナログの活用

まず、必要なファイルやプロセスをすべて理解することに集中しましょう。YAML サンプルのブックマークを保存し、各ステップを素早く完了できるようになるまで、復習を繰り返します。

出題範囲の項目は、仕事や試験の環境で経験するような、漠然としたものであったり、 詳細でないものであったりします。

1. 試験では許可された 3 つの URL ロケーションを使用して、LivenessProbes、ReadinessProbes、およびマルチコンテナ Pod の YAML 例を見つけてブックマークします。
2. 新しい nginx ウェブサーバーをデプロイします。ポート 80 に LivenessProbe と ReadinessProbe を追加します。両方のプローブとウェブサーバーが動作することをテストします。
3. `build-review1.yaml`ファイルを使用して、動作しない Deployment を作成します。両方のコンテナが Running になり、`READY` 状態になるように Deployment を修正します。Web サーバーはポート 80 でリスンし、プロキシはポート 8080 でリスンします。
4. Web サーバーのデフォルトページを表示させます。成功したら、コンテナログに `GET` のアクティビティログがあることを確認します。メッセージは以下のようなものです。時間や IP アドレスが異なる場合があります。

   ```
   192.168.124.0 - - [30/Jan/2020:03:30:31 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.58.0" "-"
   ```

5. この確認で作成したリソースを削除します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 4 章
# デザイン

## 課題 4.1: Deployment の計画

**Overview**

この演習では、一般的なネットワークプラグインを調査します。各 **kubelet** エージェントは、一度に 1 つのプラグインを使用します。複雑なため、クラスター全体ではアプリケーションのデプロイ前に設定される 1 つのプラグインを使用します。一部のプラグインは、ネットワークポリシーのようなセキュリティ設定を処理しません。ネットワークポリシーを使用するデプロイメントを設計した場合、エラーは発生しませんが、ポリシーは適用されません。開発者は通常、このような仕組みはあまり気にしないでしょうが、新しく分離されたマイクロサービスの機能の可用性とトラブルシューティングに影響を与える可能性があります。

まだ **Container Network Interface** (**CNI**) 仕様は新しいものの、コミュニティはその仕様 (https://github.com/containernetworking/cni) に向かって進んでいます。これは、急速に変化するコンテナネットワーキングの領域において、最良の柔軟性と機能を提供するものです。

一般的な代替案は **kubenet** で、ルーティングとクロスノードネットワーキングの処理をクラウドプロバイダに依存する基本プラグインです。以前の演習では、**Project Calico** を設定しました。クラシックモードと外部モードも可能です。最近、Kubernetes を意図したいくつかのソフトウェア定義ネットワークプロジェクトが作成され、定期的に新機能が追加されています。**Calico** は、継続的かつ頻繁に変更される動的なプロジェクトであることを認識しておいてください。

### ネットワークプラグイン早わかり

開発者がクラスターネットワーキングを設定する必要はありませんが、使用されているプラグインやその特有の機能、癖を理解する必要があるかもしれません。このセクションでは、最も一般的に使用されるプラグインを簡単に確認し、必要に応じてより多くの情報を見つけることができるようにします。

1. ノードが CNI プラグインを使用していることを確認します。CNI の起動プロセスを読み進めましょう。各メッセージは、メッセージの種類とタイムスタンプで始まり、その後にメッセージの詳細が表示されます。**TAB** を使用してノード名にを補完できます。コントローラーとプロキシ Pod の両方を調べます。

```
student@cp:~$ kubectl -n kube-system logs kube-controller-manager-<TAB>
```

```
I0216 17:06:11.729548       1 serving.go:348] Generated self-signed cert in-memory
I0216 17:06:12.318263       1 controllermanager.go:196] Version: v1.26.1
I0216 17:06:12.321279       1 dynamic_cafile_content.go:156] "Starting controller" name=....
I0216 17:06:12.321311       1 secure_serving.go:200] Serving securely on 127.0.0.1:10257
....
```

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~$ kubectl -n kube-system logs kube-proxy-<TAB>
```

```
I0216 17:06:19.299029       1 node.go:163] Successfully retrieved node IP: 10.128.0.67
I0216 17:06:19.299337       1 server_others.go:138] "Detected node IP" address="10.128.0.67"
I0216 17:06:19.299515       1 server_others.go:561] "Unknown proxy mode, assuming iptables proxy"
↪  proxyMode=""
```

2. CNI プロバイダーは数多く存在します。以下のリストは、より一般的な選択肢の一部ですが、すべてを網羅しているわけではありません。多くの新しいプラグインが開発されているため、あなたのニーズをよりよく満たす別のプラグインが存在するかもしれません。これらのウェブサイトを利用して、質問に答えてください。私たちは正確な回答を保つよう努力していますが、この分野には多くの注目と開発があり、頻繁に変化していることを認識してください。
   - **Project Calico**
     https://docs.projectcalico.org/v3.0/introduction/
   - **Calico with Canal**
     https://docs.projectcalico.org/v3.0/getting-started/kubernetes/installation/hosted/canal
   - **Weave Works**
     https://www.weave.works/docs/net/latest/kubernetes/kube-addon
   - **Flannel**
     https://github.com/coreos/flannel
   - **Cilium**
     http://cilium.io/
   - **Kube Router**
     https://www.kube-router.io
3. vxlans を許可するプラグインはどれですか？
4. レイヤー 2 プラグインはどれですか？
5. レイヤー 3 プラグインはどれですか？
6. ネットワークポリシーを許可するものどれですか？
7. すべての TCP と UDP のトラフィックを暗号化できるのはどれですか？

## マルチコンテナ **Pod** に関する考察

本章で学んだ情報をもとに、以下の問いを考えてみてください。

1. 1 つの Pod に複数のアプリケーションを配置する方法と、1 つの Pod に 1 つのアプリケーションを配置する方法とでは、どちらが最も柔軟に対応できるでしょうか？
2. 最もきめ細かいスケーラビリティを実現できる Deployment 方式はどれでしょうか？
3. 一番性能が良いのはどれでしょうか？
4. 1Pod あたり何個の IP アドレスが割り当てられるでしょうか？
5. コンテナが同じ Pod 内で通信する方法には、どのようなものがあるでしょうか？
6. 1 つの Pod に複数のコンテナを用意すべき理由は何でしょうか？


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



**本当に知っていますか?**

いつ、なぜマルチコンテナ Pod を使うのでしょうか?

YAML の例をネットで見つけましたか?

これらの質問に簡単に答えられない場合は、マルチコンテナ Pod のタイプと分離に関するコンテンツに戻り、復習してください。前の章で 2 つ目のロギングとレディネスのコンテナを追加することに触れましたが、今後の演習でロギングについてさらに詳しく説明します。

## 解 4.1

### プラグインの解答

1. vxlans を許可するプラグインはどれですか?

   **Canal、Project Calico、Flannel、Weave Net、Cilium**

2. レイヤー 2 プラグインはどれですか?

   **Canal、Flannel、Weave Net**

3. レイヤー 3 プラグインはどれですか?

   **Project Calico、Romana、Kube Router**

4. ネットワークポリシーを許可するものどれですか?

   **Project Calico、Canal、Kube Router、Weave Net、Cilium**

5. すべての TCP と UDP のトラフィックを暗号化できるのはどれですか?

   **Project Calico、Weave Net、Cilium**

### マルチ **Pod** の解答

1. 1 つの Pod に複数のアプリケーションを配置する方法と、1 つの Pod に 1 つのアプリケーションを配置する方法とでは、どちらが最も柔軟に対応できるでしょうか?

   **1** つの **Pod** に **1** つのアプリケーション

2. 最もきめ細かいスケーラビリティを実現できる Deployment 方式はどれでしょうか?

   **1** つの **Pod** に **1** つのアプリケーション

3. 一番性能が良いのはどれでしょうか?

   **1** つの **Pod** に複数のアプリケーション.

4. 1Pod あたり何個の IP アドレスが割り当てられるでしょうか?

   **1** つ

5. コンテナが同じ Pod 内で通信する方法には、どのようなものがあるでしょうか?

   **IPC**、ループバック、共有ファイルシステムアクセス

6. 1 つの Pod に複数のコンテナを用意すべき理由は何でしょうか?

   無駄のないコンテナには、ロギングなどの機能がない場合があります。無駄のない実行を維持しつつ、**Ambassadors** や **Sidecar** コンテナのように、必要に応じて機能を追加することが可能です。

## 課題 4.2: 期間を考慮したアプリケーションの設計: Job の作成


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


ほとんどのアプリケーションは常に利用できるようにデプロイされますが、中には特定の回数だけ実行するJob や、定期的に実行する CronJob というものもあります。

1. 3 秒間スリープして停止するコンテナを実行する Job を作成します。

```
student@cp:~$ vim job.yaml
```

job.yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: sleepy
spec:
  template:
    spec:
      containers:
      - name: resting
        image: busybox
        command: ["/bin/sleep"]
        args: ["3"]
      restartPolicy: Never
```

2. Job を作成し、ベリファイして詳細を表示します。この例では、Job を 3 秒後にチェックし、完了後に再度チェックすることを示しています。入力の速さによって、異なる出力が表示される場合があります。

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get job
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   0/1           3s         3s
```

```
student@cp:~$ kubectl describe jobs.batch sleepy
```

```
Name:           sleepy
Namespace:      default
Selector:       controller-uid=24c91245-d0fb-11e8-947a-42010a800002
Labels:         controller-uid=24c91245-d0fb-11e8-947a-42010a800002
                job-name=sleepy
Annotations:    <none>
Parallelism:    1
Completions:    1
Start Time:     Sun, 03 Nov 2022  04:22:50 +0000
Completed At:   Sun, 03 Nov 2022  04:22:55 +0000
Duration:       5s
Pods Statuses:  0 Running / 1 Succeeded / 0 Failed
<output_omitted>
```

```
student@cp:~$ kubectl get job
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   1/1           5s         17s
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



3. Job の設定情報を表示します。Job の実行方法に影響を与えるために使用できる 3 つのパラメーターがあります。`-o yaml` を使用してこれらのパラメータを見ることができます。`backoffLimit`、`completions`、そして`parallelism` が確認できます。次にこれらのパラメータを追加します。

```
student@cp:~$ kubectl get jobs.batch sleepy -o yaml
```

```
<output_omitted>
  uid: c2c3a80d-d0fc-11e8-947a-42010a800002
spec:
  backoffLimit: 6
  completions: 1
  parallelism: 1
  selector:
    matchLabels:
<output_omitted>
```

4. Job は完了状態のままで残り、`AGE` が増え続けるので、Job を削除します。

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

5. YAML を編集し、`completions:` パラメータを追加して5 とします。

```
student@cp:~$ vim job.yaml
```

job.yaml

```
<output_omitted>
metadata:
  name: sleepy
spec:
  completions: 5   #<--Add this line
  template:
    spec:
      containers:
<output_omitted>
```

6. Job を再度作成します。Job を表示すると、`COMPLETIONS` が 5つのうち 0 から始まっていることに注意してください。

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get jobs.batch
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   0/5           5s         5s
```

7. 実行中の Pod を表示します。ここでも、タイピングの速度によって出力が異なる場合があります。

```
student@cp:~$ kubectl get pods
```

```
NAME                        READY   STATUS          RESTARTS   AGE
nginx-67f8fb575f-g4468      1/1     Running         2          2d
registry-56cffc98d6-xlhhf   1/1     Running         1          2d
sleepy-z5tnh                0/1     Completed       0          8s
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
sleepy-zd692                1/1     Running          0          3s
<output_omitted>
```

8. 最終的にすべての Job が完了します。検証後、Job を削除してください。

```
student@cp:~$ kubectl get jobs
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   5/5           26s        10m
```

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

9. YAML をもう一度編集します。今回は `parallelism:` パラメータを追加します。一度に 2 つの Pod がデプロイされるように、2 に設定します。

```
student@cp:~$ vim job.yaml
```

job.yaml

```
<output_omitted>
  name: sleepy
spec:
  completions: 5
  parallelism: 2   #<-- Add this line
  template:
    spec:
<output_omitted>
```

10. Job を再度作成します。5 つすべてが完了するまで、Pod が 2 つずつデプロイされるのが見えるはずです。

```
student@cp:~$ kubectl create -f job.yaml
```

```
student@cp:~$ kubectl get pods
```

```
NAME                        READY   STATUS    RESTARTS   AGE
nginx-67f8fb575f-g4468      1/1     Running   2          2d
registry-56cffc98d6-xlhhf   1/1     Running   1          2d
sleepy-8xwpc                1/1     Running   0          5s
sleepy-xjqnf                1/1     Running   0          5s
try1-c9cb54f5d-b45gl        2/2     Running   0          8h
<output_omitted>
```

```
student@cp:~$ kubectl get jobs
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   3/5           11s        11s
```

11. 一定秒数後に Job を停止させるパラメータを追加します。`activeDeadlineSeconds:` を15 に設定します。15 秒間実行されると、Job とすべての Pod が終了します。

```
student@cp:~$ vim job.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



job.yaml

```yaml
<output_omitted>
  completions: 5
  parallelism: 2
  activeDeadlineSeconds: 15   #<-- Add this line
  template:
    spec:
      containers:
      - name: resting
        image: busybox
        command: ["/bin/sleep"]
        args: ["3"]
<output_omitted>
```

12. Job を削除し、再度作成します。4 回実行された後、それ以上完了することなくエージングが継続されるはずです。

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```

```
student@cp:~$ kubectl create -f job.yaml
```

```
job.batch/sleepy created
```

```
student@cp:~$ kubectl get jobs
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   2/5           6s         6s
```

```
student@cp:~$ kubectl get jobs
```

```
NAME     COMPLETIONS   DURATION   AGE
sleepy   4/5           16s        16s
```

13. オブジェクトの YAML 出力の`Status` セクションにある `message:` エントリを表示します。Job がまだ実行されていない場合、より少ない`status` が表示されるかもしれません。そうであれば、待ってもう一度試してみてください。

```
student@cp:~$ kubectl get job sleepy -o yaml
```

```
<output_omitted>
status:
  conditions:
  - lastProbeTime: "2022-11-03T16:06:10Z"
    lastTransitionTime: "2022-11-03T16:06:10Z"
    message: Job was active longer than specified deadline
    reason: DeadlineExceeded
    status: "True"
    type: Failed
  failed: 1
  startTime: "2022-11-03T16:05:55Z"
  succeeded: 4
```

14. Job を削除します。

```
student@cp:~$ kubectl delete jobs.batch sleepy
```

```
job.batch "sleepy" deleted
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


## 課題 4.3: 期間を考慮したアプリケーションの設計: CronJob の作成

CronJob は、時間が来たときにあなたの代わりにバッチ Job を作成する監視ループを作成します。ここでは、既存の Job ファイルを使用して開始することにします。

1. Job ファイルを新規ファイルにコピーします。

```
student@cp:~$ cp job.yaml cronjob.yaml
```

2. 以下に示す注釈付きファイルのように編集してください。

```
student@cp:~$ vim cronjob.yaml
```

cronjob.yaml

```yaml
apiVersion: batch/v1
kind: CronJob   #<-- Change this line
metadata:
  name: sleepy
spec:                        #<-- Remove completions:, parallelism:, and activeDeadlineSeconds:
  schedule: "*/2 * * * *"    #<-- Add Linux style cronjob syntax
  jobTemplate:               #<-- New jobTemplate and spec
    spec:
      template:   #<-- This and following lines space four to right
        spec:
          containers:
          - name: resting
            image: busybox
            command: ["/bin/sleep"]
            args: ["3"]
          restartPolicy: Never
```

3. 新しいCronJob を作成します。Job を表示します。CronJob が実行され、新しいバッチJob が生成されるまで、2 分かかります。

```
student@cp:~$ kubectl create -f cronjob.yaml
```

```
cronjob.batch/sleepy created
```

```
student@cp:~$ kubectl get cronjobs.batch
```

```
NAME     SCHEDULE      SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sleepy   */2 * * * *   False     0        <none>          8s
```

```
student@cp:~$ kubectl get job
```

```
No resources found in default namespace.
```

4. 2 分後、Job の実行が開始されるのが確認できるはずです。

```
student@cp:~$ kubectl get cronjobs.batch
```

```
NAME     SCHEDULE      SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sleepy   */2 * * * *   False     0        21s             2m1s
```

```
student@cp:~$ kubectl get jobs.batch
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
NAME                COMPLETIONS   DURATION   AGE
sleepy-1539722040   1/1           5s         18s
```

```
student@cp:~$ kubectl get jobs.batch
```

```
NAME                COMPLETIONS   DURATION   AGE
sleepy-1539722040   1/1           5s         5m17s
sleepy-1539722160   1/1           6s         3m17s
sleepy-1539722280   1/1           6s         77s
```

5. Job が 10 秒以上継続すると終了するようにします。まず、**sleep** コマンドを 30 秒間実行するように編集し、activeDeadlineSeconds: エントリをコンテナに追加します。

```
student@cp:~$ vim cronjob.yaml
```

cronjob.yaml

```yaml
....
  jobTemplate:
    spec:
      template:
        spec:
          activeDeadlineSeconds: 10  #<-- Add this line
          containers:
          - name: resting
....
            command: ["/bin/sleep"]
            args: ["30"]             #<-- Edit this line
          restartPolicy: Never
```

6. CronJob を削除して再作成します。タイマーにより、バッチJob の作成と終了に数分かかる場合があります。

```
student@cp:~$ kubectl delete cronjobs.batch sleepy
```

```
cronjob.batch "sleepy" deleted
```

```
student@cp:~$ kubectl create -f cronjob.yaml
```

```
cronjob.batch/sleepy created
```

```
student@cp:~$ sleep 120 ; kubectl get jobs
```

```
NAME                COMPLETIONS   DURATION   AGE
sleepy-1539723240   0/1           61s        61s
```

```
student@cp:~$ kubectl get cronjobs.batch
```

```
NAME     SCHEDULE      SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sleepy   */2 * * * *   False     1        72s             94s
```

```
student@cp:~$ kubectl get jobs
```

```
NAME                COMPLETIONS   DURATION   AGE
sleepy-1539723240   0/1           75s        75s
```

```
student@cp:~$ kubectl get jobs
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
NAME                COMPLETIONS   DURATION   AGE
sleepy-1539723240   0/1           2m19s      2m19s
sleepy-1539723360   0/1           19s        19s
```

```
student@cp:~$ kubectl get cronjobs.batch
```

```
NAME     SCHEDULE      SUSPEND   ACTIVE   LAST SCHEDULE   AGE
sleepy   */2 * * * *   False     2        31s             2m53s
```

7. CronJob を削除してクリーンアップします。

```
student@cp:~$  kubectl delete cronjobs.batch sleepy
```

```
cronjob.batch "sleepy" deleted
```

## 課題 4.4: Label の使用

Label を作成し、操作します。Deployment、ReplicaSet、Pod の Label の相互作用を理解しましょう。

1. design2 という新しい Deployment を作成します。

```
student@cp:~$ kubectl create deployment design2 --image=nginx
```

```
deployment.apps/design2 created
```

2. design2Deployment のワイドな **kubectl get** 出力を表示し、SELECTOR をメモしておきます。

```
student@cp:~$ kubectl get deployments.apps design2 -o wide
```

```
NAME     READY  UP-TO-DATE  AVAILABLE  AGE    CONTAINERS  IMAGES  SELECTOR
design2  1/1    1           1          2m13s  nginx       nginx   app=design2
```

3. **-l** オプションを使用すると、セレクタを使用して Deployment 内で実行されている Pod を一覧表示します。実行中の Pod は 1 つだけのはずです。

```
student@cp:~$ kubectl get -l app=design2 pod
```

```
NAME                        READY   STATUS    RESTARTS   AGE
design2-766d48574f-5w274    1/1     Running   0          3m1s
```

4. Deployment のセレクタを使用して、Pod の詳細を YAML 形式で表示します。今回は **–selector** オプションを使用します。出力される Pod の Label を探します。Deployment のものと一致するはずです。

```
student@cp:~$ kubectl get --selector app=design2 pod  -o yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cni.projectcalico.org/podIP: 192.168.113.222/32
  creationTimestamp: "2020-01-31T16:29:37Z"
  generateName: design2-766d48574f-
  labels:
    app: design2
    pod-template-hash: 766d48574f
....
```

5. Pod の Label を好きな色に編集してください。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
student@cp:~$ kubectl edit pod design2-766d48574f-5w274
```

```yaml
....
  labels:
    app: orange                        #<<-- Edit this line
    pod-template-hash: 766d48574f
  name: design2-766d48574f-5w274
....
```

6. 次に、Deployment に含まれる Pod の数を表示します。次に、名前に `design2` が含まれているものがいくつあるか確認します。Pod の `AGE` を確認してください。

```
student@cp:~$ kubectl get deployments.apps design2 -o wide
```

```
NAME     READY  UP-TO-DATE  AVAILABLE  AGE  CONTAINERS  IMAGES  SELECTOR
design2  1/1    1           1          56s  nginx       nginx   app=design2
```

```
student@cp:~$ kubectl get pods | grep design2
```

```
design2-766d48574f-5w274   1/1     Running            0          82s
design2-766d48574f-xttgg   1/1     Running            0          2m12s
```

7. `design2` の Deployment を削除します。

```
student@cp:~$ kubectl delete deploy design2
```

```
deployment.apps "design2" deleted
```

8. 名前に`design2` が含まれる Pod をもう一度確認します。最初に Deployment を作成したときの`AGE` を持つ Pod が 1 つ見つかるはずです。Label が編集されると、ステータスが仕様と一致し、意図した Label で動作するレプリカが存在するように、Deployment は新しい Pod を作成します。

```
student@cp:~$ kubectl get pods | grep design2
```

```
design2-766d48574f-5w274  1/1    Running           0          38m
```

9. 前のステップで好きな色に編集した Label と**-l** を使って Pod を削除します。コマンドの詳細は省略されています。前のステップでこれらのコマンドを把握しておいてください。

## 課題 4.5: Pod のリソースの制限と要件の設定

1. `vish/stress` イメージを実行する新しい Pod を作成します。YAML `stress.yaml` ファイルは、コースの tarball に含まれています。

2. cp ノードとワーカーノードで **top** コマンドを実行します。1 つのワーカーノードの CPU の大部分を消費している`stress` コマンドが見つかるはずです。**q** を使用して top を終了します。そして Deployment を削除します。

3. `stress.yaml`ファイルを編集し、以下の制限と要求を追加します。

```
student@cp:~$
```

```yaml
....
        name: stressmeout
        resources:                 #<<-- Add this and following six lines
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
            limits:
              cpu: "1"
              memory: "1Gi"
            requests:
              cpu: "0.5"
              memory: "500Mi"
          args:
          - -cpus
  ....
```

4. Deployment を再度作成して、Pod のステータスを確認します。`OOMKilled` ステータスが表示され、再起動の回数が増加していることが確認できるはずです。再起動の初期に Pod を参照すると、ステータスが`Running` になることがあります。十分な時間待つと、`CrashLoopBackOff` が表示されるかもしれません。

```
student@cp:~$ kubectl get pod stressmeout-7fbbbcc887-v9kvb
```

```
NAME                            READY   STATUS      RESTARTS   AGE
stressmeout-7fbbbcc887-v9kvb    0/1     OOMKilled   2          32s
```

5. Deployment を削除し、編集します。`limits:` パラメータを変更して、Pod が実行できるようにしますが、余分なリソースが増え過ぎないようにします。メモリ制限を stress コマンドで要求される値に正確に設定してみてください。

   cp ノードでの Pod の実行を許可しているため、リソース不足で `kube-apiserver` が再起動するなどの問題が発生する可能性があります。そのため、`kubernetes.io/hostname` の内蔵の Label を使用するように `nodeSelector` を追加します。

```
student@cp:~$ kubectl delete -f stress.yaml

student@cp:~$ vim stress.yaml
```

```
  ....
      spec:
        nodeSelector:                      #<-- Uncomment and edit
          kubernetes.io/hostname: worker   #<-- to by YOUR worker hostname
        containers:

  ....
          resources:
            limits:
              cpu: "2"
              memory: "2Gi"
            requests:
  ....
```

6. Deployment を作成し、Pod がエラーなく実行されることを確認します。**top** を使用して、stress コマンドがノードの 1 つで実行されていることを確認し、Pod の詳細を表示して、CPU とメモリの制限が使用されていることを確認します。また、**kubectl describe node** コマンドを使用して、cp とワーカーノードが使用しているリソースを表示します。コマンドの詳細は省略されています。これまでの手順を使って、それぞれのコマンドの詳細を把握するようにしてください。

7. `limits` や `requests` をノードのリソースよりも高い数値に変更し、コンテナや Pod の処理方法を評価します。リソースが完全に割り当てられるまで、しばらく時間がかかる場合があります。

8. 終了後、`stressmeout` の Deployment を削除します。

```
student@cp:~$ kubectl delete deploy stressmeout
```

```
deployment.apps "stressmeout" deleted
```

## 課題 4.6: シンプルな initContainer


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



1. Pod の作成についてはすでに学習済みであるため、この演習では特に手順を説明しません。以前の内容を復習する必要があるかどうかを判断するために使用することができます。コースの tarball にも含まれている次の YAML を使用して、initContainer を持つ Pod を作成します。それは失敗し、どのように見えるかを表示し、エラーを調査します。コマンドを次のように変更します。**/bin/false** を **/bin/true** などの成功するコマンドに変更して、再度 Pod を作成します。

init-tester.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-tester
  labels:
    app: inittest
spec:
  containers:
  - name: webservice
    image: nginx
  initContainers:
  - name: failed
    image: busybox
    command: [/bin/false]
```

## 課題 4.7: CustomResourceDefinition の探索

1. 現在クラスター内にある CRD を表示します

```
student@cp:~$ kubectl get crd
```

```
AME                                                    CREATED AT
bgpconfigurations.crd.projectcalico.org                 2021-10-13T04:57:12Z
bgppeers.crd.projectcalico.org                          2021-10-13T04:57:12Z
blockaffinities.crd.projectcalico.org                   2021-10-13T04:57:12Z
clusterinformations.crd.projectcalico.org               2021-10-13T04:57:12Z
felixconfigurations.crd.projectcalico.org               2021-10-13T04:57:12Z
globalnetworkpolicies.crd.projectcalico.org             2021-10-13T04:57:13Z
globalnetworksets.crd.projectcalico.org                 2021-10-13T04:57:13Z
hostendpoints.crd.projectcalico.org                     2021-10-13T04:57:13Z
ipamblocks.crd.projectcalico.org                        2021-10-13T04:57:13Z
ipamconfigs.crd.projectcalico.org                       2021-10-13T04:57:13Z
ipamhandles.crd.projectcalico.org                       2021-10-13T04:57:13Z
ippools.crd.projectcalico.org                           2021-10-13T04:57:13Z
kubecontrollersconfigurations.crd.projectcalico.org     2021-10-13T04:57:14Z
networkpolicies.crd.projectcalico.org                   2021-10-13T04:57:14Z
networksets.crd.projectcalico.org                       2021-10-13T04:57:14Z
serviceprofiles.linkerd.io                              2021-10-13T05:00:28Z
trafficsplits.split.smi-spec.io                         2021-10-13T05:00:28Z
```

2. IPPools CRD を詳しく見てみます。

```
student@cp:~$ kubectl get crd ippools.crd.projectcalico.org -o yaml
```

```
....
spec:
  conversion:
    strategy: None
  group: crd.projectcalico.org
  names:
    kind: IPPool
    listKind: IPPoolList
    plural: ippools
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
      singular: ippool
    scope: Cluster
    versions:
    - name: v1
  ....
```

3. この新しく宣言されたリソースが存在するかどうかを表示します。

```
student@cp:~$ kubectl get IPPool
```

```
NAME                  AGE
default-ipv4-ippool   11h
```

4. リソースの詳細なビューを表示します。IP 情報は、クラスターが使用しているエフェメラル Pod の IP と一致する必要があることに注意してください。

```
student@cp:~$ kubectl get IPPool -o yaml
```

```
  ....
      uid: 0640c9e5-25f0-40f8-9c36-1d9be26a2516
    spec:
      blockSize: 26
      cidr: 192.168.0.0/16
      ipipMode: Always
      natOutgoing: true
      nodeSelector: all()
      vxlanMode: Never
  kind: List
  metadata:
  ....
```

## 課題 4.8: 出題範囲の確認

**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

カリキュラム概要にある CKAD の出題範囲リストを再確認し、この章で取り上げたトピックの参考にしてください。

- Job と CronJob を理解する
- リソースの要件、制限、Quota の把握と定義
- マルチコンテナ Pod のデザインパターンを理解する（例 sidecar **init**、他）
- Kubernetes を拡張するリソースを発見し、利用する（CRD）。

まず、必要なファイルやプロセスをすべて理解することに集中しましょう。必要な YAML サンプルのブックマークがあり、各ステップを素早く完了し、各オブジェクトが適切に実行されていることを確認できるまで、復習を繰り返します。

1. `design-review1.yaml`ファイルを見つけて使用し、Pod を作成します。
2. `design-pod1` の CPU とメモリのリソース要件を決定します。
3. CPU の制限がコンテナが要求する量のちょうど 2 倍になるように、Pod のリソース要件を編集します。(ヒント 0.22 を引く)


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



4. Pod が`Running` ステータスを示すまで、Pod のメモリリソースの制限を増やします。これには、複数の編集と試行が必要な場合があります。`Running` ステータスが 1 分以上持続するために必要な最小量を決定してください。
5. `design-review2.yaml`ファイルを使用して、さまざまな Label を持つ複数の Pod を作成します。
6. –selector の値`tux` のみを使用して、それらの Pod のみを削除します。これは前のステップで作成した複数の Pod の半分になるはずです。ヒントは、キーの値を決定するために Pod の設定を表示する必要があります。
7. `busybox` と`sleep 30` コマンドを実行する新しい CronJob を作成します。CronJob を 3 分ごとに実行させ、Job のステータスを表示し、作業を確認します。毎週、現在の時刻から 10 分後に Pod が実行されるように設定を変更します。例えば、現在の時刻が午後 2 時 14 分だった場合、毎週月曜日の午後 2 時 24 分に実行するように設定します。
8. このレビュー中に作成されたオブジェクトを削除します。10 分後に実行されるかどうかを確認したい場合は、CronJob 以外を削除するとよいでしょう。その後、そのオブジェクトも削除してください。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 5 章

# Deployment 設定

## 課題 5.1: Deployment を構成する: ConfigMap

**重要**

演習の一部を繰り返したい場合や、タイプミスや空白の問題でファイルが使いにくい場合に備えて、`$HOME/app1/simpleapp.yaml`ファイルのコピーを保存しておいてください。

```
student@cp:~$ cp $HOME/app1/simpleapp.yaml $HOME/beforeLab5.yaml
```

セキュリティ編の演習で **Secret** の使い方を説明します。

**概要**

この演習では、本番で必要となる可能性のある設定を行い、Deployment にリソースを追加します。

**ConfigMap** がデータを取り込む方法には、リテラル値から、ファイルから、ファイルのディレクトリからという 3 つの方法があります。

1. 原色の情報を含む **ConfigMap** を作成します。**ConfigMap** に取り込むための一連のファイルを作成することにします。まず、`primary` というディレクトリを作成し、そこに 4 つのファイルを入れます。次に、ホームディレクトリに好きな色のファイルを作成します。

```
student@cp:~/app1$ cd

student@cp:~$ mkdir primary
student@cp:~$ echo c > primary/cyan
student@cp:~$ echo m > primary/magenta
student@cp:~$ echo y > primary/yellow
student@cp:~$ echo k > primary/black
student@cp:~$ echo "known as key" >> primary/black
student@cp:~$ echo blue > favorite
```

2. 3 つの方法それぞれで **ConfigMap** を生成します。

```
student@cp:~$ kubectl create configmap colors \
  --from-literal=text=black \
  --from-file=./favorite \
  --from-file=./primary/
```

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
configmap/colors created
```

3. 新しく作成された **ConfigMap** を表示します。取り込まれたデータの表示方法に注目してください。

```
student@cp:~$ kubectl get configmap colors
```

```
NAME      DATA      AGE
colors    6         11s
```

```
student@cp:~$ kubectl get configmap colors -o yaml
```

```
apiVersion: v1
data:
  black: |
    k
    known as key
  cyan: |
    c
  favorite: |
    blue
  magenta: |
    m
  text: black
  yellow: |
    y
kind: ConfigMap
metadata:
<output_omitted>
```

4. アプリケーションの YAML ファイルを更新して、**ConfigMap** を環境パラメータとして利用できるようにします。`env:` の行から `key:favorites` までの 6 行を追加します。

```
student@cp:~$ vim $HOME/app1/simpleapp.yaml
```

**simpleapp.yaml**

```
....
    spec:
      containers:
      - image: 10.105.119.236:5000/simpleapp
        env:^^I^^I^^I^^I#<-- Add from here
        - name: ilike
          valueFrom:
            configMapKeyRef:
              name: colors
              key: favorite  ^^I^^I#<-- to here
        imagePullPolicy: Always
....
```

5. Deployment を削除し、新しいパラメータで再作成します。

```
student@cp-lab-7xtx:~$ kubectl delete deployment try1
```

```
deployment.apps "try1" deleted
```

```
student@cp-lab-7xtx:~$ kubectl create -f $HOME/app1/simpleapp.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
deployment.apps/try1 created
```

6. try1 Pod は完全に準備ができている状態ではないにもかかわらず、実行されており、有用です。**kubectl exec** を使用して、変数の値を表示します。Pod の状態を表示し、simpleapp コンテナ内でilike の値を見ることができるかどうかを確認します。なお、ダブルダッシュ(- -)の使用は、シェルに以下を標準入力として渡すように指示します。

```
student@cp:~$ kubectl get pod
```

```
<output_omitted>
```

```
student@cp:~$ kubectl exec -c simpleapp -it try1-5db9bc6f85-whxbf \
    -- /bin/bash -c 'echo $ilike'
```

```
blue
```

7. YAML ファイルをもう一度編集し、今度は **ConfigMap** を使う別の方法を追加します。envFrom はenv と同じインデント、configMapRef はconfigMapKeyRef と同じインデントで、3 行を追加します。

```
student@cp:~$ vim $HOME/app1/simpleapp.yaml
```

simpleapp.yaml

```
....
            configMapKeyRef:
               name: colors
               key: favorite
         envFrom:^^I      #<-- Add this and the following two lines
         - configMapRef:
             name: colors
         imagePullPolicy: Always
....
```

8. もう一度 Deployment を削除し、再作成します。Pod の再起動を確認します。

```
student@cp:~$ kubectl delete deployment try1
```

```
deployment.apps "try1" deleted
```

```
student@cp:~$ kubectl create -f $HOME/app1/simpleapp.yaml
```

```
deployment.apps/try1 created
```

```
student@cp:~$ kubectl get pods
```

```
NAME                      READY   STATUS        RESTARTS   AGE
nginx-6b58d9cdfd-9fnl4    1/1     Running       1          23h
registry-795c6c8b8f-hl5w  1/1     Running       2          23h
try1-d4fbf76fd-46pkb      1/2     Running       0          40s
try1-d4fbf76fd-9kw24      1/2     Running       0          39s
try1-d4fbf76fd-bx9j9      1/2     Running       0          39s
try1-d4fbf76fd-jw8g7      1/2     Running       0          40s
try1-d4fbf76fd-lppl5      1/2     Running       0          39s
try1-d4fbf76fd-xtfd4      1/2     Running       0          40s
```

9. Pod のtry1 コンテナ内の設定を表示します。以下の出力は、数カ所で切り捨てられています。コンテナ名を省略して挙動を観察します。また、最初にコンテナにログインするのではなく、すべての環境変数を確認するためにコマンドを実行します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~$ kubectl exec -it try1-d4fbf76fd-46pkb -- /bin/bash -c 'env'
```

```
Defaulting container name to simpleapp.
Use 'kubectl describe pod/try1-d4fbf76fd-46pkb -n default' to see all of the containers in this pod.
REGISTRY_PORT_5000_TCP_ADDR=10.105.119.236
HOSTNAME=try1-d4fbf76fd-46pkb
TERM=xterm
yellow=y
<output_omitted>
REGISTRY_SERVICE_HOST=10.105.119.236
KUBERNETES_SERVICE_PORT=443
REGISTRY_PORT_5000_TCP=tcp://10.105.119.236:5000
KUBERNETES_SERVICE_HOST=10.96.0.1
text=black
REGISTRY_SERVICE_PORT_5000=5000
<output_omitted>
black=k
known as key

<output_omitted>
ilike=blue
<output_omitted>
magenta=m

cyan=c
<output_omitted>
```

10. 柔軟性と拡張性を高めるために、**ConfigMap** は YAML ファイルから作成し、必要に応じてデプロイと再デプロイを行うことができます。クラスターに取り込まれたデータは、他のオブジェクトと同じ方法で取得することができます。今度は YAML ファイルから別の **ConfigMap** を作成します。

```
student@cp:~$ vim car-map.yaml
```

car-map.yaml

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: fast-car
  namespace: default
data:
  car.make: Ford
  car.model: Mustang
  car.trim: Shelby
```

```
student@cp:~$ kubectl create -f car-map.yaml
```

```
configmap/fast-car created
```

11. 取り込まれたデータを表示し、作成されたファイルと同じように出力されることに注意してください。

```
student@cp:~$ kubectl get configmap fast-car -o yaml
```

```
apiVersion: v1
data:
  car.make: Ford
  car.model: Mustang
  car.trim: Shelby
kind: ConfigMap
metadata:
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
<output_omitted>
```

12. simpleapp.yamlファイルに **ConfigMap** の設定をボリュームとして追加します。try1 Deployment の両方のコンテナは、volumeMounts ステートメントを使用して、同じボリュームにアクセスできます。volume 節は containers 節と同じ深さであり、コンテナが宣言された後に来るべきであることを忘れないでください。下の例では、status: の直前にボリュームが追加されています。

```
student@cp:~$ vim $HOME/app1/simpleapp.yaml
```

simpleapp.yaml

```yaml
....
    spec:
      containers:
      - image: 10.105.119.236:5000/simpleapp
        volumeMounts:             #<-- Add this and following two lines
        - mountPath: /etc/cars
          name: car-vol
        env:
        - name: ilike
....
      securityContext: {}
      terminationGracePeriodSeconds: 30
      volumes:                    #<-- Add this and following four lines
      - name: car-vol
        configMap:
          defaultMode: 420
          name: fast-car
status:
....
```

13. Deployment を削除して再作成します。

```
student@cp:~$ kubectl delete deployment try1
```

```
deployment.apps "try1" deleted
```

```
student@cp:~$ kubectl create -f $HOME/app1/simpleapp.yaml
```

```
deployment.apps/try1 created
```

14. Deployment が実行されていることを確認します。コンテナ内の /tmp/healthy ファイルの作成をまだ自動化していないため、**for** ループを使用してファイルを作成するまで AVAILABLE カウントがゼロのままであることに注意してください。次のステップでこれを改善します。

```
student@cp:~$ kubectl get deployment
```

```
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
nginx      1/1     1            1           1d
registry   1/1     1            1           1d
try1       0/6     6            0           39s
```

15. ここまででヘルスチェックはコマンド実行の成功を確認していました。既存のreadinessProbe のコマンドを編集して、マウントされた ConfigMap ファイルの存在を確認し、Deployment を再作成することにします。1 分後には、Deployment のそれぞれの Pod に対して両方のコンテナが利用可能になるはずです。必ず、goproxy セクションではなく、simpleapp セクションを編集してください。

```
student@cp:~$ kubectl delete deployment try1
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
deployment.apps "try1" deleted
```

```
student@cp:~$ vim $HOME/app1/simpleapp.yaml
```

simpleapp.yaml

```
....
        readinessProbe:
          exec:
            command:
            - ls^^I^^I^^I#<-- Add/Edit this and following line.
            - /etc/cars
          periodSeconds: 5
....
```

```
student@cp:~$ kubectl create -f $HOME/app1/simpleapp.yaml
```

```
deployment.apps/try1 created
```

16. 約 1 分待って、Deployment と Pod を表示します。6 つのレプリカがすべて実行され、2/2 のコンテナが内部で準備完了状態にあることが報告されているはずです。

```
student@cp:~$ kubectl get deployment
```

```
NAME       READY   UP-TO-DATE   AVAILABLE   AGE
nginx      1/1     1            1           1d
registry   1/1     1            1           1d
try1       6/6     6            6           1m
```

```
student@cp:~$ kubectl get pods
```

```
NAME                        READY     STATUS    RESTARTS   AGE
nginx-6b58d9cdfd-9fnl4      1/1       Running   1          1d
registry-795c6c8b8f-hl5wf   1/1       Running   2          1d
try1-7865dcb948-2dzc8       2/2       Running   0          1m
try1-7865dcb948-7fkh7       2/2       Running   0          1m
try1-7865dcb948-d85bc       2/2       Running   0          1m
try1-7865dcb948-djrcj       2/2       Running   0          1m
try1-7865dcb948-kwlv8       2/2       Running   0          1m
try1-7865dcb948-stb2n       2/2       Running   0          1m
```

17. コンテナにマウントされた新しいボリューム内のファイルを表示します。ConfigMap 内に作成したデータと一致するはずです。ファイルには carriage-return がないため、次のプロンプトの前に表示されます。

```
student@cp:~$ kubectl exec -c simpleapp -it try1-7865dcb948-stb2n \
    -- /bin/bash -c 'cat /etc/cars/car.trim'
```

```
Shelby student@cp:~$
```

## 課題 5.2: Deployment を構成する: ストレージのアタッチ

Kubernetes でアクセスできるストレージにはいくつかの種類があり、ストレージの柔軟性はスケーラビリティに不可欠です。この演習では、NFS サーバーを構成します。NFS サーバーでは、新しい **PersistentVolume (pv)** と、それを使用する **PersistentVolumeClaim (pvc)** を作成することになります。

1. pv と pvc の YAML サンプルファイルを http://kubernetes.io/docs と http://kubernetes.io/blog で検索してください。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



2. cp ノードで NFS をセットアップするには、tar ボールの `CreateNFS.sh`スクリプトを使用します。このスクリプトは、サーバーの設定、エクスポート`/opt/sfw`を作成し、`/opt/sfw/hello.txt`というファイルを作成します。tar ファイルを解凍した場所を覚えていない場合は、**find** コマンドでファイルの場所を特定します。この例では、`$HOME` ディレクトリに絞り込んで検索しています。環境に応じて変更してください。tarfile の複数のサブディレクトリに同じファイルがある場合があります。

```
student@cp:~$ find $HOME -name CreateNFS.sh
```

```
<some_path>/CreateNFS.sh
```

```
student@cp:~$ cp <path_from_output_above>/CreateNFS.sh $HOME

student@cp:~$ bash $HOME/CreateNFS.sh
```

```
Hit:1 http://us-central1.gce.archive.ubuntu.com/ubuntu xenial InRelease
Get:2 http://us-central1.gce.archive.ubuntu.com/ubuntu xenial-updates InRelease [102 kB]

<output_omitted>

Should be ready. Test here and second node

Export list for localhost:
/opt/sfw *
```

3. **2** 台目のノードからリソースをマウントしてテストします。クライアントソフトウェアをインストールすることから始めます。

```
student@worker:~$ sudo apt-get -y install nfs-common nfs-kernel-server
```

```
<output_omitted>
```

4. テストでは、2 番目のノードから **showmount** を使用してエクスポートされたディレクトリを確認することができます。

```
student@worker:~$ showmount -e cp #<-- Edit to be first node' s name or IP
```

```
Export list for cp:
/opt/sfw *
```

5. ディレクトリをマウントします。`/etc/fstab`を編集しない限り、永続的なマウントにはならないことに注意してください。指定するノード名は cp ノードの名前に変更してください。

```
student@worker:~$ sudo mount cp:/opt/sfw /mnt
```

6. スクリプトで作成された`hello.txt`ファイルが表示できることを確認します。

```
student@worker:~$ ls -l /mnt
```

```
total 4
-rw-r--r-- 1 root root 9 Sep 28 17:55 hello.txt
```

7. cp ノードに戻り、**PersistentVolume** の種類を持つオブジェクトの YAML ファイルを作成します。付属のサンプルファイルでは、`server:` パラメータを編集する必要があります。cp サーバーのホスト名と、前のステップで作成したディレクトリを使用します。構文のみがチェックされるので、名前やディレクトリが正しくなくてもエラーにはなりませんが、間違ったリソースを使用した Pod は起動しません。なお、`accessModes` は現在のところ実際のアクセスには影響しないので、通常は Label として使用します。

```
student@cp:~$ find $HOME -name PVol.yaml
```

```
<some_long_path>/PVol.yaml
```

```
student@cp:~$ cp <path_output_from_above>/PVol.yaml $HOME

student@cp:~$ vim PVol.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


PVol.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pvvol-1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  nfs:
    path: /opt/sfw
    server: cp                        #<-- Edit to match cp node name or IP
    readOnly: false
```

8. **pvvol-1** という名前の新しい 1Gi ボリュームを作成し、確認します。ステータスが`Available` と表示されていることに注意してください。先ほど、イメージレジストリ用に 2 つの永続ボリュームを作成したことを思い出してください。

```
student@cp:~$ kubectl create -f PVol.yaml
```

```
persistentvolume/pvvol-1 created
```

```
student@cp:~$ kubectl get pv
```

```
NAME             CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      CLAIM   STORAGECLASS  REASON
↪  AGE
pvvol-1          1Gi        RWX            Retain           Available
↪  4s
registryvm       200Mi      RWO            Retain           Bound       default/nginx-claim0
↪  4d
task-pv-volume   200Mi      RWO            Retain           Bound       default/registry-claim0
↪  4d
```

9. 新しいボリュームができたので、それを Pod で使用するために **PersistentVolumeClaim (pvc)** を使用します。ローカルレジストリから 2 つの既存の要求があるはずです。

```
student@cp:~/$ kubectl get pvc
```

```
NAME              STATUS   VOLUME           CAPACITY   ACCESS MODES   STORAGECLASS   AGE
nginx-claim0      Bound    registryvm       200Mi      RWO                           4d
registry-claim0   Bound    task-pv-volume   200Mi      RWO                           4d
```

10. **PersistentVolumeClaim** という種類の yaml ファイルを作成またはコピーします。

```
student@cp:~$ vim pvc.yaml
```

pvc.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-one
spec:
  accessModes:
  - ReadWriteMany
  resources:
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
9       requests:
10        storage: 200Mi
```

11. 新しい pvc を作成し、ステータスがbound であることを確認します。200Mi が提案されたにもかかわらず、サイズが1Gi であることに注意してください。少なくともそのサイズのボリュームしか使用できないため、少なくとも指定したサイズのスペースがある最初のボリュームが選択されました。

```
student@cp:~$ kubectl create -f pvc.yaml
```

```
persistentvolumeclaim/pvc-one created
```

```
student@cp:~$ kubectl get pvc
```

```
NAME           STATUS    VOLUME       CAPACITY   ACCESS MODES   STORAGECLASS   AGE
nginx-claim0      Bound     registryvm       200Mi      RWO                           4d
pvc-one           Bound     pvvol-1          1Gi        RWX                           4s
registry-claim0   Bound     task-pv-volume   200Mi      RWO                           4d
```

12. 次に、物理ボリュームのステータスを見ます。bound と表示されるはずです。

```
student@cp:~$ kubectl get pv
```

```
NAME           CAPACITY ACCESS MODES RECLAIM POLICY STATUS
 CLAIM       STORAGECLASS   REASON    AGE
pvvol-1        1Gi      RWX           Retain         Bound
 default/pvc-one                      14m
registryvm     200Mi    RWO           Retain         Bound
 default/nginx-claim0                 4d
task-pv-volume 200Mi    RWO           Retain         Bound
 default/registry-claim0              4d
```

13. simpleapp.yaml ファイルを編集して、2 つの新しいセクションを追加します。1 つはボリュームマウントポイントを使用するコンテナのためのセクションで、既にcar-vol というエントリがあるはずです。もう 1 つのセクションは、Deployment 全般にボリュームを追加するもので、configMap ボリュームセクションの後に置くことができます。

```
student@cp:~$ vim $HOME/app1/simpleapp.yaml
```

**simpleapp.yaml**

```
1  ....
2          volumeMounts:
3          - name: car-vol
4            mountPath: /etc/cars
5          - name: nfs-vol                    #<-- Add this and following line
6            mountPath: /opt
7  ....
8        volumes:
9         - name: car-vol
10          configMap:
11            defaultMode: 420
12            name: fast-car
13        - name: nfs-vol                    #<-- Add this and following two lines
14          persistentVolumeClaim:
15            claimName: pvc-one
16 status:
17 ....
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


14. Deployment を削除して再作成します。

```
student@cp:~$ kubectl delete deployment try1 ; kubectl create -f $HOME/app1/simpleapp.yaml
```

```
deployment.apps "try1" deleted
deployment.apps/try1 created
```

15. Deployment 内のいずれかの Pod の詳細を表示すると、/opt以下にマウントされたnfs-vol が表示されるはずです。Pod 名を使用する場合は、タブキーによるコマンドライン補完を使用すると便利です。

```
student@cp:~$ kubectl describe pod try1-594fbb5fc7-5k7sj
```

```
<output_omitted>
    Mounts:
      /etc/cars from car-vol (rw)
      /opt from nfs-vol (rw)
      /var/run/secrets/kubernetes.io/serviceaccount from default-token-j7cqd (ro)
<output_omitted>
```

## 課題 5.3: ConfigMap を使ったコンテナの構成

以前の演習では、ロギングを処理するために 2 番目のコンテナを追加しました。ConfigMap の使用方法とストレージのアタッチについて学びましたので、basic な Pod を構成してみましょう。

1. 先ほどのシンプルな Pod の YAML を見直します。Ambassador スタイルのロギングコンテナを Pod に追加しましたが、ロギングを完全に構成していなかったことを思い出してください。

```
student@cp:~$ cat basic.yaml
```

```
<output_omitted>
  containers:
  - name: webcont
    image: nginx
    ports:
    - containerPort: 80
  - name: fdlogger
    image: fluent/fluentd
```

2. まず、各コンテナに共有ストレージを追加するところから始めましょう。ここではPV とPVC を提供するためにhostPath ストレージクラスを使用します。まず、ディレクトリを作成します。

```
student@cp:~$ sudo mkdir /tmp/weblog
```

3. ここで、hostPath ストレージクラスにそのディレクトリを使用するための新しいPV を作成します。manual というstorageClassName を使用し、その名前を使用するPVC のみがリソースをバインドできるようにします。

```
student@cp:~$ vim weblog-pv.yaml
```

**weblog-pv.yaml**

```yaml
kind: PersistentVolume
apiVersion: v1
metadata:
  name: weblog-pv-volume
  labels:
    type: local
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```yaml
spec:
  storageClassName: manual
  capacity:
    storage: 100Mi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/tmp/weblog"
```

4. 新しいPV を作成し、それが存在しており、Available というステータスが表示されていることを確認します。

```
student@cp:~$ kubectl create -f weblog-pv.yaml
```

```
persistentvolume/weblog-pv-volume created
```

```
student@cp:~$ kubectl get pv weblog-pv-volume
```

```
NAME             CAPACITY ACCESS MODES  RECLAIM POLICY
  STATUS        CLAIM   STORAGECLASS   REASON   AGE

weblog-pv-volume 100Mi    RWO           Retain
  Available             manual                  21s
```

5. 次に、先ほど作成したPV を使用するためのPVC を作成します。

```
student@cp:~$ vim weblog-pvc.yaml
```

**weblog-pvc.yaml**

```yaml
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: weblog-pv-claim
spec:
  storageClassName: manual
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 100Mi
```

6. PVC を作成し、以前に作成したPV に Bound として表示されることを確認します。

```
student@cp:~$ kubectl create -f weblog-pvc.yaml
```

```
persistentvolumeclaim/weblog-pv-claim created
```

```
student@cp:~$ kubectl get pvc weblog-pv-claim
```

```
NAME             STATUS  VOLUME           CAPACITY ACCESS MODES
   STORAGECLASS   AGE
weblog-pv-claim  Bound   weblog-pv-volume 100Mi    RWO
   manual         79s
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


7. Pod にストレージを追加する準備が整いました。3 つのセクションを編集します。最初のセクションは、Pod にストレージを追加することを宣言し、次に、各コンテナにボリュームを利用できるようにする場所を指示するセクションを 2 つ追加します。

```
student@cp:~$ vim basic.yaml
```

basic.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: basicpod
  labels:
    type: webserver
spec:
  volumes:                          #<-- Add three lines, same depth as containers
    - name: weblog-pv-storage
      persistentVolumeClaim:
        claimName: weblog-pv-claim
  containers:
  - name: webcont
    image: nginx
    ports:
    - containerPort: 80
    volumeMounts:                   #<-- Add three lines, same depth as ports
      - mountPath: "/var/log/nginx/"
        name: weblog-pv-storage     # Must match volume name above
  - name: fdlogger
    image: fluent/fluentd
    volumeMounts:                   #<-- Add three lines, same depth as image:
      - mountPath: "/var/log"
        name: weblog-pv-storage     # Must match volume name above
```

8. この時点で、再び Pod を作成することができます。シェルを作成すると、**nginx** の`access.log`はもはや`stdout`を指すシンボリックリンクではなく、書き込み可能な長さがゼロのファイルになっていることがわかります。ログファイルの **tailf** を実行したままにしておきます。

```
student@cp:~$ kubectl create -f basic.yaml
```

```
pod/basicpod created
```

```
student@cp:~$ kubectl exec -c webcont -it basicpod -- /bin/bash
```

On Container

```
root@basicpod:/#  ls -l /var/log/nginx/access.log
```

```
-rw-r--r-- 1 root root 0 Oct 18 16:12 /var/log/nginx/access.log
```

```
root@basicpod:/# tail -f /var/log/nginx/access.log
```

9. cp ノードへの 2 つ目の接続を開きます。まだ Pod を公開する `Service` を設定していないため、Pod の IP を使用します。

```
student@cp:~$ kubectl get pods -o wide
```

```
NAME      READY STATUS   RESTARTS  AGE    IP               NODE
   NOMINATED NODE
basicpod 2/2   Running  0         3m26s  192.168.213.181  cp
   <none>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



10. **curl** を使用して、Web サーバーのウェルカムページを表示します。コマンドが完了すると、ログに新しいエントリーが追加されているのがわかるはずです。GET の直後に、成功を示す200 応答が表示されます。**ctrl-c** と **exit** を使用して、ホストシェルのプロンプトに戻ることができます。

```
student@cp:~$ curl http://192.168.213.181
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

**On Container**

192.168.32.128 - - [18/Oct/2018:16:16:21 +0000] "GET / HTTP/1.1" 200 612 "-" "curl/7.47.0" "-"

11. webcont コンテナが PV に書き込んでいることがわかったので、そのディレクトリをソースとして使用するようにロガーを設定します。より柔軟にするために、ConfigMap を使って **fluentd** を設定します。

    Fluentd にはデータの入出力のためのオプションがたくさんあります。ここでは、webcont コンテナのファイルから読み込み、fdlogger コンテナの標準出力に書き込むことにします。データの設定の詳細は、こちらの **fluentd** のドキュメントに記載されています。https://docs.fluentd.org/v1.0/categories/config-file

```
student@cp:~$ vim weblog-configmap.yaml
```

**weblog-configmap.yaml**

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: fluentd-config
  namespace: default
data:
  fluentd.conf: |
    <source>
      @type tail
      format none
      path /var/log/access.log
      tag count.format1
    </source>

    <match *.**>
    @type stdout
    id stdout_output
    </match>
```

12. 新しい ConfigMap を作成します。

```
student@cp:~$ kubectl create -f weblog-configmap.yaml
```

```
configmap/fluentd-config created
```

13. basicpod の両方のコンテナのログを表示します。起動情報が表示されるはずですが、HTTP トラフィックは表示されません。

```
student@cp:~$ kubectl logs basicpod webcont
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
10-listen-on-ipv6-by-default.sh: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
```

```
student@cp:~$  kubectl logs basicpod fdlogger
```

```
2020-09-02 19:32:59 +0000 [info]: reading config file path="/etc/fluentd-config/fluentd.conf"
2020-09-02 19:32:59 +0000 [info]: starting fluentd-0.12.29
2020-09-02 19:32:59 +0000 [info]: gem 'fluent-mixin-config-placeholders' version '0.4.0'
2020-09-02 19:32:59 +0000 [info]: gem 'fluent-mixin-plaintextformatter' version '0.2.6'

<output_omitted>

  <source>
    @type tail
    format none
    path /var/log/access.log
<output_omitted>
```

14. ここで、**fluentd** コンテナが configmap をボリュームとしてマウントし、config ファイル内の変数を参照するように、Pod の yaml ファイルを編集します。Pod へのボリュームの宣言、`env` パラメータ、fluentd コンテナへのボリュームのマウントという 3 つの領域を追加することになります。

```
student@cp:~$ vim basic.yaml
```

basic.yaml

```yaml
....
  volumes:
    - name: weblog-pv-storage
      persistentVolumeClaim:
        claimName: weblog-pv-claim
    - name: log-config                    #<-- This and two lines following
      configMap:
        name: fluentd-config              # Must match existing configMap
....
    image: fluent/fluentd
    env:                                  #<-- This and two lines following
    - name: FLUENTD_OPT
      value: -c /etc/fluentd-config/fluentd.conf
....
    volumeMounts:
      - mountPath: "/var/log"
        name: weblog-pv-storage
      - name: log-config                  #<-- This and next line
        mountPath: "/etc/fluentd-config"
```

15. この時点で、Pod を削除して再作成することで、ConfigMap が新しい Pod で使用されるようになるなどの変更を行うことができます。

```
student@cp:~$ kubectl delete pod basicpod
```

```
pod "basicpod" deleted
```

```
student@cp:~$ kubectl create -f basic.yaml
```

```
pod/basicpod created
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
student@cp:~$ kubectl get pod basicpod -o wide
```

```
NAME       READY   STATUS    RESTARTS   AGE   IP               NODE     NOMINATED....
basicpod   2/2     Running   0          8s    192.168.171.122  worker   <none>   ....
```

16. **curl** を数回使用して、basicpod が提供するデフォルトのページを見ます。

```
student@cp:~$ curl http://192.168.171.122
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
<output_omitted>
```

17. 両方のコンテナのログを見ます。標準的な起動情報に加えて、先ほど使用した curl コマンドからの HTTP リクエストもfdlogger の出力の最後に表示されているはずです。

```
student@cp:~$ kubectl logs basicpod webcont
```

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
```

```
student@cp:~$  kubectl logs basicpod fdlogger
```

```
2020-09-02 19:32:59 +0000 [info]: reading config file path="/etc/fluentd-config/fluentd.conf"
2020-09-02 19:32:59 +0000 [info]: starting fluentd-0.12.29
2020-09-02 19:32:59 +0000 [info]: gem 'fluent-mixin-config-placeholders' version '0.4.0'
2020-09-02 19:32:59 +0000 [info]: gem 'fluent-mixin-plaintextformatter' version '0.2.6'

<output_omitted>

  <source>
    @type tail
    format none
    path /var/log/access.log

<output_omitted>

2020-09-02 19:47:38 +0000 count.format1: {"message":"192.168.219.64 - - [02/Sep/2020:19:47:38 +0000]
↪  \"GET / HTTP/1.1\" 200 612 \"-\" \"curl/7.58.0\" \"-\""}
2020-09-02 19:47:41 +0000 count.format1: {"message":"192.168.219.64 - - [02/Sep/2020:19:47:41 +0000]
↪  \"GET / HTTP/1.1\" 200 612 \"-\" \"curl/7.58.0\" \"-\""}
2020-09-02 19:47:47 +0000 count.format1: {"message":"192.168.219.64 - - [02/Sep/2020:19:47:47 +0000]
↪  \"GET / HTTP/1.1\" 200 612 \"-\" \"curl/7.58.0\" \"-\""}
```

## 課題 5.4: ローリングアップデートとロールバック

**Podman** を使ってsimpleapp のイメージを作成したので、コンテナの更新を行います。**docker** を使用している場合は、コマンドの構文が同じであることを忘れないでください。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


1. ソースを少し変更して、新しいイメージを作成します。このアプリケーションでは、アップデートとロールバックを使用します。最後の行にコメントを追加して、新しいイメージとして生成します。

```
student@cp:~$ cd $HOME/app1

student@cp:~/app1$ vim simple.py

<output_omitted>
## Sleep for five seconds then continue the loop

  time.sleep(5)

## Adding a new comment so image is different.
```

2. 現在のイメージ、タグ、どれくらい前に作成されたかを表示します。

```
student@cp:~/app1$ sudo podman images |grep simple
```

```
10.97.177.111:5000/simpleapp        latest    fd6e141c3a2e  14 hours ago   925 MB
localhost/simpleapp                 latest    fd6e141c3a2e  14 hours ago   925 MB
```

3. 再度イメージを構築します。新しいコンテナとイメージが作成されます。成功したらベリファイを行います。ローカルイメージには、異なるイメージ ID と最新の作成時刻があるはずです。

```
student@cp:~/app1$ sudo podman build -t simpleapp .
```

```
STEP 1: FROM python:2
STEP 2: ADD simple.py /
--> c1620c253cb
STEP 3: CMD [ "python", "./simple.py" ]
STEP 4: COMMIT simpleapp
--> fd46f4115eb
fd46f4115ebc87a064c15c57b08779b02deec97ab1abe92713689342ece496bd
```

```
student@cp:~/app1$ sudo podman images | grep simple
```

```
localhost/simpleapp                 latest    fd46f4115ebc  43 seconds ago  925 MB
10.97.177.111:5000/simpleapp        latest    fd6e141c3a2e  14 hours ago    925 MB
```

4. 更新されたイメージにタグを付け、ローカルにホストされているレジストリにプッシュします。今回は画像名の末尾にv2 というタグを使用します。

```
student@cp:~/app1$ sudo podman tag simpleapp $repo/simpleapp:v2

student@cp:~/app1$ sudo podman push $repo/simpleapp:v2
```

```
Getting image source signatures
Copying blob 46829331b1e4 skipped: already exists
Copying blob 47458fb45d99 skipped: already exists
Copying blob 461719022993 skipped: already exists
Copying blob a3c1026c6bcc skipped: already exists
Copying blob d35c5bda4793 skipped: already exists
Copying blob f1d420c2af1a skipped: already exists
Copying blob bbcb11d3fa81 done
Copying blob ceee8816bb96 skipped: already exists
Copying blob da7b0a80a4f2 skipped: already exists
Copying blob e571d2d3c73c skipped: already exists
Copying config fd46f4115e done
Writing manifest to image destination
Storing signatures
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



5. もう一度画像を確認してください。同じイメージ ID で、v2 のタグが付いたエントリーが追加されているはずです。

```
student@cp:~/app1$ sudo podman images |grep simple
```

```
10.97.177.111:5000/simpleapp        v2        fd46f4115ebc  3 minutes ago  925 MB
localhost/simpleapp                 latest    fd46f4115ebc  3 minutes ago  925 MB
10.97.177.111:5000/simpleapp        latest    fd6e141c3a2e  14 hours ago   925 MB
```

6. ワーカーノード上で動作するターミナルに接続します。バージョンを尋ねずにイメージをプルします。これは、latest のイメージをプルします。次に v2 をプルします。デフォルトでは新しいバージョンのイメージをプルしなかったことに注意してください。文字の、latest のタグは最新のイメージと何の関係もありません。

```
student@worker:~$ sudo podman pull $repo/simpleapp
```

```
Trying to pull 10.97.177.111:5000/simpleapp:latest...
Getting image source signatures
Copying blob ca488b1eb9fa skipped: already exists
Copying blob 1f22e54987ac skipped: already exists
Copying blob 33be93024b52 skipped: already exists
Copying blob 41b428188c3d skipped: already exists
Copying blob 6dae3f0239bb skipped: already exists
Copying blob 637977187da0 skipped: already exists
Copying blob 3263dd1bdf84 [--------------------------------------] 0.0b / 0.0b
Copying blob 7506b856c5df [--------------------------------------] 0.0b / 0.0b
Copying blob 19aa3a14ed80 [--------------------------------------] 0.0b / 0.0b
Copying blob c45f0d38c649 [--------------------------------------] 0.0b / 0.0b
Copying config fd6e141c3a done
Writing manifest to image destination
Storing signatures
fd6e141c3a2e2d663f56c0fa7a4d056f73f6d1f63da6e36b19d7ef006dd8fff6
```

```
student@worker:~$ sudo podman pull $repo/simpleapp:v2
```

```
Trying to pull 10.97.177.111:5000/simpleapp:v2...
Getting image source signatures
Copying blob ca488b1eb9fa skipped: already exists
Copying blob 1f22e54987ac skipped: already exists
Copying blob 6dae3f0239bb skipped: already exists
Copying blob 41b428188c3d skipped: already exists
Copying blob 637977187da0 skipped: already exists
Copying blob 33be93024b52 skipped: already exists
Copying blob 3263dd1bdf84 skipped: already exists
Copying blob 7506b856c5df skipped: already exists
Copying blob c45f0d38c649 skipped: already exists
Copying blob 4da1215fbbb8 done
Copying config fd46f4115e done
Writing manifest to image destination
Storing signatures
fd46f4115ebc87a064c15c57b08779b02deec97ab1abe92713689342ece496bd
```

7. cp ノードに戻り、**kubectl edit** を使用して try1 Deployment のイメージを更新し、v2 を使用するようにします。パラメータは 1 つしか変更しないので、**kubectl set** コマンドを使用することもできます。設定ファイルは更新されていないため、削除や置換コマンドを実行しても新しいバージョンは含まれないことに注意してください。Pod の削除と再作成を順番に行うため、最大で 1 分ほどかかることがあります。

```
student@cp:~/app1$ kubectl edit deployment try1
```

```
....
      containers:
      - image: 10.105.119.236:5000/simpleapp:v2   #<-- Edit tag
        imagePullPolicy: Always
....
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


8. 各 Pod が再作成され、新しいバージョンのイメージが使用されていることを確認します。古い **ReplicaSet** のスケールダウンを示すメッセージもあれば、新しいイメージを使用したスケールアップを示すメッセージもあることに注意してください。

```
student@cp:~/app1$ kubectl get events
```

```
42m         Normal    ScalingReplicaSet   Deployment   Scaled up replica set try1-7fdbb5d557 to 6
32s         Normal    ScalingReplicaSet   Deployment   Scaled up replica set try1-7fd7459fc6 to 2
32s         Normal    ScalingReplicaSet   Deployment   Scaled down replica set try1-7fdbb5d557 to 5
32s         Normal    ScalingReplicaSet   Deployment   Scaled up replica set try1-7fd7459fc6 to 3
23s         Normal    ScalingReplicaSet   Deployment   Scaled down replica set try1-7fdbb5d557 to 4
23s         Normal    ScalingReplicaSet   Deployment   Scaled up replica set try1-7fd7459fc6 to 4
22s         Normal    ScalingReplicaSet   Deployment   Scaled down replica set try1-7fdbb5d557 to 3
22s         Normal    ScalingReplicaSet   Deployment   Scaled up replica set try1-7fd7459fc6 to 5
18s         Normal    ScalingReplicaSet   Deployment   Scaled down replica set try1-7fdbb5d557 to 2
18s         Normal    ScalingReplicaSet   Deployment   Scaled up replica set try1-7fd7459fc6 to 6
8s          Normal    ScalingReplicaSet   Deployment   (combined from similar events):
Scaled down replica set try1-7fdbb5d557 to 0
```

9. Deployment 内の Pod のイメージを表示します。イメージの表示だけに出力を絞ります。goproxy は変更されていないままですが、simpleapp はv2 になっているはずです。

```
student@cp:~/app1$ kubectl describe pod try1-895fccfb-ttqdn |grep Image
```

```
    Image:          10.105.119.236:5000/simpleapp:v2
    Image ID:       10.97.177.111:5000/simpleapp@sha256:c3848fd3b5cdf5f241ceb4aa7e96c5c2be9f09
f23eea1afe946f8507f3fcbc29
    Image:          registry.k8s.io/goproxy:0.1
    Image ID:       docker.io/library/goproxy@sha256:5334c7ad43048e3538775c
b09aaf184f5e8c
c8f1d93c209865c8f1d93c2098
```

10. Deployment の更新履歴を表示します。

```
student@cp:~/app1$ kubectl rollout history deployment try1
```

```
deployment.apps/try1
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

11. **rollout history** の出力で 2 つのリビジョンを比較します。イメージや Label は異なっているはずで、イメージの v2 が私たちが行った変更です。

```
student@cp:~/app1$ kubectl rollout history deployment try1 --revision=1 > one.out

student@cp:~/app1$ kubectl rollout history deployment try1 --revision=2 > two.out

student@cp:~/app1:$ diff one.out two.out
```

```
1c1
< deployment.apps/try1 with revision #1
---
> deployment.apps/try1 with revision #2
4c4
<   ^^Ipod-template-hash=1509661973
---
>   ^^Ipod-template-hash=45197796
7c7
<     Image:^^I10.105.119.236:5000/simpleapp
---
>     Image:^^I10.105.119.236:5000/simpleapp:v2
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



12. ロールアウトで元に戻す際に、**–dry-run** オプションを使用して元に戻される内容を表示します。これにより、新しいテンプレートを使用する前に確認することができます。

```
student@cp:~/app1$ kubectl rollout undo --dry-run=client deployment/try1
```

```
deployment.apps/try1 Pod Template:
  Labels:^^Iapp=try1
^^Ipod-template-hash=754bf9c75b
  Containers:
   simpleapp:
    Image:^^I10.97.177.111:5000/simpleapp
    Port:^^I<none>
    Host Port:^^I<none>
<output_omitted>
```

13. Pod を表示します。入力の速さにもよりますが、`try1` Pod は約 2 分前のものであるはずです。

```
student@cp:~/app1$ kubectl get pods
```

```
NAME                        READY     STATUS    RESTARTS   AGE
nginx-6b58d9cdfd-9fnl4      1/1       Running   1          5d
registry-795c6c8b8f-hl5wf   1/1       Running   2          5d
try1-594fbb5fc7-7dl7c       2/2       Running   0          2m
try1-594fbb5fc7-8mxlb       2/2       Running   0          2m
try1-594fbb5fc7-jr7h7       2/2       Running   0          2m
try1-594fbb5fc7-s24wt       2/2       Running   0          2m
try1-594fbb5fc7-xfffg       2/2       Running   0          2m
try1-594fbb5fc7-zfmz8       2/2       Running   0          2m
```

14. 今回はリビジョンは 2 つだけで、それもデフォルトで保持されています。もっとあれば、特定のバージョンを選択することができます。次のコマンドは、前のコマンドで**–dry-run** オプションがない場合と同じ効果を発揮します。

```
student@cp:~/app1$ kubectl rollout undo deployment try1 --to-revision=1
```

```
deployment.apps/try1 rolled back
```

15. Pod が終了して再作成されるまで、少し時間がかかることがあります。すべての Pod が再稼働するまで、チェックを続けてください。

```
student@cp:~/app1$ kubectl get pods
```

```
NAME                        READY     STATUS        RESTARTS   AGE
nginx-6b58d9cdfd-9fnl4      1/1       Running       1          5d
registry-795c6c8b8f-hl5wf   1/1       Running       2          5d
try1-594fbb5fc7-7dl7c       2/2       Terminating   0          3m
try1-594fbb5fc7-8mxlb       0/2       Terminating   0          2m
try1-594fbb5fc7-jr7h7       2/2       Terminating   0          3m
try1-594fbb5fc7-s24wt       2/2       Terminating   0          2m
try1-594fbb5fc7-xfffg       2/2       Terminating   0          3m
try1-594fbb5fc7-zfmz8       1/2       Terminating   0          2m
try1-895fccfb-8dn4b         2/2       Running       0          22s
try1-895fccfb-kz72j         2/2       Running       0          10s
try1-895fccfb-rxxtw         2/2       Running       0          24s
try1-895fccfb-srwq4         1/2       Running       0          11s
try1-895fccfb-vkvmb         2/2       Running       0          31s
try1-895fccfb-z46qr         2/2       Running       0          31s
```

## 課題 5.5: 出題範囲の確認


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

カリキュラム概要にある CKAD の出題範囲リストを再確認し、この章で取り上げたトピックの参考にしてください。

- PersistentVolume とエフェメラルボリュームの使い分け
- Deployment の理解とローリングアップデートを行う方法
- ConfigMap の理解

まず、必要なファイルやプロセスをすべて理解することに集中しましょう。必要な YAML サンプルのブックマークがあり、各ステップを素早く完了し、各オブジェクトが適切に実行されていることを確認できるまで、復習を繰り返します。

許可された URL とサブドメインだけを使って、復習で求められているリソースを作成し設定するための YAML の例を検索してください。ページがどのように見えるかを知っておくと、作業中の YAML にすぐに戻ることができます。

1. `entree` のキーと`meatloaf` の値を使用して、`specialofday` という新しい Secret を作成します。
2. `nginx` イメージを実行する `foodie` という新しい Deployment を作成します。
3. `specialofday` の Secret を、`/food/`ディレクトリの下にボリュームとしてマウントされるように Pod に追加します。
4. `foodie` Pod 内で bash シェルを実行し、Secret が正しくマウントされたことを確認します。
5. `nginx:1.12.1-alpine` イメージを使用するように Deployment を更新し、新しいイメージが使用されていることを確認します。
6. Deployment をロールバックして、最新の安定版 nginx が再び使用されていることを確認します。
7. この演習で以前に設定した NFS サーバーを使用して、`reviewvol` という新しい 200M NFS ボリュームを作成します。
8. `reviewvol` ボリュームを使用する`reviewpvc` という新しい PVC を作成します。
9. PVC を使用するように Deployment を編集し、`/newvol` の下にボリュームをマウントします。
10. nginx コンテナで bash シェルを実行し、ボリュームがマウントされたことを確認します。
11. このレビュー中に作成されたリソースを削除します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 6 章

# セキュリティの理解

## 課題 6.1: Pod とコンテナのセキュリティコンテキストの設定

**Working with Security: Overview**

この演習では、simpleapp の YAML ファイルが長くなって読みづらくなってきたので、新しいアプリケーションのセキュリティ機能を実装します。Kubernetes のアーキテクチャでは、より小さく、分離され、一時的な複数のアプリケーションが連動することが好まれます。私たちは、それを模倣して演習を続けていきます。

この演習では、2 つの新しいアプリケーションを作成します。1 つは、ホストノードへのアクセスを制限しますが、エンコードされたデータにアクセスできるようにします。もう 1 つは、ネットワーク セキュリティ ポリシーを使用して、デフォルトで全てへのアクセスが許可された Kubernetes のポリシーから、大半へのアクセスが拒否されたネットワークに移行する予定です。まず、Pod とコンテナの セキュリティコンテキストを設定し、次に Secret の作成と消費を行い、最後にネットワークセキュリティポリシーを設定することにします。

1. まず、2 つ目のアプリケーションのために新しいディレクトリを作成します。そのディレクトリに移動します。

```
student@cp:~$ mkdir $HOME/app2

student@cp:~$ cd $HOME/app2/
```

2. 2 つ目のアプリケーション用の YAML ファイルを作成します。以下の例では、シェルへのアクセスを許可しますが、それ以上のことはできないシンプルなイメージ、busybox を使用しています。コンテナだけでなく、Pod にも `runAsUser` を追加することにします。

```
student@cp:~/app2$ vim second.yaml
```

**second.yaml**

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secondapp
spec:
  securityContext:
    runAsUser: 1000
  containers:
  - name: busy
    image: busybox
```

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
11      command:
12        - sleep
13        - "3600"
14      securityContext:
15        runAsUser: 2000
16        allowPrivilegeEscalation: false
```

3. secondapp の Pod を作成し、実行されていることを確認します。前回の Deployment とは異なり、このアプリケーションは Pod として実行されています。YAML の出力を見て、Deployment がどのように見えるかを比較してください。ステータスセクションにはおそらく最も大きな違いがあるでしょう。

```
student@cp:~/app2$ kubectl create -f second.yaml
```

```
pod/secondapp created
```

```
student@cp:~/app2$ kubectl get  pod secondapp
```

```
NAME        READY     STATUS    RESTARTS   AGE
secondapp   1/1       Running   0          21s
```

```
student@cp:~/app2$ kubectl get pod secondapp -o yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  annotations:
    cni.projectcalico.org/containerID: f706f1c343de8749cb687db40....
    cni.projectcalico.org/podIP: 192.168.158.97/32
    cni.projectcalico.org/podIPs: 192.168.158.97/32
  creationTimestamp: "2021-05-03T21:23:12Z"
  name: secondapp
<output_omitted>
```

4. Pod 内で Bourne シェルを実行します。シェルや他のプロセスのユーザー ID を確認しましょう。Pod ではなく、コンテナの設定が表示されるはずです。これにより、1 つの Pod 内の複数のコンテナで、必要に応じて UID をカスタマイズすることができます。Pod 内のコンテナは 1 つだけなので、**-c busy** オプションを使用する必要はありません。

```
student@cp:~/app2$ kubectl exec -it secondapp -- sh
```

**On Container**

```
/ $ ps aux
```

```
PID   USER     TIME  COMMAND
    1  2000        0:00   sleep 3600
    8  2000        0:00   sh
   12  2000       0:00   ps aux
```

5. ここで、カーネルのケーパビリティをチェックします。次のステップでは、これらの値を変更します。

**On Container**

```
/ $ grep Cap /proc/1/status
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
CapInh:^^I00000000000005fb
CapPrm:^^I0000000000000000
CapEff:^^I0000000000000000
CapBnd:^^I00000000000005fb
CapAmb:^^I0000000000000000

/ $ exit
```

6. ケーパビリティのシェルラッパーツールである **capsh** コマンドを使用して、出力をデコードします。いくつかのステップで出力を表示し、比較することになります。コンマで区切られた 9 つのケーパビリティのようなものがリストアップされていることに注意してください。この数は、今後のバージョンで変更される可能性があります。

```
student@cp:~/app2$ capsh --decode=00000000000005fb
```

```
0x00000000000005fb=cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,
cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service
```

7. YAML ファイルを編集して、コンテナの新しいケーパビリティを追加します。ケーパビリティは、完全な root アクセスを許可することなく、特定の昇格した特権を付与することができます。ここでは、インターフェイス、ルーティング、およびその他のネットワーク設定を許可するために、**NET_ADMIN** を設定します。また、システムクロックの設定を可能にする **SYS_TIME** も設定します。カーネル機能の詳細については、以下ををご覧ください。https://github.com/torvalds/linux/blob/master/include/uapi/linux/capability.h

   Pod が完全に終了し、その後の Pod を作成できるようになるまで、最大で 1 分かかることがあります。

```
student@cp:~/app2$ kubectl delete pod secondapp
```

```
pod "secondapp" deleted
```

```
student@cp:~/app2$ vim second.yaml
```

second.yaml

```yaml
<output_omitted>
      - sleep
      - "3600"
    securityContext:
      runAsUser: 2000
      allowPrivilegeEscalation: false
      capabilities:                      #<-- Add this and following line
        add: ["NET_ADMIN", "SYS_TIME"]
```

8. 再度 Pod を作成します。コンテナ内でシェルを実行し、`/proc/1/status`の下にある Cap の設定を確認してください。前のインスタンスと異なっているはずです。

```
student@cp:~/app2$ kubectl create -f second.yaml
```

```
pod/secondapp created
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~/app2$ kubectl exec -it secondapp -- sh
```

**On Container**

```
/ $ grep Cap /proc/1/status
```

```
CapInh:^^I00000000020015fb
CapPrm:^^I0000000000000000
CapEff:^^I0000000000000000
CapBnd:^^I00000000020015fb
CapAmb:^^I0000000000000000
```

```
/ $ exit
```

9. 出力をもう一度デコードします。インスタンスには 16 個のカンマ区切りのケーパビリティが表示されていることに注意してください。**cap_net_admin** が **cap_sys_time** と同様に表示されています。

```
student@cp:~/app2$ capsh --decode=00000000020015fb
```

```
0x00000000020015fb=cap_chown,cap_dac_override,cap_fowner,cap_fsetid,cap_kill,
cap_setgid,cap_setuid,cap_setpcap,cap_net_bind_service,cap_net_admin,cap_sys_time
```

## 課題 6.2: Secret の作成と消費

Secret は、以前の演習で取り上げた `ConfigMap` と同様の方法で消費されます。保管時のデータ暗号化（at-rest encryption）は簡単に設定できるようになりましたが、歴史的には Secret は単に base64 エンコードされたものでした。設定可能な暗号化にはいくつかの種類があります。

1. まず、暗号化されたパスワードを生成することから始めます。

```
student@cp:~/app2$ echo LFTr@1n | base64
```

```
TEZUckAxbgo=
```

2. API オブジェクトの種類を Secret に設定したオブジェクトの YAML ファイルを作成します。エンコードされたキーをパスワードパラメーターとして使用します。

```
student@cp:~/app2$ vim secret.yaml
```

**secret.yaml**

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: lfsecret
data:
  password: TEZUckAxbgo=
```

3. 新しいオブジェクトをクラスターに取り込みます。

```
student@cp:~/app2$ kubectl create -f secret.yaml
```

```
secret/lfsecret created
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



4. secondapp の YAML ファイルを編集して、Secret を /mysqlpasswordの下にマウントされたボリュームとして使用します。volumeMounts: はコンテナのname: と同じところに並んでいます。volumes: は containers: と並んでいます。Pod は 3600 秒ごと、つまり 1 時間ごとに sleep コマンドが終了したときに再起動するということに注目してください。

```
student@cp:~/app2$ vim second.yaml
```

second.yaml

```
....
      runAsUser: 2000
      allowPrivilegeEscalation: false
      capabilities:
        add: ["NET_ADMIN", "SYS_TIME"]
    volumeMounts:                          #<-- Add this and six following lines
    - name: mysql
      mountPath: /mysqlpassword
  volumes:
  - name: mysql
    secret:
      secretName: lfsecret
```

```
student@cp:~/app2$ kubectl delete pod secondapp
```

```
pod "secondapp" deleted
```

```
student@cp:~/app2$ kubectl create -f second.yaml
```

```
pod/secondapp created
```

5. Pod が動作していることを確認し、パスワードが期待される場所にマウントされているかどうかを確認します。パスワードがクリアテキストでデコードされた状態で利用できることを確認します。

```
student@cp:~/app2$ kubectl get pod secondapp
```

```
NAME        READY     STATUS    RESTARTS   AGE
secondapp   1/1       Running   0          34s
```

```
student@cp:~/app2$ kubectl exec -ti secondapp -- /bin/sh
```

On Container

```
/ $ cat /mysqlpassword/password
```

```
LFTr@1n
```

6. ディレクトリの場所を表示します。..dataへのシンボリックリンクであり、別のディレクトリへのシンボリックリンクであることに注意してください。コンテナ内のファイルシステムを見た後、ノードに戻ります。

On Container

```
/ $ cd /mysqlpassword/

/mysqlpassword $ ls
```

```
password
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
/mysqlpassword $ ls -al
```

```
total 4
drwxrwxrwt    3 root     root          100 May 19 16:06 .
dr-xr-xr-x    1 root     root         4096 May 19 16:06 ..
drwxr-xr-x    2 root     root           60 May 19 16:06 ..2021_05_19_16_06_41.694089911
lrwxrwxrwx    1 root     root           31 May 19 16:06 ..data ->
↪  ..2021_05_19_16_06_41.694089911
lrwxrwxrwx    1 root     root           15 May 19 16:06 password -> ..data/password
```

```
/mysqlpassword $ exit
```

## 課題 6.3: ServiceAccount を操作する

ServiceAccount を使用して、クラスターの役割や、特定の HTTP 動詞を使用する能力を割り当てることができます。このセクションでは、新しい ServiceAccount を作成し、Secret を表示するためのアクセス権を付与します。

1. まず、デフォルトの Namespace とすべての Namespace の Secret を表示します。

```
student@cp:~/app2$ cd
```

```
student@cp:~$ kubectl get secrets
```

```
NAME                  TYPE                                  DATA   AGE
lfsecret              Opaque                                1      6m5s
```

```
student@cp:~$ kubectl get secrets --all-namespaces
```

```
NAMESPACE     NAME                     TYPE                             DATA   AGE
default       lfsecret                 Opaque                           1      69s
kube-system   bootstrap-token-j6r4vk   bootstrap.kubernetes.io/token    7      58m

<output_omitted>
```

2. 各エージェントが API サーバーとやりとりするために、Secret を使用していることがわかります。ここでは、アクセス権を持つ新しい ServiceAccount を作成することにします。

```
student@cp:~$ vim serviceaccount.yaml
```

**serviceaccount.yaml**

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
 name: secret-access-sa
```

```
student@cp:~$ kubectl create -f serviceaccount.yaml
```

```
serviceaccount/secret-access-sa created
```

```
student@cp:~$ kubectl get serviceaccounts
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
NAME                SECRETS   AGE
default             0         1d17h
secret-access-sa    0         34s
```

3. では、クラスター全体で許可される実際のアクションをリストアップする ClusterRole を作成します。構文を見るために、既存の Role を見てみましょう。

```
student@cp:~$ kubectl get clusterroles
```

```
NAME                                                   AGE
admin                                                  1d17h
calico-cni-plugin                                      1d17h
calico-kube-controllers                                1d17h
cluster-admin                                          1d17h
<output_omitted>
```

4. admin の詳細を表示し、cluster-admin と比較します。admin は特定のアクションが許可されていますがcluster-admin には、すべてのアクションを許可するメタ文字'*' があります。

```
student@cp:~$ kubectl get clusterroles admin -o yaml
```

```
<output_omitted>
```

```
student@cp:~$ kubectl get clusterroles cluster-admin -o yaml
```

```
<output_omitted>
```

5. 上記の出力の一部を用いて、独自のファイルを作成します。

```
student@cp:~$ vim clusterrole.yaml
```

clusterrole.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
 name: secret-access-cr
rules:
- apiGroups:
  - ""
  resources:
  - secrets
  verbs:
  - get
  - list
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


6. 新しいClusterRole を作成し、検証します。

```
student@cp:~$ kubectl create -f clusterrole.yaml
```

```
clusterrole.rbac.authorization.k8s.io/secret-access-cr created
```

```
student@cp:~$ kubectl get clusterrole secret-access-cr -o yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  creationTimestamp: 2018-10-18T19:27:24Z
  name: secret-access-cr
<output_omitted>
```

7. ここで、Role をアカウントにバインドします。roleRef: を使用する別の YAML ファイルを作成します。

```
student@cp:~$ vim rolebinding.yaml
```

rolebinding.yaml

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
 name: secret-rb
subjects:
- kind: ServiceAccount
  name: secret-access-sa
roleRef:
 kind: ClusterRole
 name: secret-access-cr
 apiGroup: rbac.authorization.k8s.io
```

8. 新しいRoleBinding を作成し、検証します。

```
student@cp:~$ kubectl create -f rolebinding.yaml
```

```
rolebinding.rbac.authorization.k8s.io/secret-rb created
```

```
student@cp:~$ kubectl get rolebindings
```

```
NAME        AGE
secret-rb   17s
```

9. secondapp Pod を表示し、現在のserviceAccount を **grep** します。default のアカウントを使用していることに注意してください。

```
student@cp:~$ kubectl get pod secondapp -o yaml |grep serviceAccount
```

```
serviceAccount: default
serviceAccountName: default
    - serviceAccountToken:
```

10. second.yamlファイルを編集し、serviceAccount の利用を追加します。

```
student@cp:~$ vim $HOME/app2/second.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



second.yaml

```
....
  name: secondapp
spec:
  serviceAccountName: secret-access-sa  #<-- Add this line
  securityContext:
    runAsUser: 1000
....
```

11. secondapp Pod がまだ稼働している場合は削除し、再度作成します。serviceAccount は、もはやdefault ではありません。

```
student@cp:~$ kubectl delete pod secondapp ; kubectl create -f $HOME/app2/second.yaml
```

```
pod "secondapp" deleted
pod/secondapp created
```

```
student@cp:~$ kubectl get pod secondapp -o yaml | grep serviceAccount
```

```
serviceAccount: secret-access-sa
serviceAccountName: secret-access-sa
    - serviceAccountToken:
```

## 課題 6.4: NetworkPolicy の実装

Kubernetes の初期アーキテクチャ設計として、すべての Pod が他のすべての Pod やノードに接続できるように設計された非分離型がありました。最近のリリースでは、NetworkPolicy を使用することで Pod を分離することができます。このポリシーは、**Project Calico** のようなネットワークプラグインがポリシーに従うことができる場合にのみ効果を発揮します。**flannel** のようなプラグインで使用した場合、ポリシーは何の効果もありません。matchLabels を使用すると、namespaceSelector を使用して選択できる Namespace 内をより細かく選択することができます。複数の Label を使用することで、複雑なルールを適用することができます。詳しくは、https://kubernetes.io/docs/concepts/services-networking/network-policies をご覧ください。

1. すべてのトラフィックを拒否するデフォルトポリシーを作成することから始めます。クラスターに取り込まれると、他のポリシーで選択されていないすべての Pod に影響し、ほとんど閉じた環境を作り出します。ingress または egress のトラフィックのみを拒否したい場合は、他方のpolicyType を削除することができます。

```
student@cp:~$ cd $HOME/app2/
```

```
student@cp:~/app2$ vim allclosed.yaml
```

allclosed.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-default
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

2. 新しい NetworkPolicy をテストする前に、このポリシーが適用されていない状態でネットワークアクセスが機能することを確認する必要があります。**secondapp** をアップデートして、**nginx** が動作する新しいコンテナを追加し、アクセスをテストしてくださ


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


い。まず、以下のように、**nginx** イメージと `webserver` という名前の 2 行を追加します。Pod が終了するまでに少し時間がかかるので、Pod を削除してからファイルを編集することにします。

```
student@cp:~/app2$ kubectl delete pod secondapp
```

```
pod "secondapp" deleted
```

```
student@cp:~/app2$ vim second.yaml
```

second.yaml

```yaml
spec:
    serviceAccountName: secret-access-sa
    securityContext:
      runAsUser: 1000
    containers:
    - name: webserver                    #<-- Add this and following line
      image: nginx
    - name: busy
      image: busybox
      command:
```

3. 新しい Pod を作成します。コンテナは 1 つしか起動せず、Pod は `ContainerCreating` から Error、`CrashLoopBackOff` に移行することに注意してください。次のステップでエラーのトラブルシューティングを行います。

```
student@cp:~/app2$ kubectl create -f second.yaml
```

```
pod/secondapp created
```

```
student@cp:~/app2$ kubectl get pods
```

```
NAME                       READY  STATUS            RESTARTS   AGE
nginx-6b58d9cdfd-9fnl4     1/1    Running           1          2d
Registry-795c6c8b8f-hl5wf  1/1    Running           2          2d
secondapp                  1/2    CrashLoopBackOff  1          13s
<output_omitted>
```

4. 障害に至るまでの出来事を詳しく見てみましょう。イメージはプルされ、コンテナは開始されました。しかし、コンテナの実行で失敗しました。

```
student@cp:~/app2$ kubectl get event
```

```
<output_omitted>
25s         Normal    Scheduled   Pod    Successfully assigned default/secondapp to cp
4s          Normal    Pulling     Pod    pulling image "nginx"
2s          Normal    Pulled      Pod    Successfully pulled image "nginx"
2s          Normal    Created     Pod    Created container
2s          Normal    Started     Pod    Started container
23s         Normal    Pulling     Pod    pulling image "busybox"
21s         Normal    Pulled      Pod    Successfully pulled image "busybox"
21s         Normal    Created     Pod    Created container
21s         Normal    Started     Pod    Started container
1s          Warning   BackOff     Pod    Back-off restarting failed container
```

5. 前の出力で言及した **webserver** コンテナのログを表示します。user ディレクティブとディレクトリを作成する権限がないことに関するエラーがあることに注意してください。

```
student@cp:~/app2$ kubectl logs secondapp webserver
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
2018/04/13 19:51:13 warn 1 the "user" directive makes sense
only if the cp process runs with super-user privileges,
 ignored in /etc/nginx/nginx.conf:2
nginx: warn the "user" directive makes sense only if the cp
process runs with super-user privileges,
 ignored in /etc/nginx/nginx.conf:2
2018/04/13 19:51:13 emerg 11: mkdir\(\) "/var/cache/nginx/client temp"
failed (13: Permission denied)
nginx: emerg mkdir() "/var/cache/nginx/client\_temp" failed
(13: Permission denied)
```

6. Pod を削除します。YAML ファイルを編集して、Pod 全体の UID の設定をコメントアウトします。

```
student@cp:~/app2$ kubectl delete -f second.yaml
```

```
pod "secondapp" deleted
```

```
student@cp:~/app2$ vim second.yaml
```

```yaml
spec:
  serviceAccountName: secret-access-sa
#  securityContext:                        #<-- Comment this and following line
#    runAsUser: 1000
  containers:
  - name: webserver
```

7. 再度 Pod を作成します。今度は両方のコンテナが実行されるはずです。入力の速さによっては、前の Pod が完全に終了するのを待つ必要があるかもしれません。

```
student@cp:~/app2$ kubectl create -f second.yaml
```

```
pod/secondapp created
```

```
student@cp:~/app2$ kubectl get pods
```

```
NAME                        READY     STATUS      RESTARTS   AGE
<output_omitted>
secondapp                   2/2       Running     0          5s
```

8. `NodePort` Service を使って **webserver** を公開します。Label がないため、エラーとなるでしょう。

```
student@cp:~/app2$ kubectl expose pod secondapp --type=NodePort --port=80
```

```
error: couldn't retrieve selectors via --selector flag or
introspection: the pod has no labels and cannot be exposed
See 'kubectl expose -h' for help and examples.
```

9. YAML ファイルを編集して metadata に Label を追加し、Pod 名の直後に `example: second` の Label を追加します。なお、delete コマンドに YAML ファイルを渡すことで、複数のリソースを一度に削除することができます。Pod を削除して再作成します。Pod がシャットダウンするのに 1 分ほどかかる場合があります。

```
student@cp:~/app2$ kubectl delete -f second.yaml
```

```
pod "secondapp" deleted
```

```
student@cp:~/app2$ vim second.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secondapp
  labels:                     #<-- This and following line
    example: second
spec:
#  securityContext:
#    runAsUser: 1000
<output_omitted>
```

```
student@cp:~/app2$ kubectl create -f second.yaml
```

```
pod/secondapp created
```

```
student@cp:~/app2$ kubectl get pods
```

```
NAME                        READY     STATUS      RESTARTS   AGE
<output_omitted>
secondapp                   2/2       Running     0          15s
```

10. 再びNodePort を公開する Service を別途作成した後、Label を追加して、kubernetes の内部でリソースを結びつけるために Label が不可欠であることを説明します。

```
student@cp:~/app2$ kubectl create service nodeport secondapp --tcp=80
```

```
service/secondapp created
```

11. Service の詳細を見てみましょう。セレクタがapp: secondapp に設定されていることに注目してください。また、NodePort にも注目してください。以下の例では31655 ですが、実際は異なる場合があります。

```
student@cp:~/app2$ kubectl get svc secondapp -o yaml
```

```
apiVersion: v1
kind: Service
metadata:
  creationTimestamp: "2020-04-16T04:40:07Z"
  labels:
    example: second
  managedFields:

....

spec:
  clusterIP: 10.97.96.75
  externalTrafficPolicy: Cluster
  ports:
  - nodePort: 31665
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    app: secondapp
  sessionAffinity: None
  type: NodePort
status:
  loadBalancer: {}
```

12. **curl** と前の出力に示されたClusterIP を使用して、Service へのアクセスをテストします。Label が他のリソースと一致しない


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



ので、**curl** コマンドは失敗するはずです。ハングアップした場合は、**control-c** でシェルに戻るように終了します。

```
student@cp:~/app2$ curl http://10.97.96.75
```

13. Edit the service. We will change the label to match **secondapp**, and set the `nodePort` to a new port, one that may have been specifically opened by our firewall team, port 32000. Service を編集します。Label を **secondapp** と一致するように変更し、`nodePort` を新しいポートに設定します。このポートは、ファイアウォールを扱うチームによって特別に開放されている可能性があります。

```
student@cp:~/app2$ kubectl edit svc secondapp
```

```yaml
<output_omitted>
  ports:
  - name: "80"
    nodePort: 32000       #<-- Edit this line
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    example: second       #<-- Edit this line, note key and parameter change
  sessionAffinity: None
<output_omitted>
```

14. 更新されたポート番号が正しく表示されていることを確認し、ClusterIP に注意してください。以下の例では、予想通り ClusterIP が`10.97.96.75`、ポートが32000 と表示されています。

```
student@cp:~/app2$ kubectl get svc
```

```
NAME         TYPE      CLUSTER-IP     EXTERNAL-IP  PORT(S)        AGE
<output_omitted>
secondapp    NodePort  10.97.96.75    <none>       80:32000/TCP   5m
```

15. ハイポートへのアクセスをテストします。`ClusterIP:<low-port-number>` のノードからテストしても、外部の`hostIP:<high-port-numb` からテストしても、デフォルトの nginx welcome ページが表示されるはずです。ハイポートはランダムに生成されるので、それが利用可能であることを確認してください。両方のノードが 32000 番ポートで Web サーバーを公開しているはずです。この例では、**curl** コマンドを使用していますが、ウェブブラウザを使用することもできます。

```
student@cp:~/app2$ curl http://10.97.96.75
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

```
student@cp:~/app2$ curl ifconfig.io
```

```
35.184.219.5
```

```
[user@laptop ~]$ curl http://35.184.219.5:32000
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

16. ここで、コンテナから外界への egress をテストします。**netcat** コマンドを使って、ポート 80 で動作している Web サーバーへのアクセスを確認することにします。まず、nginx へのローカルアクセスをテストし、次にリモートサーバーをテストします。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~/app2$ kubectl exec -it -c busy secondapp -- sh
```

**On Container**

```
/ $ nc -vz 127.0.0.1 80
```

```
127.0.0.1 (127.0.0.1:80) open
```

```
/ $ nc -vz www.linux.com 80
```

```
www.linux.com (151.101.185.5:80) open
```

```
/ $ exit
```

## 課題 6.5: Testing the Policy

1. さて、ingress と egress の両方をテストしたので、NetworkPolicy を実装します。

```
student@cp:~/app2$ kubectl create -f $HOME/app2/allclosed.yaml
```

```
networkpolicy.networking.k8s.io/deny-default created
```

2. もう一度、ingress と egress のテストを使用してください。このあとの 4 つのうち 3 つは最終的にタイムアウトするはずです。まず、クラスターの外からテストを始めてみて、待ちくたびれたら中断してください。

```
[user@laptop ~]$ curl http://35.184.219.5:32000
```

```
curl: (7) Failed to connect to 35.184.219.5 port
32000: Connection timed out
```

3. そして、ホストからコンテナへテストします。

```
student@cp:~/app2$ curl http://10.97.96.75:80
```

```
curl: (7) Failed to connect to 10.97.96.75 port 80: Connection timed out
```

4. 次に、egress をテストします。コンテナからコンテナへは、フィルターが Pod の外側にあるため動作するはずです。次に、外部の Web ページへの egress をテストしてください。最終的にタイムアウトするはずです。

```
student@cp:~/app2$ kubectl exec -it -c busy secondapp -- sh
```

**On Container**

```
/ $ nc -vz 127.0.0.1 80
```

```
127.0.0.1 (127.0.0.1:80) open
```

```
/ $ nc -vz www.linux.com 80
```

```
nc: bad address 'www.linux.com'
```

```
/ $ exit
```

5. NetworkPolicy を更新し、Egress の行をコメントアウトします。その後、ポリシーを置き換えます。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
student@cp:~/app2$ vim $HOME/app2/allclosed.yaml
```

allclosed.yaml

```
....
spec:
  podSelector: {}
  policyTypes:
  - Ingress
#  - Egress            #<-- Comment out this line
```

```
student@cp:~/app2$ kubectl replace -f $HOME/app2/allclosed.yaml
```

```
networkpolicy.networking.k8s.io/deny-default replaced
```

6. 外部サイトへの egress のアクセスをテストします。ログインした状態でコンテナ内の **eth0** の IP アドレスを取得します。この IP は以下の例では192.168.55.91 となっていますが、異なる場合があります。

```
student@cp:~/app2$ kubectl exec -it -c busy secondapp -- sh
```

On Container

```
/ $ nc -vz www.linux.com 80
```

```
www.linux.com (151.101.185.5:80) open
```

```
/ $ ip a
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: tunl0@NONE: <NOARP> mtu 1480 qdisc noop qlen 1000
    link/ipip 0.0.0.0 brd 0.0.0.0
4: eth0@if59: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue
    link/ether 1e:c8:7d:6a:96:c3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.55.91/32 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::1cc8:7dff:fe6a:96c3/64 scope link
       valid_lft forever preferred_lft forever
```

```
/ $ exit
```

7. ここで、nginx コンテナのみへの ingress を許可するセレクタを追加します。**eth0** の範囲から IP を使用します。

```
student@cp:~/app2$ vim $HOME/app2/allclosed.yaml
```

allclosed.yaml

```
<output_omitted>
 policyTypes:
 - Ingress
 ingress:                    #<-- Add this and following three lines
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
  - from:
    - ipBlock:
        cidr: 192.168.0.0/16
# - Egress
```

8. ポリシーを再作成し、その構成を確認します。

```
student@cp:~/app2$ kubectl replace -f $HOME/app2/allclosed.yaml
```

```
networkpolicy.networking.k8s.io/deny-default replaced
```

```
student@cp:~/app2$ kubectl get networkpolicy
```

```
NAME           POD-SELECTOR     AGE
deny-default   <none>           3m2s
```

```
student@cp:~/app2$ kubectl get networkpolicy -o yaml
```

```
apiVersion: v1
items:
- apiVersion: networking.k8s.io/v1
  kind: NetworkPolicy
  metadata:
<output_omitted>
```

9. コンテナへのアクセスは、**curl** と **ping** の両方でテストし、使用する IP アドレスは、コンテナ内の IP から判明しました。**ping** を使うには **iputils-ping** などのインストールが必要な場合があります。

```
student@cp:~/app2$ curl http://192.168.55.91
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

```
student@cp:~/app2$ ping -c5 192.168.55.91
```

```
PING 192.168.55.91 (192.168.55.91) 56(84) bytes of data.
64 bytes from 192.168.55.91: icmp_seq=1 ttl=63 time=1.11 ms
64 bytes from 192.168.55.91: icmp_seq=2 ttl=63 time=0.352 ms
64 bytes from 192.168.55.91: icmp_seq=3 ttl=63 time=0.350 ms
64 bytes from 192.168.55.91: icmp_seq=4 ttl=63 time=0.359 ms
64 bytes from 192.168.55.91: icmp_seq=5 ttl=63 time=0.295 ms

--- 192.168.55.91 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4054ms
rtt min/avg/max/mdev = 0.295/0.495/1.119/0.312 ms
```

10. ポート80 の TCP トラフィックのイングレスのみを許可するようにポリシーを更新し、**curl** でテストしてください。うまくいくはずです。ports の項目は、数行上の from の項目と同じ並びになるはずです。

```
student@cp:~/app2$ vim $HOME/app2/allclosed.yaml
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



allclosed.yaml

```yaml
<output_omitted>
  - Ingress
  ingress:
  - from:
    - ipBlock:
        cidr: 192.168.0.0/16
    ports:                      #<-- Add this and two following lines
    - port: 80
      protocol: TCP
#   - Egress
```

```
student@cp:~/app2$ kubectl replace -f $HOME/app2/allclosed.yaml
```

```
networkpolicy.networking.k8s.io/deny-default replaced
```

```
student@cp:~/app2$ curl http://192.168.55.91
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

11. 5 回の Ping はすべて失敗し、受信はゼロになるはずです。

```
student@cp:~/app2$ ping -c5 192.168.55.91
```

```
PING 192.168.55.91 (192.168.55.91) 56(84) bytes of data.

--- 192.168.55.91 ping statistics ---
5 packets transmitted, 0 received, 100% packet loss, time 4098ms
```

12. レジストリや他の Pod にアクセスしたい場合に備えて、default-deny ポリシーを削除しておくとよいでしょう。

```
student@cp:~/app2$ kubectl delete networkpolicies deny-default
```

```
networkpolicy.networking.k8s.io "deny-default" deleted
^^I
```

## 課題 6.6: 出題範囲の確認

**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

カリキュラム概要にある CKAD の出題範囲リストを再確認し、この章で取り上げたトピックの参考にしてください。

- 認証、認可、アドミッションコントロールについて理解する。
- Secret を作成、使用する
- ServiceAccount を理解する
- セキュリティコンテキストを理解する


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


- NetworkPolicy の基本を理解する

まず、必要なファイルやプロセスをすべて理解することに集中しましょう。必要な YAML サンプルのブックマークがあり、各ステップを素早く完了し、各オブジェクトが適切に実行されていることを確認できるまで、復習を繰り返します。

1. `nginx` イメージを使用する新しい Deployment を作成します。
2. 新しく作成した Deployment を公開するために、新しい `LoadBalancer` Service を作成し、動作することをテストします。
3. netblock という新しい NetworkPolicy を作成し、この Deployment 内の Pod のみへのすべてのトラフィックをブロックします。すべての Deployment へのトラフィックがブロックされることをテストします。
4. ポート 80 のみで Pod へのトラフィックを許可するように、`netblock` のポリシーを更新します。デフォルトの nginx ウェブページにアクセスできることをテストします。
5. `security-review1.yaml`ファイルを見つけて使用し、Pod を作成します。

   ```
   student@cp:~$ kubectl create -f security-review1.yaml
   ```

6. Pod のステータスを表示します。
7. 以下のコマンドを使用して、Pod の問題の原因を突き止めます。

   ```
   student@cp:~$ kubectl get pod securityreview

   student@cp:~$ kubectl describe pod securityreview

   student@cp:~$ kubectl logs securityreview
   ```

8. エラーを発見したら、コンテナにログインして、nginx ユーザーの適切な ID を見つけます。
9. yaml を編集し、Pod がエラーなく実行されるように Pod を再作成します。
10. securityaccount という新しい`ServiceAccount` を作成します。
11. secrole という名前の ClusterRole を作成し、すべての apiGroups の Pod の作成、削除、リスト表示のみを許可するようにします。
12. ServiceAccount に clusterRole をバインドします。
13. securityaccount の token を探します。`/tmp/securitytoken` というファイルを作成します。`token:` の値が等しい、eyJh で始まり数行に及ぶ長い文字列だけを入れます。その文字列だけがファイル内に存在するように注意しましょう。
14. この復習の中で追加したリソースを削除します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 7 章

# アプリケーションの公開

## 課題 7.1: アプリケーションの公開: Service を公開する

**概要**

この演習では、アプリケーションを他の Pod やクラスター外部に公開するさまざまな方法を探ります。以前の演習で使用した NodePort に、他の Service オプションを追加します。

1. まず、デフォルトの Service タイプである ClusterIP を使用します。これはクラスター内部の IP であり、クラスター内部からしか到達できません。既存の Service を表示することから始めます。

```
student@cp:~$ kubectl get svc
```

```
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP        8d
nginx        ClusterIP   10.108.95.67     <none>        443/TCP        8d
registry     ClusterIP   10.105.119.236   <none>        5000/TCP       8d
secondapp    NodePort    10.111.26.8      <none>        80:32000/TCP   7h
```

2. secondapp の既存の Service を保存し、削除します。同じ Label、ポート、プロトコルが使用されていることを確認します。

```
student@cp:~$ cd $HOME/app2

student@cp:~/app2$ kubectl get svc secondapp -o yaml > oldservice.yaml

student@cp:~/app2$ cat oldservice.yaml

student@cp:~/app2$ kubectl delete svc secondapp
```

```
service "secondapp" deleted
```

3. 新しい YAML ファイルを使用して Service を再作成します。前の Pod と同じセレクタを使用します。作成後の新しい Service を調べ、TYPE とPORT(S) を確認します。

```
student@cp:~/app2$ vim newservice.yaml
```

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


newservice.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: secondapp
spec:
  ports:
  - port: 80
    protocol: TCP
  selector:
    example: second
  sessionAffinity: None
status:
  loadBalancer: {}
```

```
student@cp:~/app2$ kubectl create -f newservice.yaml
```

```
service/secondapp created
```

```
student@cp:~/app2$ kubectl get svc
```

```
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)      AGE
kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP      8d
nginx        ClusterIP   10.108.95.67     <none>        443/TCP      8d
registry     ClusterIP   10.105.119.236   <none>        5000/TCP     8d
secondapp    ClusterIP   10.98.148.52     <none>        80/TCP       14s
```

4. アクセスできるかをテストします。デフォルトのウェルカムページが再び表示されることを確認してください。

```
student@cp:~/app2$ curl http://10.98.148.52
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

5. ここで、もう一つ簡単な Web サーバを作成します。今回はデフォルトのページとして **nginx** と異なる **httpd** のイメージを使用します。動作が確認できたら、Service のセレクタを編集して、新しいサーバを指すようにします。その後で Service のセレクタを元に戻します。これは、デプロイメント戦略として使うことができます。いくつかのコマンドは省略されていますが、このステップを完了することができるはずです。完了が難しい場合は、以前の内容を参照してください。

```
student@cp:~/app2$ kubectl create deployment newserver --image=httpd
```

6. `newserver` Label を確認します。
7. **kubectl edit** を使用して、`newserver` の Label をセレクタとして使用するように Service を変更します。
8. Service が新しいコンテンツを表示し、デフォルトの **nginx** ページが表示されないことをテストします。

```
student@cp:~/app2$ curl http://10.98.148.52
```

```
<html><body><h1>It works!</h1></body></html>
```

9. セレクタを編集して nginx サーバに戻し、確認します。その後、`newserver` の Deployment を削除してください。
10. クラスターの外部にポートを公開するために、NodePort を作成します。これは前のステップでコマンドラインから実行しました。NodePort を作成すると、新しい ClusterIP が自動的に作成されます。YAML ファイルを再度編集し、`type: NodePort` を追加し


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



ます。また、前章で述べたように、ファイアウォールのオープンポートに一致するように高い数のポートを追加します。既存の IP は不変なので、削除して再作成する必要があります。NodePort は、新しい ClusterIP を作成します。

```
student@cp:~/app2$ vim newservice.yaml
```

newservice.yaml

```
....
    protocol: TCP
    nodePort: 32000          #<-- Add this and following line
  type: NodePort
  selector:
    example: second
```

```
student@cp:~/app2$ kubectl delete svc secondapp ; kubectl create  -f newservice.yaml
```

```
service "secondapp" deleted
service/secondapp created
```

11. Service の新しい ClusterIP とポートを見つけます。

```
student@cp:~/app2$ kubectl get svc
```

```
NAME         TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP        8d
nginx        ClusterIP   10.108.95.67   <none>        443/TCP        8d
registry     ClusterIP   10.105.119.236 <none>        5000/TCP       8d
secondapp    NodePort    10.109.134.221 <none>        80:32000/TCP   4s
```

12. secondapp Service 用の新しい ClusterIP を使用して、低いポート番号をテストします。

```
student@cp:~/app2$ curl 10.109.134.221
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

13. 外部ノードからホスト IP と高いコンテナポートへのアクセスをテストします。IP とポート番号は以下の例と異なることに注意してください。トラフィックは 192.168.0.0 ポート経由で到着するため、ネットワークポリシーが適用されていても動作するはずです。ローカルシステムに端末がない場合は、ブラウザを使用してください。

```
student@cp:~/app2$ curl ifconfig.io
```

```
35.184.219.5
```

```
user@laptop:~/Desktop$ curl http://35.184.219.5:32000
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

14. LoadBalancer の使用は、ロードバランサーが利用可能であれば、外部のプロバイダーに非同期リクエストを行います。そして NodePort を作成し、外部 IP を含む応答を待ちます。ローカルの NodePort はロードバランサが応答する前でも動作します。Service の YAML ファイルを編集して、type を LoadBalancer に変更します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
student@cp:~/app2$ vim newservice.yaml
```

newservice.yaml

```
....
  - port: 80
    protocol: TCP
    nodePort: 32000
  type: LoadBalancer    #<-- Edit this line
  selector:
    example: second
```

```
student@cp:~/app2$ kubectl delete svc secondapp ; kubectl create -f newservice.yaml
```

```
service "secondapp" deleted
service/secondapp created
```

15. クラウドプロバイダーがロードバランサーを提供するように設定されていないため、External-IP は保留（pending）状態のままです。VirtualBox で使用した場合、いくつかの問題が検出されています。

```
student@cp:~/app2$ kubectl get svc
```

```
NAME         TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
kubernetes   ClusterIP      10.96.0.1        <none>        443/TCP        8d
nginx        ClusterIP      10.108.95.67     <none>        443/TCP        8d
registry     ClusterIP      10.105.119.236   <none>        5000/TCP       8d
secondapp    LoadBalancer   10.109.26.21     <pending>     80:32000/TCP   4s
```

16. ローカルとリモートのノードから再度テストしてください。IP アドレスとポートは、環境によって下記の例とは異なるでしょう。

```
serewic@laptop:~/Desktop$ curl http://35.184.219.5:32000
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```

17. また、**CoreDNS** が提供する、Service 作成時に動的に追加される DNS 名を利用することも可能です。まず、secondapp の`busy` コンテナにログインしてください。

```
student@cp:~/app2$ kubectl exec -it secondapp -c busy -- sh
```

**On Container**

(a) **nslookup** コマンドを使用して、`secondapp` Service を見つけます。次に、コンテナイメージを提供するために設定した `registry` Service を見つけます。期待した出力が得られない場合、もう一度試してみてください。3 つのリクエストのうち 1 つくらいはうまくいくはずです。

```
/ $ nslookup secondapp
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

Name:^^Isecondapp.default.svc.cluster.local
Address: 10.96.214.133

*** Can't find secondapp.svc.cluster.local: No answer
*** Can't find secondapp.cluster.local: No answer
*** Can't find secondapp.c.endless-station-188822.internal: No answer
<output_omitted>
^^I^^I
```

```
/ $ nslookup registry
```

```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

Name:^^Iregistry.default.svc.cluster.local
Address: 10.110.95.21
<output_omitted>
^^I^^I
```

(b) コマンドで表示された DNS サーバーの IP に関連する FQDN を調べます。コマンドで表示される IP は下記の例と異なる場合があります。

```
/ $ nslookup 10.96.0.10
```

```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

10.0.96.10.in-addr.arpa^^Iname = kube-dns.kube-system.svc.cluster.local
```

(c) Service 名の解決を試みますが、レコードは戻ってこないはずです。その後、FQDN で試してみてください。同様にレコードは戻らずにエラーが出力されるはずです。default Namespace のみがチェックされていることに注意してください。FQDN はいつも答えが返ってくるわけではないので、何度か FQDN を確認する必要があるかもしれません。

```
/ $ nslookup kube-dns
```

```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

** server can't find kube-dns.default.svc.cluster.local: NXDOMAIN

*** Can't find kube-dns.svc.cluster.local: No answer
*** Can't find kube-dns.cluster.local: No answer
*** Can't find kube-dns.c.endless-station-188822.internal: No answer
^^I^^I
```

```
/ $ nslookup kube-dns.kube-system.svc.cluster.local
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

Name:^^Ikube-dns.kube-system.svc.cluster.local
Address: 10.96.0.10

*** Can't find kube-dns.kube-system.svc.cluster.local: No answer
^^I^^I
```

(d) コンテナから出ます。

```
/ $ exit
```

18. multitenant という名前の新しい Namespace とmainapp という名前の新しい Deployment を作成します。Deployment のポート 80 を名前shopping を使用して公開します。

```
student@cp:~/app2$ kubectl create ns multitenant
```

```
namespace/multitenant created
```

```
student@cp:~/app2$ kubectl -n multitenant create deployment mainapp --image=nginx
```

```
deployment.apps/mainapp created
```

```
student@cp:~/app2$ kubectl -n multitenant expose deployment mainapp --name=shopping \
                        --type=NodePort --port=80
```

```
service/shopping exposed
```

19. secondapp busy コンテナにログインし、mainapp へのアクセスをテストします。

```
student@cp:~/app2$ kubectl  exec -it secondapp -c busy -- sh
```

**On Container**

(a) **nslookup** を使用して、新しい Service のアドレスを調べます。まず、Service 名だけで調べることから始めます。次に、Service 名に Namespace を追加します。busybox と他のアプリケーションが CoreDNS とどのように相互作用するかによって、いくつかの不都合な点があります。あなたの作業は動作するかもしれないし、いかないかもしれません。それぞれ何度か試してみてください。

```
/ $ nslookup shopping
```

```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

** server can't find shopping.default.svc.cluster.local: NXDOMAIN

*** Can't find shopping.svc.cluster.local: No answer
<output_omitted>
^^I^^I^^I^^I
```

```
/ $ nslookup shopping.multitenant
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

** server can't find shopping.multitenant: NXDOMAIN

*** Can't find shopping.multitenant: No answer
```

```
/ $ nslookup shopping.multitenant.svc.cluster.local
```

```
Server:         10.96.0.10
Address:^^I10.96.0.10:53

Name:^^Ishopping.multitenant.svc.cluster.local
Address: 10.101.4.142

*** Can't find shopping.multitenant.svc.cluster.local: No answer
```

(b) 今度は、Service 名、そして Namespace を使った Service 名の順に使ってみて動作するかどうか試してみてください。Namespace を使った DNS は、デフォルトページへのアクセス権がなくても動作するはずです。RBAC を使用してアクセスを許可することができます。返された Service の ClusterIP を確認すると、新しく作成された Service と一致するはずです。

```
/ $ wget shopping
```

```
wget: bad address 'shopping'
```

```
/ $ wget shopping.multitenant
```

```
Connecting to shopping.multitenant (10.101.4.142:80)
wget: can't open 'index.html': Permission denied
```

(c) このエラーがパーミッションに関するものであることがわかるように、もう一度試してみます。ローカルに書き込むのではなく、標準出力であるダッシュ(-)に書き込みます。

```
~ $ wget -O - shopping.multitenant
```

```
Connecting to shopping.multitenant (10.103.211.64:80)
writing to stdout
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>

....

</html>
-                    100% |*********************************************|   612  0:00:00
↪  ETA
written to stdout
```

## 課題 7.2: Service Mesh and Ingress Controller


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


クラスターの外部に公開する Service が大量にある場合や、ホストノードの低い番号のポートを公開する場合は、Ingress コントローラーをデプロイします。Kubernetes.io では nginx と GCE のコントローラーがよく紹介されていますが、多くの中から選ぶことができます。Istio、Linkerd、Contour、Aspen などのサービスメッシュを使用すれば、さらに多くの機能とメトリクスを利用できます。

1. 独自のスクリプトを使って、linkerd をインストールします。かなり多くの出力があります。そのすべてを表示する代わりに、多くの出力は省略されています。スクリプトの出力に目を通し、すべてに緑のチェックマークがつくことを確認してください。いくつかのステップは完了するのに数分かかることがあります。インストールを簡単にするために、各コマンドをここにリストアップしています。また、これらの手順は`setupLinkerd.txt`ファイルに記載されています。

   Linkerd の最新バージョンは、いくつかの問題がある可能性があります。そのため、私たちは 2.12.2 や 2.11.4 などの安定したバージョンの Linkerd を使用することにしています。時間と興味が許す限り、他のバージョンも自由に試してみてください。必要に応じて、**linkerd uninstall** と **linkerd uninject** を調査してください。

   コマンドによっては、様々な Pod をスピンアップするのに時間がかかるものがあります。

```
student@cp:~/app2$ curl -sL run.linkerd.io/install > setup.sh

student@cp:~/app2$ vim setup.sh

#!/bin/sh

set -eu

LINKERD2_VERSION=${LINKERD2_VERSION:-stable-2.12.2}  #<-- Edit to earlier stable version
INSTALLROOT=${INSTALLROOT:-"${HOME}/.linkerd2"}

happyexit() {
....

student@cp:~/app2$ sh setup.sh

student@cp:~/app2$ export PATH=$PATH:/home/student/.linkerd2/bin

student@cp:~/app2$ linkerd check --pre

student@cp:~/app2$ linkerd install --crds | kubectl apply -f -

student@cp:~/app2$ linkerd install | kubectl apply -f -

student@cp:~/app2$ linkerd check

student@cp:~/app2$ linkerd viz install | kubectl apply -f -

student@cp:~/app2$ linkerd viz check

student@cp:~/app2$ linkerd viz dashboard &
```

2. デフォルトでは、GUI は localhost 上で利用可能で、最後の dashboard コマンドで設定します。ノードにクラウドプロバイダーを使用している場合、外部からのアクセスを許可するために Service と Deployment を編集する必要があります。61 行目にある`-enforced-host` の = 以降の文字をすべて削除するように編集してください。

```
student@cp:~/app2$ kubectl -n linkerd-viz edit deploy web
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```yaml
spec:
      containers:
      - args:
        - -linkerd-controller-api-addr=linkerd-controller-api.linkerd.svc.cluster.local:8085
        - -linkerd-metrics-api-addr=metrics-api.linkerd-viz.svc.cluster.local:8085
        - -cluster-domain=cluster.local
        - -grafana-addr=grafana.linkerd-viz.svc.cluster.local:3000
        - -controller-namespace=linkerd
        - -viz-namespace=linkerd-viz
        - -log-level=info
        - -enforced-host=                                #<-- Remove everything after equal sign
        image: cr.l5d.io/linkerd/web:stable-2.12.2
        imagePullPolicy: IfNotPresent
```

3. ここで、http の nodePort と type を NodePort になるように編集します。

```
student@cp:~/app2$ kubectl edit svc web -n linkerd-viz
```

```yaml
....
ports:
  - name: http
    nodePort: 31500                              #<-- Add line with an easy to remember port
    port: 8084
....
  sessionAffinity: None
  type: NodePort                                 #<-- Edit type to be NodePort
status:
  loadBalancer: {}
....
```

4. ローカルブラウザーでパブリック IP にアクセスするテストを行ってください。IP は、以下に示すものとは異なるでしょう。

```
student@cp:~/app2$ curl ifconfig.io
```

```
104.197.159.20
```

5. ローカルシステムからブラウザを開き、パブリック IP と31500 のような大きな数字の nodePort にアクセスします。これは、出力に表示される localhost のポートではありません。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


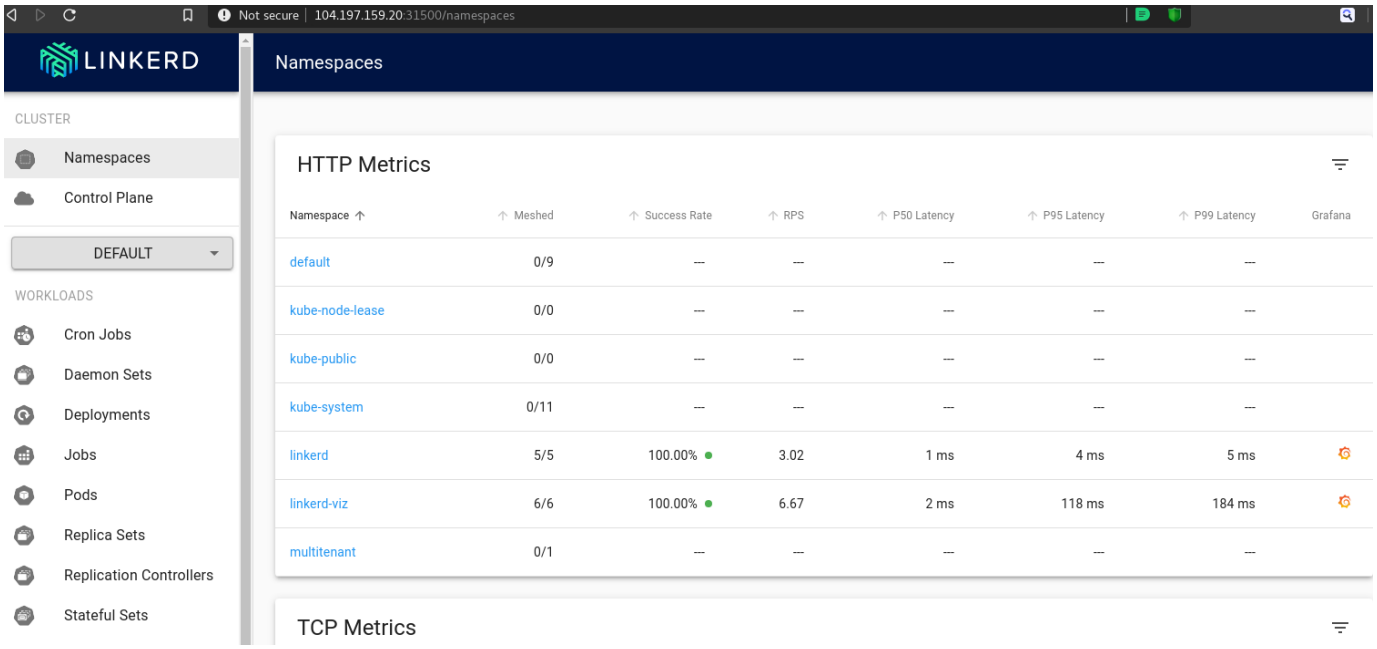


Figure 7.1: **Main Linkerd Page**

6. Linkerd が Deployment などのオブジェクトに注意を払うようにするためには、そのオブジェクトに Annotation を追加する必要があります。**linkerd inject** コマンドはこの Annotation 追加を行います。YAML を生成し、それを **linkerd** にパイプし、再度 **kubectl** にパイプします。オブジェクトの作成方法についてエラーが出るかもしれませんが、正常に処理されます。バックスラッシュを省略すれば、コマンドは 1 行で実行できます。

```
student@cp:~/app2$  kubectl -n multitenant get deploy mainapp -o yaml | \
     linkerd inject - | kubectl apply -f -
```

```
   <output_omitted>
```

7. GUI を確認すると、`multitenant` の Namespace と Pod がメッシュ化され、その名前がリンクになっていることが確認できるはずです。

8. Pod へのトラフィックを発生させ、GUI でトラフィックを観察します。

```
student@cp:~$ kubectl -n multitenant get svc
```

```
   NAME       TYPE       CLUSTER-IP     EXTERNAL-IP   PORT(S)        AGE
   shopping   NodePort   10.102.8.205   <none>        80:32518/TCP   4h54m
```

```
student@cp:~$ curl 10.102.8.205
```

```
   <!DOCTYPE html>
   <html>
   <head>
   <title>Welcome to nginx!</title>
   <output_omitted>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



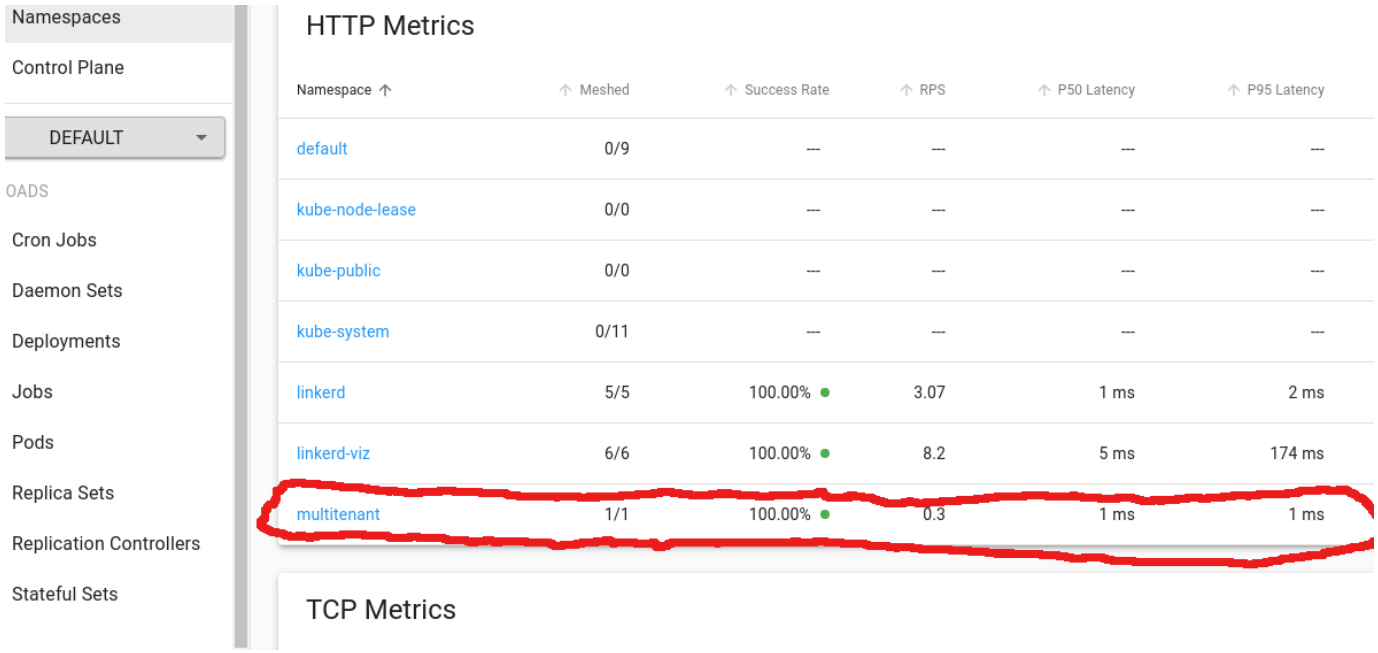


Figure 7.2: **Now shows meshed**

9. `mainapp` の Deployment をスケールアップします。トラフィックを発生させ、すべての Pod のメトリクスを取得します。

```
student@cp:~$ kubectl -n multitenant scale deploy mainapp --replicas=5
```

```
deployment.apps/mainapp scaled
```

```
student@cp:~$ curl 10.102.8.205   #Several times
```

10. GUI で提供されるその他の情報の一部を紹介します。

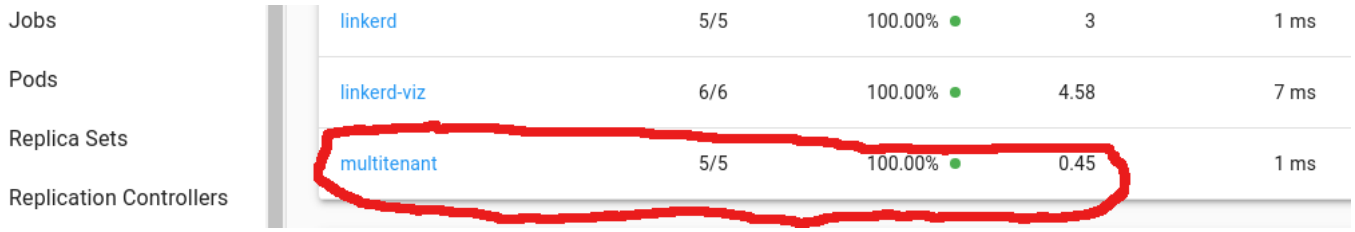


Figure 7.3: **Five meshed pods**

11. Linkerd には Ingress コントローラーが付属していませんので、トラフィックを管理するために Ingress コントローラーをインストールする必要があります。Ingress コントローラーをインストールするため、**Helm** チャートを活用します。ハブを検索すると、多くのチャートが利用が可能であることがわかります。

```
student@cp:~$ helm search hub ingress
```

```
URL                                             ^^ICHART VERSION    ^^I
APP VERSION       ^^IDESCRIPTION
https://artifacthub.io/packages/helm/k8s-as-hel...^^I1.0.2          ^^I
v1.0.0            ^^IHelm Chart representing a single Ingress Kubern...
https://artifacthub.io/packages/helm/openstack-...^^I0.2.1          ^^I
v0.32.0           ^^IOpenStack-Helm Ingress Controller
<output_omitted>
https://artifacthub.io/packages/helm/api/ingres...^^I3.29.1         ^^I
0.45.0            ^^IIngress controller for Kubernetes using NGINX a...
https://artifacthub.io/packages/helm/wener/ingr...^^I3.31.0         ^^I
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
0.46.0            ^^IIngress controller for Kubernetes using NGINX a...
https://artifacthub.io/packages/helm/nginx/ngin...^^I0.9.2            ^^I
1.11.2            ^^INGINX Ingress Controller
<output_omitted>
```

12. ここでは **NGINX** が提供する人気の Ingress コントローラーを使用します。

```
student@cp:~$ helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
```

```
"ingress-nginx" has been added to your repositories
```

```
student@cp:~$ helm repo update
```

```
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "ingress-nginx" chart repository
Update Complete. -Happy Helming!-
```

13. values.yamlファイルをダウンロードして編集し、Deployment の代わりに DaemonSet を使用するように変更します。こうすることで、外部のロードバランサーを使用する場合にトラフィックを処理するための Pod が各ノード上に存在することになります。

```
student@cp:~$ helm fetch ingress-nginx/ingress-nginx --untar
```

```
student@cp:~$ cd ingress-nginx
```

```
student@cp:~/ingress-nginx$ ls
```

```
CHANGELOG.md  Chart.yaml  OWNERS  README.md  ci  templates  values.yaml
```

```
student@cp:~/ingress-nginx$ vim values.yaml
```

values.yaml

```
....
  ## DaemonSet or Deployment
  ##
  kind: DaemonSet                        #<-- Change to DaemonSet, around line 150

  ## Annotations to be added to the controller Deployment or DaemonSet
....
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



14. 次に、チャートを使ってコントローラーをインストールします。ドット (.) を使ってカレントディレクトリを検索していることに注意してください。

```
student@cp:~/ingress-nginx$ helm install myingress .
```

```
NAME: myingress
LAST DEPLOYED: Wed May 19 22:24:27 2021
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
The ingress-nginx controller has been installed.
It may take a few minutes for the LoadBalancer IP to be available.
You can watch the status by running
'kubectl --namespace default get services -o wide -w myingress-ingress-nginx-controller'

An example Ingress that makes use of the controller:
<output_omitted>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


15. 現在、Ingress コントローラーが稼働していますが、ルールはまだありません。存在するリソースを表示します。**-w** オプションを使用して、Ingress コントローラー Service が表示されるのを確認します。利用可能になったら、**ctrl-c** で終了し、次のコマンドに進みます。

```
student@cp:~$ kubectl get ingress --all-namespaces
```

```
No resources found
```

```
student@cp:~$ kubectl --namespace default get services -o wide -w myingress-ingress-nginx-controller
```

```
NAME                                  TYPE           CLUSTER-IP      EXTERNAL-IP
   PORT(S)                      AGE   SELECTOR
myingress-ingress-nginx-controller   LoadBalancer   10.104.227.79   <pending>
   80:32558/TCP,443:30219/TCP   47s   app.kubernetes.io/component=controller,
   app.kubernetes.io/instance=myingress,app.kubernetes.io/name=ingress-nginx
```

```
student@cp:~$ kubectl get pod --all-namespaces |grep nginx
```

```
default      myingress-ingress-nginx-controller-mrqt5    1/1     Running   0      20s
default      myingress-ingress-nginx-controller-pkdxm    1/1     Running   0      62s
default      nginx-b68dd9f75-h6ww7                       1/1     Running   0      21h
```

16. これで、HTTP ヘッダーにマッチするルールを Service に追加できます。tarball に含まれる course ファイルを確認することを忘れないでください。

```
student@cp:~$ vim ingress.yaml
```

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: ingress-test
  annotations:
    nginx.ingress.kubernetes.io/service-upstream: "true"
  namespace: default
spec:
  ingressClassName: nginx
  rules:
  - host: www.example.com
    http:
      paths:
      - backend:
          service:
            name: secondapp
            port:
              number: 80
        path: /
        pathType: ImplementationSpecific
```

17. 作成後、Ingress が動作していることを確認します。一致するヘッダーを渡さない場合は、404 エラーが発生するはずです。

```
student@cp:~$ kubectl create -f ingress.yaml
```

```
ingress.networking.k8s.io/ingress-test created
```

```
student@cp:~$ kubectl get ingress
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
NAME           CLASS    HOSTS              ADDRESS   PORTS   AGE
ingress-test   nginx    www.example.com              80      5s
```

```
student@cp:~$ kubectl get pod -o wide |grep myingress
```

```
myingress-ingress-nginx-controller-mrqt5   1/1     Running   0          8m9s    192.168.219.118
  cp   <none>           <none>
myingress-ingress-nginx-controller-pkdxm   1/1     Running   0          8m9s    192.168.219.118
  cp   <none>           <none>
```

```
student@cp:~/ingress-nginx$ curl 192.168.219.118
```

```
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

18. Ingress コントローラーの Service を確認し、その IP アドレスにアクセスした場合 404 エラーが発生することを再度確認します。アドミッションコントローラーにはアクセスしないでください。

```
student@cp:~/ingress-nginx$ kubectl get svc |grep ingress
```

```
myingress-ingress-nginx-controller             LoadBalancer   10.104.227.79    <pending>
  80:32558/TCP,443:30219/TCP   10m
myingress-ingress-nginx-controller-admission   ClusterIP      10.97.132.127    <none>
  443/TCP                      10m
```

```
student@cp:~/ingress-nginx$ curl 10.104.227.79
```

```
<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx</center>
</body>
</html>
```

19. ここで、一致するヘッダーを渡すと、デフォルトのウェブサーバーページが表示されるはずです。

```
student@cp:~/ingress-nginx$ curl -H "Host: www.example.com" http://10.104.227.79
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
<output_omitted>
```

20. Linkerd 用の ingress pods に Annotation を追加することができます。いくつか警告が出ますが、コマンドは動作します。

```
student@cp:~/ingress-nginx$ kubectl get ds myingress-ingress-nginx-controller -o yaml  | \
    linkerd inject --ingress - | kubectl apply -f -
```

```
daemonset "myingress-ingress-nginx-controller" injected

Warning: resource daemonsets/myingress-ingress-nginx-controller is missing the
kubectl.kubernetes.io/last-applied-configuration annotation which is required
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
by kubectl apply. kubectl apply should only be used on resources created
declaratively by either kubectl create --save-config or kubectl apply. The
missing annotation will be patched automatically.
daemonset.apps/myingress-ingress-nginx-controller configured
```

21. Top ページに移動し、Namespace を default に変更します。更に Resource を `daemonset/myingress-ingress-nginx-controller` に変更します。start に押して、さらに多くのトラフィックを Ingress コントローラーに渡します。GUI でトラフィックメトリクスを表示します。次のステップで追加された別のページを見ることができるように、top を実行させてください。

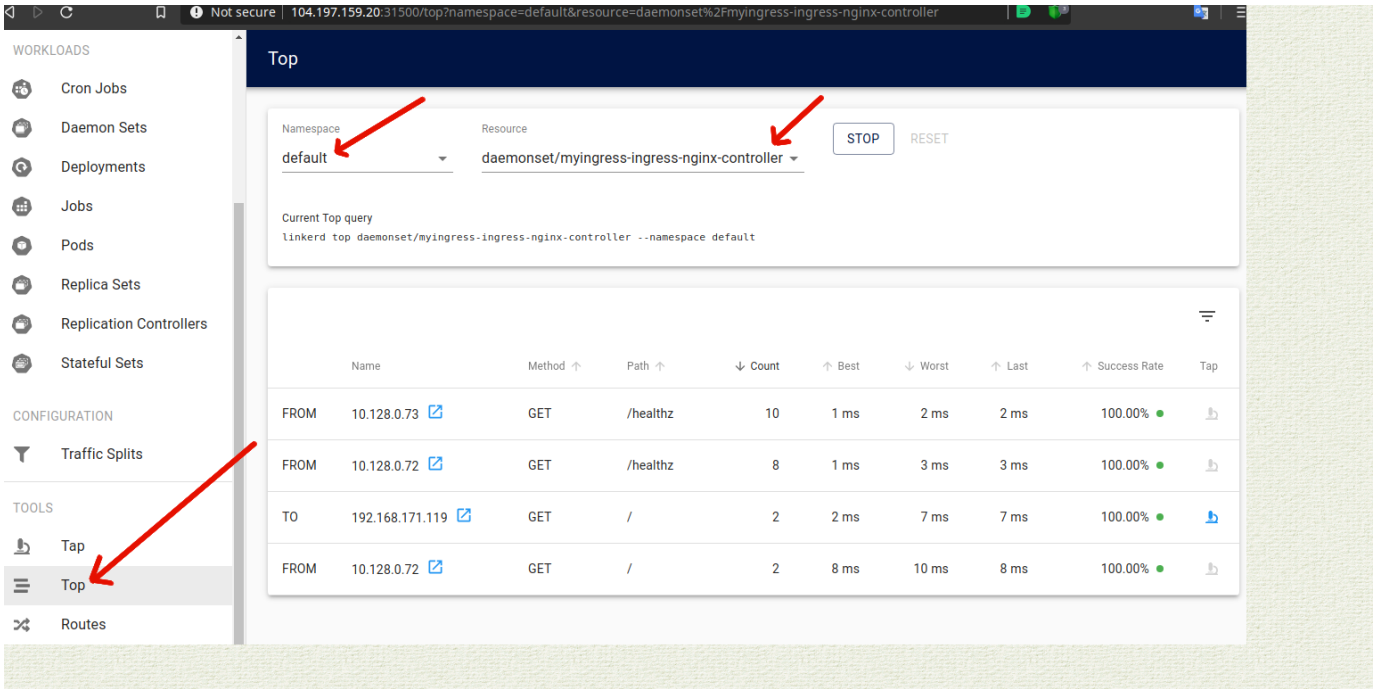


Figure 7.4: **Ingress Traffic**

22. この時点で、さらに多くのウェブサーバーを追加していくことになります。もう 1 つ設定し、それを何度でも繰り返すことができます。まずは、別の **nginx** サーバーをデプロイすることから始めます。Label を付けて、ポート80 を公開します。

```
student@cp:~/app2$ kubectl create deployment thirdpage --image=nginx
```

```
deployment.apps "thirdpage" created
```

23. Ingress コントローラーが照合するための Label を割り当てます。Pod 名は一意であり、**Tab** キーで補完することができます。

```
student@cp:~/app2$ kubectl label pod thirdpage-<tab> example=third
```

24. 新しいサーバーを NodePort として公開します。

```
student@cp:~/app2$ kubectl expose deployment thirdpage --port=80 --type=NodePort
```

```
service/thirdpage exposed
```

25. では、インストールをカスタマイズしていきます。新しい Pod の内部で bash シェルを実行します。Pod 名の末尾は異なるものになります。コンテナ内に **vim** またはエディターをインストールし、nginx の`index.html`ファイルを編集して、ウェブページのタイトルが`Third Page` になるようにします。以下の例では、コマンドの出力の多くが示されていないことに注意してください。

```
student@cp:~/app2$ kubectl exec -it thirdpage-<Tab> -- /bin/bash
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



**On Container**

```
root@thirdpage-:/# apt-get update

root@thirdpage-:/# apt-get install vim -y

root@thirdpage-:/# vim /usr/share/nginx/html/index.html
```

```
<!DOCTYPE html>
<html>
<head>
<title>Third Page</title>     #<-- Edit this line
<style>
<output_omitted>
```

```
root@thirdpage-:/$ exit
```

26. thirdpage Service を指すように、Ingress ルールを編集します。既存の`host` 節をコピーして、`host` と`name` 部分を編集するのが最も簡単かもしれません。

```
student@cp:~/app2$ kubectl edit ingress ingress-test
```

**ingress-test**

```yaml
....
spec:
  rules:
  - host: thirdpage.org
    http:
      paths:
      - backend:
          service:
            name: thirdpage
            port:
              number: 80
        path: /
        pathType: ImplementationSpecific
  - host: www.example.com
    http:
      paths:
      - backend:
          service:
            name: secondapp
            port:
              number: 80
        path: /
        pathType: ImplementationSpecific
status:
....
```

27. リモートシステムと同様に、ローカルで **curl** を使用して 2 番目の`Host:` 設定をテストします。`<title>` がデフォルトではないページを表示することを確認してください。どちらかのノードの IP を使用します。Linkerd GUI に新しい`TO` ラインが表示されるはずです。矢印のついた小さな青いボックスを選択すると、トラフィックが thirdpage に流れていることがわかります。

```
student@cp:~/app2$ curl -H "Host: thirdpage.org" http://10.128.0.7/
```

```
<!DOCTYPE html>
<html>
<head>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
<title>Third Page</title>
<style>
<output_omitted>
```

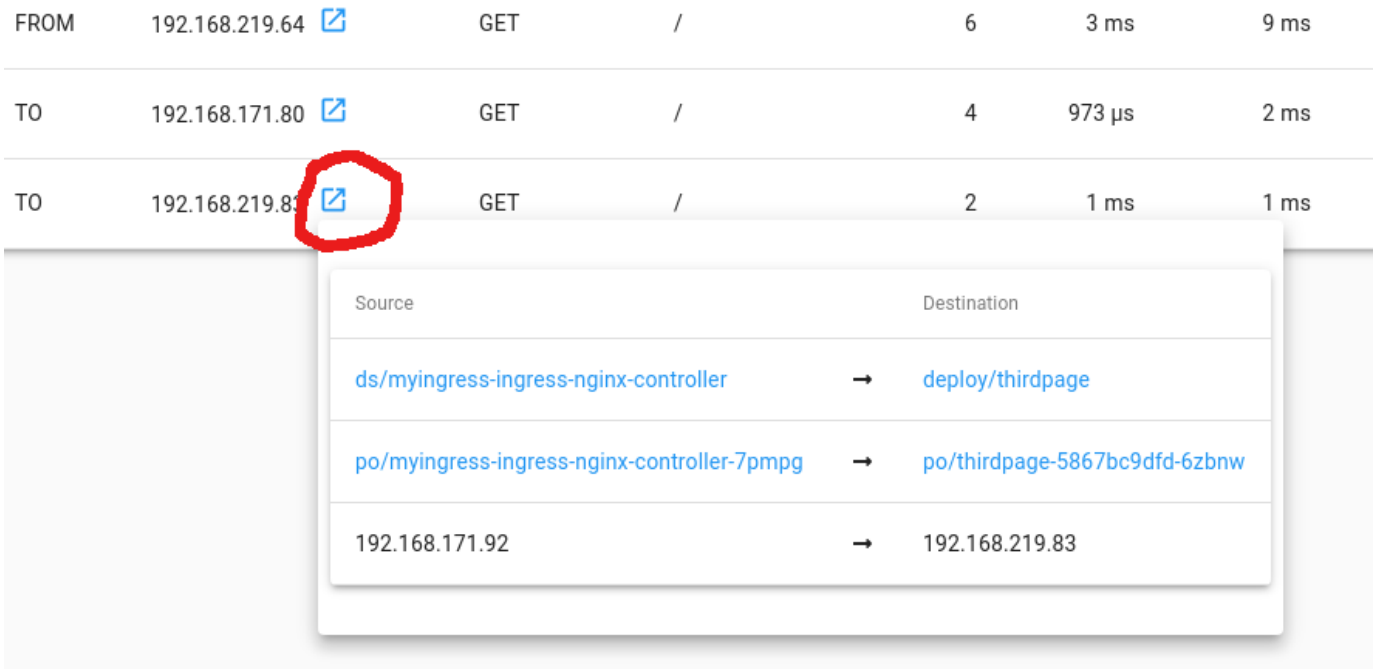


Figure 7.5: **Linkerd Top Metrics**

28. Service セレクタを編集するときに行ったように、Ingress ルールを編集して別のサーバーを指すようにする方法を検討します。**httpd** サーバーを再度デプロイし、Ingress ルールの編集だけで `thirdpage` から新しい`httpd` サーバーへのトラフィックを変更するテストを行います。

## 課題 7.3: 出題範囲の確認

**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

カリキュラム概要にある CKAD の出題範囲リストを再確認し、この章で取り上げたトピックの参考にしてください。

- Kubernetes のプリミティブを使用して、一般的なデプロイメント戦略を実装する（blue/gren や canary など）。
- Helm パッケージマネージャーを使用して、既存のパッケージをデプロイする。
- Service を介したアプリケーションへのアクセスの提供およびトラブルシューティングを行う。
- Ingress ルールによってアプリケーションを公開する。

まず、必要なファイルやプロセスがすべて理解できているかどうかを確認することに重点を置いてください。各ステップを素早く完了し、各オブジェクトが正しく動作していることを確認できるまで、復習を繰り返します。

試験で許可された 3 つの URL ロケーションを使用して、以下のことを行うための YAML の例を探し、熟知しておいてください。

1. `webone` という新しい Pod を作成し、`nginx` Service を稼働させます。その際、ポート 80 を公開します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



2. `webone-svc` という名前の新しい Service を作成します。この Service は、クラスターの外からアクセスできるようにする必要があります。
3. Service IP へのトラフィックが Web サーバーのコンテンツを表示するように、Pod と Service の両方をセレクタで更新します。
4. Service の種類を変更し、クラスター内からのみアクセスできるようにします。外部からアクセスできなくなったこと、ノード内からはアクセスできることをテストします。
5. `webtwo` という名前の別の Pod をデプロイし、今度は`wlniao/website` イメージを実行します。クラスター内からのリクエストのみが機能するようにした`webtwo-svc` という名前の別の Service を作成します。各サーバーのデフォルトページが異なることに注意してください。
6. webone.comへのリクエストには`nginx` のデフォルトページが表示され、webtwo.orgへのリクエストには`wlniao/website` のデフォルトページが表示されるように、Ingress コントローラーをインストールして設定します。
7. この出題範囲の確認で作成されたリソースを削除します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE

# 第 8 章

# アプリケーションのトラブルシューティング

## 課題 8.1: トラブルシューティング: アプリケーションのモニタリング

**概要**

マルチノード、非結合、一時的な環境では、トラブルシューティングが困難な場合があります。さらに、変化のスピードが速いとさらに難しくなります。特定のエラーとその修正に集中してその対処方法を記憶するのではなく、変化のペースが遅くなり、さまざまな分野がさらに成熟するまでは、トラブルシューティングの流れを学び、仮定を見直す方がより有効かもしれません。

1. secondapp の Pod を表示すると、Running と表示されているはずです。これは、中のアプリケーションが正常に動作していることを意味するのではなく、Pod が動作していることを意味します。RESTARTS は、実行するために記述したコマンドによるものです。コマンド実行が完了したときにも Pod は存在し、コントローラーは Pod 内部に別のコンテナを再起動します。RESTARTS の数は、演習の実行時間によって異なります。

```
student@cp/app2:~$ cd
student@cp:~$ kubectl get pods secondapp
```

```
NAME                        READY     STATUS      RESTARTS   AGE
secondapp                   2/2       Running     49         2d
```

2. Pod をよく見てください。kubectl describe pod の出力結果の各行をチェックするように着実に作業します。問題がある場合、同じノードやボリュームで他の Pod が問題を起こしていないか?などを確認します。さらに各コンテナの状態を確認します。busy とwebserver の両方が Running と出力されるはずです。webserver の Restart Count は 0、busy の Restart Count は 49 であることに注意してください。この場合、Pod は 49 時間稼働しているため、このような状態になることが予想されます。

```
student@cp:~$ kubectl describe pod secondapp
```

```
Name:         secondapp
Namespace:    default
Node:         worker-wdrq/10.128.0.2
Start Time:   Fri, 13 Apr 2022 20:34:56 +0000
Labels:       example=second
Annotations:  <none>
Status:       Running
IP:           192.168.55.91
Containers:
  webserver:
<output_omitted>
    State:          Running
```

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
      Started:      Fri, 13 Apr 2022 20:34:58 +0000
    Ready:          True
    Restart Count:  0
<output_omitted>

  busy:
<output_omitted>

    State:          Running
      Started:      Sun, 15 Apr 2022 21:36:20 +0000
    Last State:     Terminated
      Reason:       Completed
      Exit Code:    0
      Started:      Sun, 15 Apr 2022 20:36:18 +0000
      Finished:     Sun, 15 Apr 2022 21:36:18 +0000
    Ready:          True
    Restart Count:  49
    Environment:    <none>
```

3. Pod の状態には 3 つの値があります。Pod は Initialized、Ready そして PodScheduled を報告することを確認します。

```
<output_omitted>
Conditions:
  Type           Status
  Initialized    True
  Ready          True
  PodScheduled   True
<output_omitted>
```

4. エラーや警告が表示されるイベントがあるかどうかを確認し、問題の原因を特定します。

```
Events:
  Type    Reason   Age                From                   Message
  ----    ------   ----               ----                   -------
  Normal  Pulling  34m (x50 over 2d)  kubelet, worker-wdrq  pulling
image "busybox"
  Normal  Pulled   34m (x50 over 2d)  kubelet, worker-wdrq  Successfully
pulled image "busybox"
  Normal  Created  34m (x50 over 2d)  kubelet, worker-wdrq  Created
container
  Normal  Started  34m (x50 over 2d)  kubelet, worker-wdrq  Started
container
```

5. 各コンテナのログを確認します。ログ出力からエラーに関連する部分をふるいにかける必要があるかもしれません。コンテナによっては、busy に見られるように、全く出力がない場合もあります。

```
student@cp:~$ kubectl logs secondapp webserver
```

```
192.168.55.0 - - [13/Apr/2022:21:18:13 +0000] "GET / HTTP/1.1" 200
612 "-" "curl/7.47.0" "-"
192.168.55.0 - - [13/Apr/2022:21:20:35 +0000] "GET / HTTP/1.1" 200
612 "-" "curl/7.53.1" "-"
127.0.0.1 - - [13/Apr/2022:21:25:29 +0000] "GET" 400 174 "-" "-" "-"
127.0.0.1 - - [13/Apr/2022:21:26:19 +0000] "GET index.html" 400 174
"-" "-" "-"
<output_omitted>
```

```
student@cp:~$ kubectl logs secondapp busy
```

```
student@cp:~$
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



6. コンテナが DNS を使用し、外部と通信できることを確認します。secondapp の UID をまだ UID **2000** に制限していることを忘れないでください。これにより、一部のコマンドの実行が妨げられる可能性があります。また、アプリケーションが期待したタスクを完了できないなどのエラーも発生する可能性があります。

```
student@cp:~$ kubectl exec  -it  secondapp -c busy -- sh
```

**On Container**

```
/ $ nslookup www.linuxfoundation.org
```

```
/ $ nslookup www.linuxfoundation.org
Server:^^I^^I10.96.0.10
Address:^^I10.96.0.10:53

Non-authoritative answer:
Name:^^Iwww.linuxfoundation.org
Address: 23.185.0.2

*** Can't find www.linuxfoundation.org: No answer
```

```
/ $ cat /etc/resolv.conf
```

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local
cluster.local c.endless-station-188822.internal
google.internal
options ndots:5
```

7. **nc (NetCat)** を使って、リモートノードへのアクセスをテストします。**nc** にはいくつかのオプションがあり、問題がローカルノードにあるのか、ノード間にあるのか、ターゲットにあるのかをトラブルシュートするのに役立ちます。以下の例では接続が完了せず、**control-c** で中断しています。

```
/ $ nc www.linux.com 25
```

```
^Cpunt!
```

8. 問題を名前解決に絞り込むために、IP アドレスを使用してテストします。この場合、使用する IP は、Google の DNS サーバーでよく知られている IP です。次の例は、インターネットの名前解決は機能しているが、UID の問題で `index.html` ファイルへのアクセスができないことを示しています。

```
/ $ wget http://www.linux.com/
```

```
Connecting to www.linux.com (151.101.45.5:80)
Connecting to www.linux.com (151.101.45.5:443)
wget: can't open 'index.html': Permission denied

/ $ exit
```

9. トラフィックが正しい Pod に送信されていることを確認します。Service と Endpoint の両方の詳細を確認してください。単純なタイプミスがあると、トラフィックが適切な Pod に到達しないことがあるので、使用中のポートに細心の注意を払いましょう。Label や Selector にも誤字がないことを確認してください。

```
student@cp:~$ kubectl get svc
```

```
NAME         TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
kubernetes   ClusterIP   10.96.0.1        <none>        443/TCP    10d
nginx        ClusterIP   10.108.95.67     <none>        443/TCP    10d
registry     ClusterIP   10.105.119.236   <none>        5000/TCP   10d
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
secondapp    LoadBalancer   10.109.26.21     <pending>     80:32000/TCP   1d
thirdpage    NodePort       10.109.250.78    <none>        80:31230/TCP   1h
```

```
student@cp:~$ kubectl get svc secondapp -o yaml
```

```
<output_omitted>
  clusterIP: 10.109.26.21
  externalTrafficPolicy: Cluster
  ports:
  - nodePort: 32000
    port: 80
    protocol: TCP
    targetPort: 80
  selector:
    example: second
<output_omitted>
```

10. Service の Endpoint が存在し、Namespace、ポート、プロトコルなどが期待される値があることを確認します。

```
student@cp:~$ kubectl get ep
```

```
NAME         ENDPOINTS            AGE
kubernetes   10.128.0.3:6443      10d
nginx        192.168.55.68:443    10d
registry     192.168.55.69:5000   10d
secondapp    192.168.55.91:80     1d
thirdpage    192.168.241.57:80    1h
```

```
student@cp:~$ kubectl get ep secondapp -o yaml
```

```
apiVersion: v1
kind: Endpoints
metadata:
  creationTimestamp: 2022-04-14T05:37:32Z
<output_omitted>
```

11. コンテナ、Service、Endpoint が動作している場合、問題は **kube-proxy** のようなインフラストラクチャサービスにある可能性があります。インフラストラクチャサービスが実行されていることを確認し、ログにエラーがないか確認してください。ノードが 2 つあるので、2 つのプロキシを確認することになります。**kubeadm** でクラスターを構築したため、プロキシはコンテナとして実行されます。他のシステムでは、**journalctl** を使用するか、`/var/log/kube-proxy.log`を参照する必要があるかもしれません。

```
student@cp:~$ ps -elf |grep kube-proxy
```

```
<output_omitted>
4 S root      2652  2621  0  80   0 - 186908 -     15:23 ?        00:00:03 /usr/local/bin/kube-proxy
--config=/var/lib/kube-proxy/config.conf --hostname-override=cp
0 S student  31656  9707  0  80   0 -  3715 pipe_w 19:02 pts/0    00:00:00 grep --color=auto
↪  kube-proxy
```

```
student@cp:~$ journalctl -a | grep proxy
```

```
Apr 15 15:44:43 worker-nzjr audit[742]: AVC apparmor="STATUS"
 operation="profile_load" profile="unconfined" \
  name="/usr/lib/lxd/lxd-bridge-proxy" pid=742 comm="apparmor_parser"
Apr 15 15:44:43 worker-nzjr kernel: audit: type=1400
 audit(1523807083.011:11): apparmor="STATUS" \
  operation="profile_load" profile="unconfined" \
   name="/usr/lib/lxd/lxd-bridge-proxy" pid=742 comm="apparmor_parser"
Apr 15 15:45:17 worker-nzjr kubelet[1248]: I0415 15:45:17.153670
 1248 reconciler.go:217] operationExecutor.VerifyControllerAttachedVolume\
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
    started for volume "xtables-lock" \
     (UniqueName: "kubernetes.io/host-path/e701fc01-38f3-11e8-a142-\
     42010a800003-xtables-lock") \
     pod "kube-proxy-t8k4w" (UID: "e701fc01-38f3-11e8-a142-42010a800003")
```

12. プロキシログを 2 つとも見てください。**I** で始まる行は `info`、E は `errors` です。この例では、最後のメッセージに Endpoint のリストへの参照アクセスが RBAC によって拒否されたと書かれています。これは、Helm 経由のデフォルトインストールが RBAC を認識していなかったためです。これはあくまで例であり、演習のノードでエラーが表示されることはないでしょう。

    コマンドライン補完を使用しない場合は、最初に可能な Pod 名を表示します。

```
student@cp:~$ kubectl -n kube-system get pod
```

```
student@cp:~$ kubectl -n kube-system logs kube-proxy-fsdfr
```

```
I0405 17:28:37.091224       1 feature_gate.go:190] feature gates: map[]
W0405 17:28:37.100565       1 server_others.go:289] Flag proxy-mode=""
unknown, assuming iptables proxy
I0405 17:28:37.101846       1 server_others.go:138] Using iptables Proxier.
I0405 17:28:37.121601       1 server_others.go:171] Tearing down
inactive rules.
<output_omitted>
E0415 15:45:17.086081       1 reflector.go:205] \
  k8s.io/kubernetes/pkg/client/informers/informers_generated/
  internalversion/factory.go:85: \
  Failed to list *core.Endpoints: endpoints is forbidden: \
    User "system:serviceaccount:kube-system:kube-proxy" cannot \
   list endpoints at the cluster scope:\
 [clusterrole.rbac.authorization.k8s.io "system:node-proxier" not found, \
   clusterrole.rbac.authorization.k8s.io "system:basic-user" not found,
 clusterrole.rbac.authorization.k8s.io \
 "system:discovery" not found]
```

13. プロキシが問題の Service に対して期待されるルールを作成していることを確認します。Service に使用されている宛先ポート（この例では **32000**）を確認します。

```
student@cp:~$ sudo iptables-save |grep secondapp
```

```
-A KUBE-NODEPORTS -p tcp -m comment --comment "default/secondapp:" \
-m tcp --dport 32000 -j KUBE-MARK-MASQ
-A KUBE-NODEPORTS -p tcp -m comment --comment "default/secondapp:" \
-m tcp --dport 32000 -j KUBE-SVC-DAASHM5XQZF5XI3E
-A KUBE-SERVICES ! -s 192.168.0.0/16 -d 10.109.26.21/32 -p tcp \
-m comment --comment "default/secondapp: \
      cluster IP" -m tcp --dport 80 -j KUBE-MARK-MASQ
-A KUBE-SERVICES -d 10.109.26.21/32 -p tcp -m comment --comment \
"default/secondapp: cluster IP" -m tcp \
      --dport 80 -j KUBE-SVC-DAASHM5XQZF5XI3E
<output_omitted>
```

14. **iptables** がターゲットにしているポートを確認することでプロキシが機能していることを確認します。もし失敗した場合は、2 つ目のターミナルを開き、リクエストを行ったときのプロキシログをそのまま表示します。

```
student@cp:~$ curl localhost:32000
```

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<output_omitted>
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


## 課題 8.2: YAML ファイルの更新

API は、オブジェクトが成熟し、新しい機能や設定が追加されるにつれて、変化し続けています。その結果、以前のバージョンの Kubernetes で動作していた YAML は、API の廃止に伴い更新が必要になることがあります。

1. コースの tarball から`brokendeploy.yaml`ファイルを探します。Kubernetes のいくつかのリリースの前に動作した、Deployment を作成してみてください。現在のオブジェクトが必要とするものを理解するために、コマンドラインから新しいデプロイメントを作成します。デプロイして Pod が実行できるようになるまで、壊れた Deployment の YAML を編集します。

```
student@cp:~$ find $HOME -name brokendeploy.yaml

student@cp:~$ cp <path_from_above> .

student@cp:~$ kubectl create -f brokendeploy.yaml
```

## 課題 8.3: オプション演習: Conformance Testing

**cncf.io** グループは、適合する Kubernetes クラスターとみなされるものを正式化する過程にあります。このプロジェクトが成熟する一方、**Sonobuoy** https://sonobuoy.io/が提供する便利な既存ツールがあります。このツールを動作させるためには、より新しいバージョンの **Golang** がインストールされていることを確認する必要があります。github からコードをダウンロードし、git か go で（最も慣れ親しんだツールのどちらか）で確認できます。物事はすぐに変わるので、これらの手順はうまくいかないかもしれません。

1. コンパイルされたバイナリをダウンロードします。リンクが変わっている必要がある場合に備えて、短い URL が最初に表示され、次に長い URL が表示されます。同じファイルをダウンロードする必要があります。

```
student@cp:~$ curl -sLO https://tinyurl.com/4cdeyz55

student@cp:~$ mv 4cdeyz55 sonobuoy.tar.gz

student@cp:~$ tar -xvf sonobuoy.tar.gz
```

```
LICENSE
sonobuoy
```

```
student@cp:~$ curl -sLO \
https://github.com/vmware-tanzu/sonobuoy/releases/download/v0.56.8/sonobuoy_0.56.8_linux_amd64.tar.gz
```

2. テストを実行します。`--wait` オプションを使用すると、テストが終了するまで画面をキャプチャすることができます。テストが終了するまでに 1 時間程度かかる可能性があります。テストオブジェクトが作成されたことを示す出力が得られるはずです。

```
student@cp:~$ sudo mv sonobuoy /usr/local/bin/

student@cp:~$ sonobuoy run --wait
```

```
INFO[0000] create request issued                name=sonobuoy namespace=
resource=namespaces
INFO[0000] create request issued                name=sonobuoy-serviceaccount
namespace=sonobuoy resource=serviceaccounts
INFO[0000] create request issued                name=sonobuoy-serviceaccount
-sonobuoy namespace= resource=clusterrolebindings
INFO[0000] create request issued                name=sonobuoy-serviceaccount
-sonobuoy namespace= resource=clusterroles
INFO[0000] create request issued                name=sonobuoy-config-cm
namespace=sonobuoy resource=configmaps
INFO[0000] create request issued                name=sonobuoy-plugins-cm
namespace=sonobuoy resource=configmaps
INFO[0000] create request issued                name=sonobuoy namespace=
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE



```
sonobuoy resource=pods
INFO[0000] create request issued                        name=sonobuoy-aggregator
namespace=sonobuoy resource=services
14:45:09          PLUGIN       NODE     STATUS   RESULT                                PROGRESS
14:45:09             e2e     global    running            Passed:  0, Failed:  0, Remaining:354
14:45:09    systemd-logs       cp-1   complete
14:45:09    systemd-logs   worker-1   complete
14:45:09
14:45:09 Sonobuoy is still running. Runs can take 60 minutes or more depending
on cluster and plugin configuration.

<output_omitted>

16:38:29             e2e     global    running            Passed:345, Failed:  3, Remaining:  6
16:38:49             e2e     global   complete            Passed:351, Failed:  3, Remaining:  0
16:38:49 Sonobuoy plugins have completed. Preparing results for download.
16:39:09             e2e     global   complete   failed   Passed:351, Failed:  3, Remaining:  0
16:39:09    systemd-logs       cp-1   complete   passed
16:39:09    systemd-logs   worker-1   complete   passed
16:39:09 Sonobuoy has completed. Use `sonobuoy retrieve` to get results.
```

3. テストが完全に完了するまで待ちたくない場合は、2 番目のターミナルセッションを開き、cp ノードに接続し、**status** と **logs** のサブコマンドを使用してください。

```
student@cp:~$ sonobuoy status
```

```
      PLUGIN  STATUS      RESULT   COUNT
PROGRESS
       e2e    running     1   Passed: 31, Failed:  0, Remaining:323
systemd-logs  complete    2

Sonobuoy is still running. Runs can take 60 minutes or more depending on
cluster and plugin configuration.
```

```
student@cp:~$ sonobuoy logs
```

```
namespace="sonobuoy" pod="sonobuoy-systemd-logs-daemon-set-34bef90c2026439b-x7q7g"
container="sonobuoy-worker"
time="2022-07-23T14:44:51Z" level=trace msg="Invoked command single-node with args
[] and flags [level=trace logtostderr=true sleep=-1 v=6]"
time="2022-07-23T14:44:51Z" level=info msg="Waiting for waitfile"
waitfile=/tmp/sonobuoy/results/done
time="2022-07-23T14:44:51Z" level=info msg="Starting to listen on port 8099 for
progress updates and will relay them to
https://[192.168.193.162]:8080/api/v1/progress/by-node/cp-1/systemd-logs"
....
```

4. テスト結果が出るまで待ちます。もしくはコンテナにログインしてテスト結果を探します。

```
student@cp:~$ sonobuoy retrieve
```

```
202207231444_sonobuoy_7d9c5bb5-e547-4827-bb2e-07e12f8fed25.tar.gz
```

5. テスト結果を確認します。

```
student@cp:~$ sonobuoy results 202207231444_sonobuoy_7d9c5bb5-e547-4827-bb2e-07e12f8fed25.tar.gz
```

```
Plugin: e2e
Status: failed
Total: 6971
Passed: 351
```


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)

© The Linux Foundation 2023: DO NOT COPY OR DISTRIBUTE


```
Failed: 3
Skipped: 6617

Failed tests:
[sig-scheduling] SchedulerPredicates [Serial] validates that NodeSelector is respected if not matching
↪  [Conformance]
<output_omitted>
```

6. Sonobuoy が作成した Pod と Namespace を削除します。

```
student@cp:~$ sonobuoy delete --wait
```

## 課題 8.4: 出題範囲の確認

**重要**

本確認に含まれる元となるページや内容は、常に変更される可能性があります。最新の情報をあなたの責任を以て確認してください。

カリキュラム概要にある CKAD の出題範囲リストを再確認し、この章で取り上げたトピックの参考にしてください。

- API の非推奨の理解
- 提供されたツールを使った Kubernetes アプリケーションの監視
- Kubernetes でのデバッグ

まず、必要なファイルやプロセスをすべて理解することに集中しましょう。必要な YAML サンプルのブックマークがあり、各ステップを素早く完了し、各オブジェクトが適切に実行されていることを確認できるまで、レビューを繰り返します。

1. `troubleshoot-review1.yaml`ファイルを検索して、Deployment を作成するために使用します。**create** コマンドは失敗します。ファイルを編集して、単一の Pod が問題なく少なくとも 1 分間実行されるように問題を修正します。修正すべき点はいくつかあります。

```
student@cp:~$ kubectl create -f troubleshoot-review1.yaml
```

```
<Fix any errors found here>
```

修正すると、このようになります。

```
student@cp:~$ kubectl get deploy igottrouble
```

```
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
igottrouble   1/1     1            1           5m13s
```

2. このレビュー中に作成されたリソースを削除します。


© Copyright The Linux Foundation 2023. All rights reserved.(無断転載・無断複製禁止)